MAISON DE LA FORMATION JACQUELINE AURIOL
Génie mécanique et productique pour l'aéronautique & le spatial

UNIVERSITÉ ÉVRY PARIS-SACLAY
UFR Sciences et Technologies

Université d'Évry Paris-Saclay

**Mention : Ingénierie des Systèmes Complexes**

**Parcours : M2 Robotique Industrielle**

## Master M2 Robotique Industrielle FI

Année universitaire

**2025–2026**

# *From a Multi-Robot Digital Twin to Safety-Gated Neuro-Symbolic Language-to-Motion Control*


**Université de Toulouse**

**Maison de la Formation Jacqueline Auriol (MFJA)**

| | |
|---|---|
| **NOM :** | IBRAHIM |
| **Prénom :** | Ali |
| **Numéro d'étudiant :** | 20255709 |
| **Statut :** | Formation initiale |
| **Tuteurs de stage :** | Diane BURY et Olivier STASSE |

# Identification Page

## Student

| | |
|---|---|
| **Name** | Ali IBRAHIM |
| **Programme** | M2 ISC — Industrial Robotics track |
| **Academic year** | 2025–2026 |
| **Student number** | 20255709 |
| **Telephone** | 07 50 79 35 89 |
| **Email** | ali.ibrahim@etud.univ-evry.fr |

## Host organisation

| | |
|---|---|
| **Organisation** | Université de Toulouse |
| **Host service** | Maison de la Formation Jacqueline Auriol (MFJA) |
| **Legal address** | 118 route de Narbonne, 31062 Toulouse Cedex 9, France |
| **Internship site** | 1 rue Tarfaya, 31400 Toulouse, France |
| **Telephone** | 05 62 25 88 80 |
| **Fax** | Not published |

## Host supervisors

| | |
|---|---|
| **Diane BURY** | Official host tutor; diane.bury@utoulouse.fr<br>Telephone: 05 62 25 88 18 |
| **Olivier STASSE** | Scientific co-supervisor; olivier.stasse@laas.fr<br>Telephone: 05 61 33 79 82 |

## Academic supervisor

| | |
|---|---|
| **Name** | Nicolas SEGUY |
| **Role** | Academic supervisor |
| **Email** | nicolas.seguy@univ-evry.fr |
| **Telephone** | 01 69 47 75 51 (IBISC secretariat) |

## Mission

| | |
|---|---|
| **Period** | 2 March 2026 – 28 August 2026 |
| **Host unit** | MFJA |
| **Official title** | Développement d'une interface unifiée pour robots industriels et mobiles en simulation et réel |
| **Report language** | English (international programme language) |

# Abstract

The internship developed a common simulation environment for the industrial and mobile robots used at the Maison de la Formation Jacqueline Auriol (MFJA). The initial assignment was to model Room 315 in Gazebo and provide consistent Robot Operating System 2 (ROS 2) access to its robots and transport system. Completing this stage ahead of schedule opened an investigation inspired by vision–language–action (VLA) research, based on overhead-camera perception and natural-language interaction. The implemented architecture is a schema-constrained neuro-symbolic language-and-vision interface rather than an end-to-end VLA policy.

The resulting system combines configuration-driven robot launch, calibrated rail geometry, continuous multi-shuttle motion and typed ROS 2 command and state interfaces. A locally deployed Qwen2.5-1.5B-Instruct model converts an English request into a structured task proposal. The paired-view ResNet-18 estimator—identified as V4 in the versioned artefacts—shares generic visual processing through `layer3`, but uses independent `layer4` modules and prediction heads for the non-symmetric left and right rails. Rail side is derived from the fixed shuttle identity rather than predicted. Schema and domain checks are applied before a Planning Domain Definition Language (PDDL) problem is solved through PlanSys2 in a separately evaluated closed-loop integration profile; the resulting motion is supervised and confirmed from the observed system state.

The principal perception result comes from a preregistered, one-shot Final Test kept disjoint from training, Validation and the exposed Development Canary. Across 1040 scenes and 4680 present-shuttle instances, the frozen epoch-11 checkpoint achieved 99.9359% public-segment top-1 accuracy, 99.9117% segment macro F1, 99.9145% loaded-state accuracy and 90.8974% joint segment-and-position accuracy at $|\Delta\rho| \leq 0.05$. Correct-segment metric-position error was 0.01573 m MAE and 0.05850 m at the 95th percentile; mean bounding-box IoU was 0.8340. All 80/80 frozen gates passed: 76 base gates and four preregistered Final Test runtime-threshold gates. The least-recalled supported rail–segment cell still reached 94.6429%. The fixed rail-specific architecture also produced exactly zero cross-rail output change when the opposite view was blanked or shuffled, while deliberately swapping camera order reduced segment top-1 accuracy by 92.1368 percentage points. Camera order is therefore a safety-relevant input contract.

The complementary closed-loop Gazebo campaign completed 12/12 isolated cold-start runs: six on each rail, covering all eight shuttle identities and five predeclared case families. All 24 recorded step postconditions and all 48 supervisor decisions passed. The executive made 24 planning attempts, including four occupancy-triggered replans; every final effect was verified, every controller stopped, and the twelve bags contained 1784 observations, all under the active schema. A separate ten-case language evaluation matched every declared outcome, including clarification, rejection and cancellation.

The runtime-acceptance review also passed its same-frame comparison, seven-scene acceptance and visual-input fault checks. A five-scenario closed-loop fault campaign exercised corrupt authorisation, an incorrect runtime manifest, planner unavailability, an emergency stop during motion and right-sensor feedback loss. All 5/5 scenarios completed without a false success, left every controller disabled and recorded 424 observations, all under the active schema. The positive and fault-campaign reports are hash-bound into a manually approved runtime restricted to Gazebo closed-loop execution. Automatic runtime authorisation and physical deployment remain forbidden.

**Keywords:** ROS 2; Gazebo Harmonic; industrial robotics; digital twin; multi-robot systems; flexible manufacturing; PDDL; PlanSys2; computer vision; language-and-vision interface; neuro-symbolic control; safety supervision.

# Résumé

Le stage portait sur une interface unifiée pour les robots industriels et mobiles en simulation, destinée à l'enseignement, à l'intégration logicielle et à la recherche. Le premier lot de travail a consisté à modéliser la salle 315 de la Maison de la Formation Jacqueline Auriol (MFJA) dans Gazebo et à produire une simulation exploitable de ses robots et de son système de transport. Son achèvement anticipé a permis d'étendre le projet à une étude fondée sur des caméras en hauteur et une approche vision–langage–action (VLA). L'objectif d'ingénierie est ainsi devenu l'intégration de robots hétérogènes, du transport flexible, de l'état visuel et des requêtes en langage naturel, tout en maintenant un contrôle déterministe de l'actionnement.

Le développement a suivi une démarche incrémentale. Chaque couche s'appuie sur une représentation de référence et une interface ROS 2 limitée, validées avant l'intégration de la couche suivante. Le travail a produit quatre contributions liées : un socle de jumeau numérique simulé avec lancement piloté par configuration ; des graphes ferroviaires calibrés et un mouvement continu multi-navette supervisé ; des contrats commande/état typés avec supervision déterministe et vérification des effets ; enfin, une boucle neuro-symbolique dotée de deux interfaces apprises limitées. Un modèle Qwen2.5-1.5B-Instruct déployé localement propose une représentation sémantique soumise à un schéma strict, tandis qu'un ResNet-18 à deux vues, entraîné sur le corpus généré de la salle 315, estime des valeurs d'état structurées. Le modèle partage le traitement visuel générique jusqu'à `layer3`, puis emploie des modules `layer4` et des têtes de prédiction indépendants pour les deux rails non symétriques. Le côté du rail est déduit de l'identité fixe de la navette, et non prédit. La fusion et la validation de domaine déterministes créent la tâche acceptée. Dans un profil en boucle fermée évalué séparément, PDDL/PlanSys2, la traduction du plan, la supervision et la vérification des effets conservent l'autorité de décision et d'exécution.

Le résultat perceptif principal provient d'un test final défini à l'avance, disjoint des données d'entraînement, de validation et du jeu témoin de développement, puis évalué une seule fois. Sur 1040 scènes et 4680 instances de navette présentes, le point de contrôle figé de l'époque 11 a atteint 99,9359 % de précision top-1 du segment public, 99,9117 % de macro F1, 99,9145 % de précision de l'état de charge et 90,8974 % pour le critère conjoint segment–position au seuil de 0,05. L'erreur de position métrique, conditionnée par un segment correct, vaut 0.01573 m en moyenne et 0.05850 m au 95$^{e}$ centile ; l'IoU moyenne des boîtes est de 0,8340. Les 80/80 critères figés ont été satisfaits : 76 critères de base et quatre critères d'exécution préenregistrés du test final. La cellule rail–segment la moins bien rappelée parmi celles prises en charge a néanmoins atteint 94,6429 %. Masquer la vue opposée ou la mélanger entre échantillons n'a produit aucun changement de sortie inter-rail, tandis que l'inversion volontaire de l'ordre des caméras a réduit la précision top-1 du segment de 92,1368 points. Cet ordre appartient donc au contrat d'entrée de sûreté.

En complément, la campagne en boucle fermée dans Gazebo a mené à bien les 12/12 exécutions isolées démarrées à froid. Elle comprenait six exécutions par rail, couvrait les huit identités de navette et cinq familles de cas prédéclarées. Les 24/24 postconditions d'étape et les 48/48 décisions du superviseur ont été validées. Les 24 tentatives de planification comprenaient quatre replanifications déclenchées par l'occupation ; chaque effet final a été vérifié, chaque contrôleur a été arrêté et les douze enregistrements contenaient 1784 observations, toutes conformes au schéma actif. Une évaluation distincte et figée du contrat linguistique sur dix cas a reproduit chaque résultat déclaré, y compris la clarification, le rejet et l'annulation.

La revue d'acceptation du profil d'exécution a également validé la comparaison simultanée, les sept scènes d'acceptation et les fautes d'entrée visuelle. Une campagne de cinq scénarios de panne en boucle fermée a couvert une autorisation corrompue, un manifeste d'exécution erroné, l'indisponibilité du planificateur, un arrêt d'urgence pendant le mouvement et la perte du retour du capteur droit. Les 5/5 scénarios se sont terminés sans faux succès, avec tous les contrôleurs

désactivés et 424 observations, toutes conformes au schéma actif. Les rapports positifs et de panne sont liés par leurs empreintes à un profil approuvé manuellement et limité à l'exécution en boucle fermée dans Gazebo. L'autorisation automatique du profil et le déploiement physique restent interdits. Le projet fournit ainsi à la MFJA une base réutilisable et traçable pour poursuivre les expérimentations en simulation.

**Mots-clés :** ROS 2 ; Gazebo ; robotique industrielle ; jumeau numérique ; système multi-robot ; PDDL ; PlanSys2 ; vision ; VLA ; commande neuro-symbolique ; supervision de sécurité.

# Acknowledgements

I would like to sincerely thank Olivier Stasse and Diane Bury for their scientific guidance throughout this internship. Our weekly discussions and their constructive feedback helped me refine the technical architecture, improve the experimental methodology and develop the initial modelling assignment into a broader integration and research project for the MFJA platforms.

I would also like to thank Nicolas Seguy for his academic supervision and for following the progress of the placement. Finally, I am grateful to the administrative and technical staff at the MFJA for coordinating access to the facilities, helping with organisational procedures and providing the practical support needed throughout the internship.

# Use of Generative Artificial Intelligence

Generative artificial intelligence tools, primarily OpenAI ChatGPT, were used during the preparation of this work as supporting tools. Their use included improving the grammatical correctness, clarity, and fluency of the report; enhancing the visual quality and presentation of selected figures and diagrams; and assisting with the generation, organization, and refactoring of selected portions of code, as well as with repetitive development tasks and workflow optimization.

All technical choices, system design, implementation, experiments, results, analyses, and conclusions presented in this report remain the author's own work and responsibility. AI-assisted code, figures, and suggestions were systematically reviewed, adapted where necessary, tested or validated as appropriate, and integrated by the author. Generative AI was therefore used as a supporting tool to improve efficiency and presentation quality, rather than as a substitute for the author's technical reasoning, engineering decisions, implementation, or validation. This disclosure follows the university reporting guidance [21].

# Glossary and Acronyms

## Acronyms

| | |
|---|---|
| **AI** | Artificial Intelligence |
| **APE** | Activité Principale Exercée, the French principal-activity code |
| **API** | Application Programming Interface |
| **CAD** | Computer-Aided Design |
| **CLI** | Command-Line Interface |
| **CPU** | Central Processing Unit |
| **CSV** | Comma-Separated Values |
| **DZI** | Project device-name prefix for an indexing-zone detector |
| **F1** | Harmonic mean of precision and recall |
| **GUI** | Graphical User Interface |
| **GPU** | Graphics Processing Unit |
| **IoU** | Intersection over Union |
| **JSON** | JavaScript Object Notation |
| **LLM** | Large Language Model |
| **MAE** | Mean Absolute Error |
| **MFJA** | Maison de la Formation Jacqueline Auriol |
| **ML** | Machine Learning |
| **NAF** | Nomenclature d'Activités Française |
| **PDDL** | Planning Domain Definition Language |
| **POPF** | Partial Order Planning Forwards |
| **QoS** | Quality of Service |
| **RGB** | Red–Green–Blue colour representation |
| **ROS** | Robot Operating System |
| **SDF** | Simulation Description Format |
| **STL** | Stereolithography mesh format |
| **TF** | ROS transform tree |
| **URDF** | Unified Robot Description Format |
| **VLA** | Vision–Language–Action |
| **VLM** | Vision–Language Model |
| **YAML** | YAML Ain't Markup Language |

## Project-specific terms

**Digital twin** A versioned digital representation of a defined physical system that includes its relevant information flows.

**Fail-closed** A policy that withholds an operation when required state is missing, stale, contradictory or from an unapproved source.

**Validation** The development partition used to select epoch 11, fit the segment temperature and freeze runtime thresholds; it is not the Final Test.

**Development Canary** A post-selection regression partition exposed during development and used to check the frozen candidate without selecting it.

**Final Test** The disjoint partition opened once only after the checkpoint, evaluator, gates and environment had been frozen.

**`ObservedState`** A versioned collection of current observed facts and their provenance at one state identifier.

**SHA-256** A cryptographic digest used here to identify the exact files or model weights associated with an evaluation.

**`TaskGoal`** The fixed task description created after rule-based validation and operator confirmation of a semantic draft.

# Contents

# List of Figures

# List of Tables

# Chapter 1

# General Introduction

**Context and engineering objective.** The MFJA robotics platforms bring together industrial manipulators, a mobile robot, transport cells, cameras and programmable devices. Integrating this equipment in simulation requires compatible models, coordinate frames, namespaces, commands and state descriptions. The challenge was therefore broader than constructing a visually accurate Gazebo scene: the simulated equipment also had to behave consistently enough to support routing, planning and experimentation.

The project also examined whether camera observations and English requests could describe a task without sending motor commands directly. The language and vision paths supply information about the request and the scene. PlanSys2 produces the symbolic sequence, and the supervisor checks each requested step before execution. Neither learned model produces a plan or an action vector.

**Objectives and principal contribution.** The project pursued four engineering objectives within simulation:

**O1.** establish a maintainable Gazebo representation of the MFJA third floor and Room 315, with configuration-driven launch and unified ROS 2 access to the supported simulated robots;

**O2.** implement continuous multi-shuttle transport over calibrated rail geometry and explicit topology, including device semantics, routing, reservations and collision-avoidance constraints;

**O3.** define typed command and state interfaces, together with supervision rules for checking commands and confirming their effects; and

**O4.** design and evaluate a neuro-symbolic workflow that combines natural-language goal interpretation and paired-camera state estimation with PDDL/PlanSys2 planning and supervised execution.

The four objectives form a progression from the simulated environment and its transport system to task-level interaction through language and vision. The objective identifiers O1–O4 are used again in table 11.5 to connect each objective to its evaluated result.

**Method and headline results.** Development followed short build, test and review cycles. Each part was first implemented and checked separately, then connected to the preceding layers. This made it possible to resolve modelling, interface and coordination issues before evaluating the complete workflow. The software, dataset and model versions used for the reported evaluations are listed in appendix A.

The complete workflow and the visual model were evaluated separately. The frozen visual estimator passed all 80 gates on a one-shot Final Test of 1040 scenes and 4680 present-shuttle instances: segment top-1 accuracy was 99.9359%, loaded-state accuracy was 99.9145%, and joint segment/position accuracy at $|\Delta s_{ratio}| \leq 0.05$ was 90.8974%. Independently, all 12 isolated English-to-motion runs reached their declared terminal state. These numbers, their evidence chain and their simulation-only limits are presented in chapter 11.

**Report structure.** The host environment, mission and principal design choices are introduced in chapters 2 to 4. The engineering foundations are then developed in chapters 5 to 8, followed by the language integration, paired-camera data generation and visual training process in chapter 9. Chapter 10 follows one English instruction through planning, supervised motion and result checking, and chapters 11 and 12 present the results and future work.

Chapter 2

# Host Organisation and MFJA Robotics Platforms

## 2.1 Université de Toulouse

The internship was formally hosted by Université de Toulouse, whose legal address is 118 route de Narbonne, 31062 Toulouse Cedex 9. The work itself was carried out at the MFJA site, 1 rue Tarfaya, 31400 Toulouse. As a public university, the institution combines higher education, research and knowledge transfer. Its official Nomenclature d'activités française/Activité principale exercée (NAF/APE) code is 85.42Z [8], which INSEE classifies as *higher education*, including first-, second- and third-cycle university programmes [7].

Université de Toulouse has operated as an experimental public institution since January 2025. Official figures published for 2026 indicate approximately 39,000 students and 4,200 staff [20]. These figures concern the university as a whole, while the internship took place within the smaller MFJA environment.

## 2.2 Maison de la Formation Jacqueline Auriol

The MFJA is a shared centre for engineering education on the Toulouse Aerospace innovation campus. Its activities cover research and innovation, knowledge and skills transfer, and initial and continuing education. According to its official presentation, the site brings together more than 1,500 students from 15 mechanical-engineering programmes associated with Université de Toulouse, INSA and ISAE. It supports work in mechanical engineering, manufacturing, aeronautics, automation and robotics in collaboration with industrial partners and research laboratories [19].

Because the facilities are shared across teaching and research activities, the software developed for the platform must support several configurations, remain usable by different operators and be maintainable beyond a single development environment. These requirements influenced the package and launch architecture presented in Chapters 5–6.

## 2.3 The Room 315 robotics environment

Room 315 contains two flexible transport cells with a common public topology but distinct calibrated geometries, each with four work positions associated with industrial robots. In the configuration used for this project, the right rail serves Yaskawa HC10DT positions 1–2 and Stäubli TX2-60L positions 3–4, while the left rail serves Yaskawa HC10 positions 1–2 and KUKA KR6 R900 sixx positions 3–4. Each circuit contains outer and inner branches, four switching zones, indexing detectors and stoppers. A shuttle may be empty or carry a payload.

This arrangement brings together several engineering disciplines:

- mechanical layout, computer-aided design (CAD), kinematics and collision geometry;
- industrial automation, sensors, stoppers and route switching;
- robot middleware, namespacing, timing and distributed interfaces;
- planning, scheduling and multi-agent resource conflicts;
- computer vision, data collection and model evaluation;
- human–robot interaction through task-level language.

Together, these disciplines make Room 315 a practical platform for studying the integration of robotics, transport, perception and control. Figure 2.1 shows the simulated layout; Chapters 7–8 define the rail topology, device semantics and supervisory constraints.

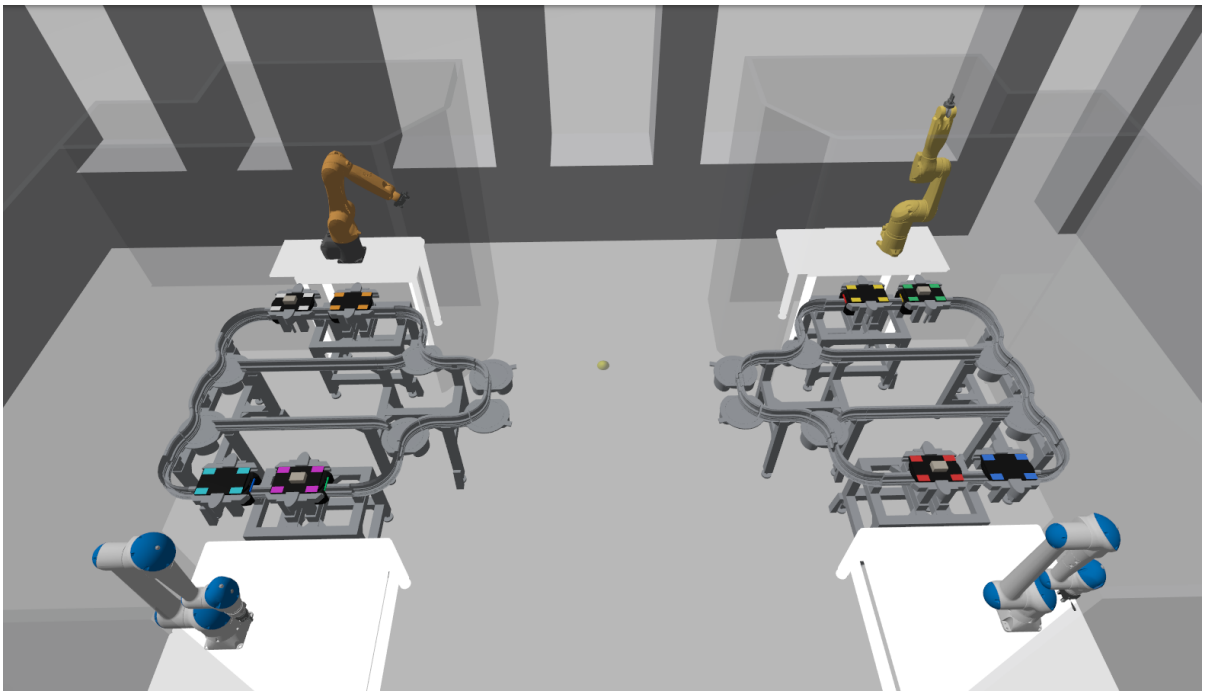


**Figure 2.1:** Gazebo view of Room 315. Four industrial-robot workstations surround two rail-transport cells containing eight stopped shuttles. The measured rail geometries and device locations are presented in chapter 7.

## 2.4 Organisation and responsibilities

Figure 2.2 presents the supervision and responsibilities directly associated with the internship.

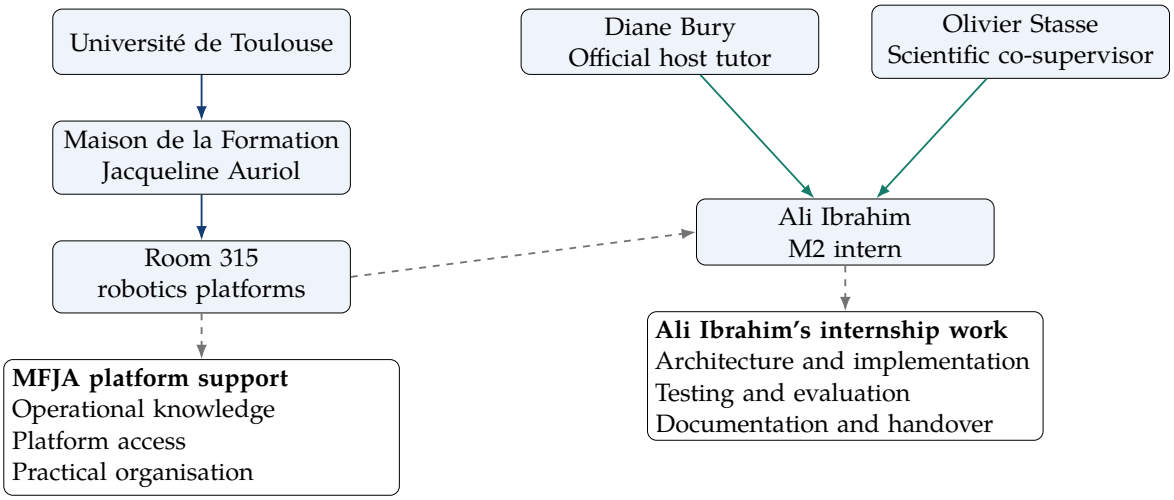



**Figure 2.2:** Host-unit placement, parallel supervision and distribution of internship responsibilities.

Diane Bury was the official host tutor named for the placement, and Olivier Stasse acted as scientific co-supervisor. Together they provided scientific direction and reviewed choices related to the platform. MFJA personnel supported access, scheduling and practical organisation. I was responsible for the architecture, implementation, evaluation, documentation and handover described in chapter 3.

Within the MFJA, the host unit for this work was the Room 315 robotics-platform activity, connecting shared teaching infrastructure with research prototyping.

## 2.5 Position of the mission within the host

The mission connects platform operation with research prototyping. Platform managers, students and researchers can use the system to launch robots and rail scenarios reproducibly. The same foundation can support synthetic-data generation, task-planning experiments and comparisons between simulation configurations. Possible uses include a teaching exercise on ROS 2 topics, a planner benchmark, a visual-perception dataset or a simulation demonstrator.

This range of uses motivated the emphasis on reusable configuration, explicit interfaces and documented operating procedures.

## 2.6 Economic and strategic value

Because the host is a public university rather than a commercial company, turnover and export revenue are not meaningful indicators of this mission's value. No commercial proxy or unsupported financial figure is introduced here. For institutional scale only, the university's 2026–2029 strategic document reports a 2024 global budget of nearly EUR 474 million and EUR 114 million in purchases; these university-wide figures are not presented as project revenue or return on investment [20]. The mission is instead assessed through practical reuse, training capacity and reduced integration effort. The main benefits are:

- **reuse**: one model and launch framework avoids recreating an integration for each practical class or research project;
- **risk reduction**: simulation and supervised execution check naming, topology and commands before access to costly equipment;
- **asset visibility**: a digital twin makes robot capabilities available for remote preparation, including capabilities that might otherwise remain under-used;
- **training capacity**: repeatable scenario generation creates data and exercises without monopolising the physical cell;
- **maintenance**: typed interfaces, tests and documentation reduce dependence on knowledge held by a single developer;
- **technology transfer**: the MFJA can demonstrate current ROS 2, planning and AI methods while maintaining clear safety boundaries.

These benefits align with the MFJA's institutional role of sharing costly engineering resources and connecting education with contemporary industrial methods.

Chapter 3

# Mission, Project Management and Personal Contribution

## 3.1 Mission and work packages

The signed internship agreement defined the broad official mission, *Développement d'une interface unifiée pour robots industriels et mobiles en simulation et réel*: unified ROS 2 access to industrial and mobile robots across simulated and real platforms [22]. The implemented and evaluated scope reported here was limited to simulation; no deployment on the physical platforms is claimed. The first practical assignment was more specific: model Room 315 in Gazebo and establish a usable simulation environment for its robotic and transport equipment. Completing this objective ahead of schedule left time for a research work package combining overhead-camera perception with natural-language task interaction. The internship dates were unchanged.

The implemented work is organised into two phases:

**Phase 1—simulation foundation** Room 315 Gazebo modelling and a reusable, selectively launched multi-robot and rail simulation foundation.

**Phase 2—neuro-symbolic research** Paired cameras, visual-state estimation, validated language goals, PDDL/PlanSys2 and safety-gated neuro-symbolic control.

The phases describe the project chronology, whereas objectives O1–O4 in chapter 1 define the units used to assess the completed work. The simulation phase established the multi-robot and transport foundations. The neuro-symbolic phase extended the typed interfaces and connected visual and language inputs to planning, supervised execution and effect verification.

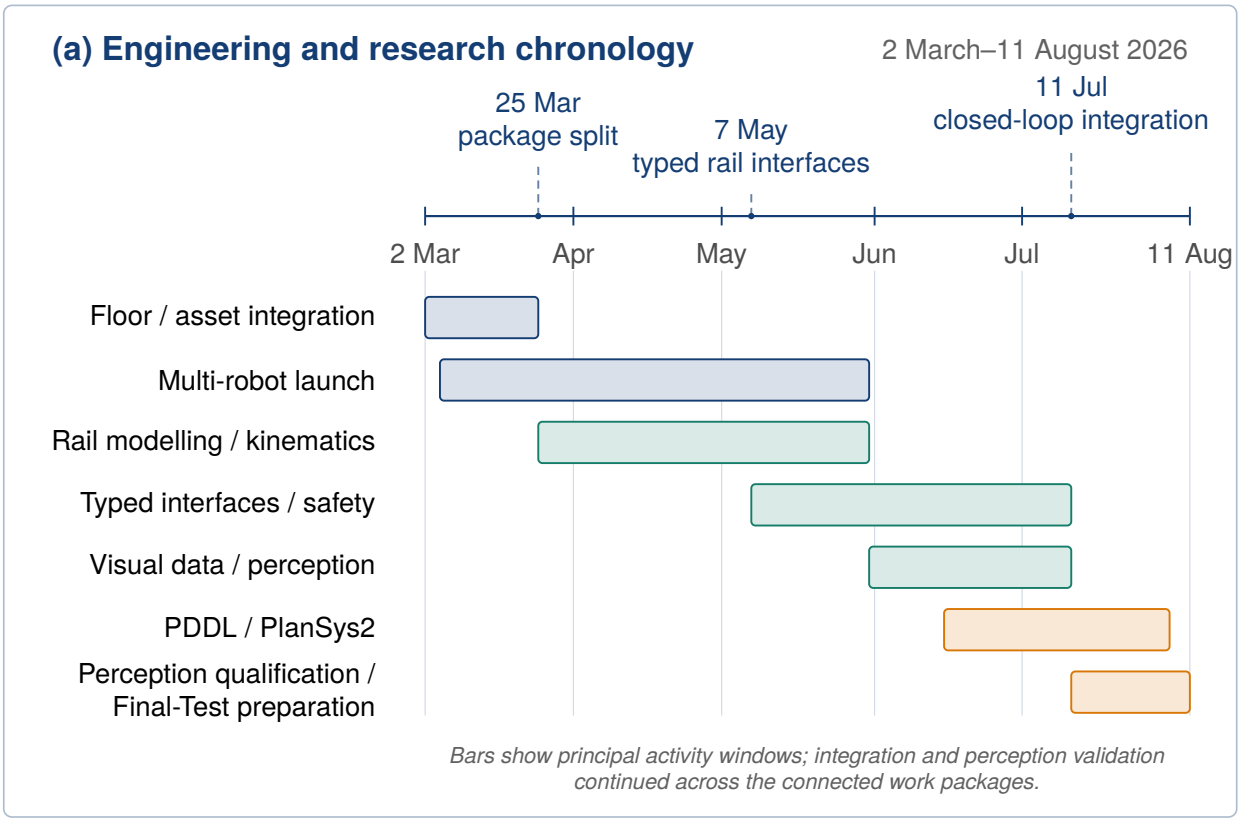
## 3.2 Chronology and decision gates

The internship schedule was fixed from the outset. As shown in figure 3.1, the technical phases overlap because integration, testing and correction continued after each main decision gate.

## 3.3 Decision progression

Figure 3.2 summarises how the project progressed from the initial simulation to the integrated neuro-symbolic system. Each stage depended on working results from the preceding stage.

Major technical decisions were taken after weekly discussions with the supervisors. The agreed solutions were then implemented and checked through configuration, tests and integration runs.



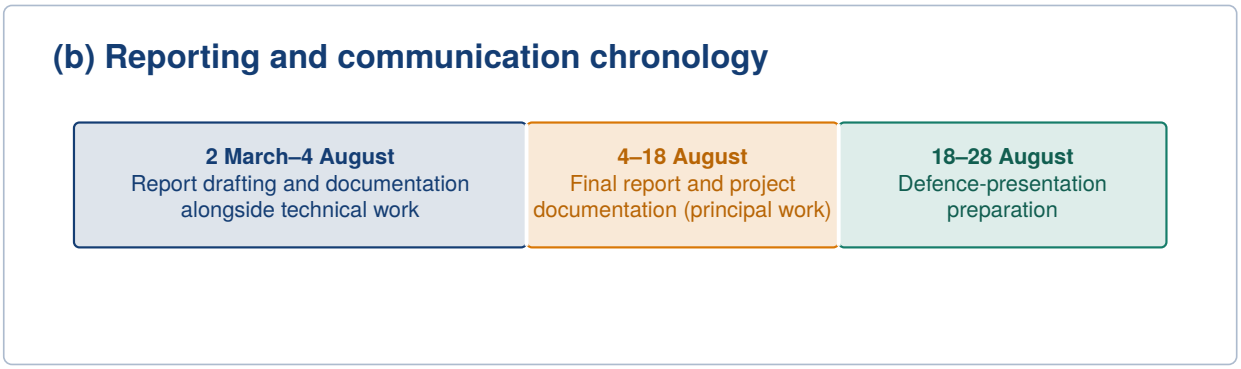




**Figure 3.1:** Internship chronology as of 11 August 2026. Engineering, integration, visual-model validation and sealed final-Test preparation extend through 11 August in panel (a), overlapping the report-focused period that began on 4 August. Documentation continues to 18 August; the final period, 18–28 August, is reserved for defence-presentation preparation.

**Phase 1 — initial assigned task**

**1. World and robots**

**Engineering need:** unify heterogeneous assets, names and topics. **Implementation:** correct the CAD scene, separate package ownership and launch from a namespaced registry. **Result:** select any supported simulated-robot subset.

↓

**2. Shuttle motion**

**Engineering need:** repeatable motion through switch transitions. **Implementation:** use arc-length Hermite paths and Gazebo pose control. **Result:** repeatable continuous motion with an explicit kinematic model.

↓

**3. Topology and devices**

**Engineering need:** encode legal routes, stops and device semantics. **Implementation:** declare directed segments, switches, slots, stoppers and sensors in YAML. **Result:** planner-readable dual-rail state and coordination.

**Phase 2 — additional research direction**

**4. Visual and task interfaces**

**Engineering need:** interpret English tasks and the current camera scene. **Implementation:** pair overhead RGB views, keep labels separate from model input, and convert semantic proposals into confirmed task goals. **Result:** structured language and visual inputs for planning.

↓

**5. Planning and supervised execution**

**Engineering need:** convert the current task and state into supervised motion. **Implementation:** build a fresh PDDL problem, select one symbolic step, translate it, pass it through the supervisor and check the resulting state. **Result:** one-step planning and execution that repeats until completion.

↓

**6. Experiments and results**

**Engineering need:** measure perception and confirm complete task behaviour. **Implementation:** use grouped visual splits, model metrics, software tests and a 12-run cold-start operator-interface campaign. **Result:** model measurements and complete positive runs across both rails, all eight identities and five case families.

**Figure 3.2:** Project progression as a dependency chain. Each stage converts a technical objective into an engineering result required by the next stage.

## 3.4 Requirements and review process

Requirements were expressed at three levels. Launch arguments, topics and runbooks define how operators use the system. Executable contracts and tests specify what each component must accept, reject or preserve. Artefact manifests and hashes give datasets and models reproducible identities. The main requirements across these levels were robot isolation, stable shuttle identity, partial device updates without unintended changes, current state before planning, fixed translation rules and a post-command observation after each executed plan step.

Each property was checked where it was implemented: geometry and coordinate frames, interface semantics, shared-rail coordination and learned-model output. The safeguards and data-leakage controls are detailed in chapters 7 to 9.

The agreed solutions were reflected in configuration and version-controlled code, then exercised through the corresponding tests and experiments. The identifiers for the relevant sources, datasets and models are listed in appendix A; the corresponding objective-to-result assessment is reported in table 11.5.

## 3.5 Personal responsibility, collaboration and resources

Within the shared repository, I was responsible for the third-floor and Room 315 models; the selective multi-robot launch architecture; calibrated rail transport, device logic and typed interfaces; task contracts, PDDL/PlanSys2 integration and supervised one-step execution; the paired-camera dataset and visual training pipeline; and the tests, manifests, runbooks and handover documentation.

The supervisors provided scientific direction, technical review and knowledge of the platform. I used the weekly discussions to present alternatives and the information available for decisions that affected shared interfaces, software safeguards or experimental claims. Once a decision had been agreed, I was responsible for implementing it consistently in the relevant configuration, code, tests and documentation.

In practice, resource management concerned the fixed schedule, shared platform access, simulation and training time, dataset and checkpoint storage, and the testing workload. I kept compute-intensive perception and language nodes optional, prioritised reusable configuration and regression tests, and prepared build and run instructions so that the host could continue using the work. No purchasing budget was assigned to me; GPU time, storage capacity, simulation time and access to the shared platform were therefore managed as the constrained project resources.

## 3.6 Autonomy and reflective engineering examples

I divided the broad integration objective into testable increments and kept early implementation choices reversible. I retained a design only after its interfaces and implementation were consistent and the corresponding regression tests passed. I brought decisions affecting shared interfaces, experimental claims or software safeguards to the supervisors for review before finalising them. The following two examples show how this process worked in practice.

### 3.6.1 Selecting a defensible level of rail fidelity

At the beginning of the rail work, I compared a dynamic wheel–rail simulation with a kinematic rail-following model. The measured mechanical parameters required for a credible dynamic model were not available, and that option was expected to require more integration time and computation. The project questions instead concerned routing, sensor crossings, slot arrival, perception and supervised execution. I therefore proposed the calibrated graph and arc-length solution detailed in chapter 7. I discussed the comparison with the supervisors during a weekly review. The agreed priority was repeatable behaviour at the level required by those questions. Mechanical effects that

could not be supported by the available data were left outside the model. I then implemented the model, checked the configured geometry and tested routing, sensor feedback and slot-arrival behaviour. This decision taught me to match model fidelity to the question being studied and to treat missing data, schedule and computing capacity as engineering constraints.

### 3.6.2 Choosing PDDL/PlanSys2 over end-to-end VLA control

For the second phase, I compared an end-to-end vision–language–action model that would infer actions directly with VLA and hybrid planning architectures discussed with my supervisors. This comparison supported a clear division of responsibilities: the visual model estimates the scene state from the paired camera images, while PDDL/PlanSys2 determines the action sequence, which is then executed through the supervisor and controllers. This approach was better suited to Room 315 because its rail topology and device rules are explicit, whereas the available dataset contains scene-state labels rather than action demonstrations. It also made the plans easier to inspect and execution errors easier to locate. This became the architecture implemented in chapters 9 and 10.

## 3.7 Competencies and professional learning

Table 3.1 relates the principal competencies developed during the internship to concrete examples from the project.

**Table 3.1:** Engineering competencies and concrete project examples.

| Competency | Project example |
|---|---|
| Systems analysis | Separation of geometry, perception, planning, supervision and actuation-provider ownership |
| Robotics integration | ROS 2/Gazebo assets, controllers, launch composition, namespaces and device feedback |
| Algorithm design | Arc-length graph, topology-aware routing, reservations, clearance and effect checks |
| AI/ML method | Local language-model integration, leakage-controlled paired-image data, grouped splitting and multi-head visual training |
| Software quality | Typed contracts, focused regression tests, configuration manifests and reproducible result records |
| Project management and communication | Incremental milestones, supervisory reviews, diagrams, runbooks, result tables and explicit scope |

These two decisions changed how I assessed technical alternatives. The comparisons showed me that each choice had to be justified by the available data, the behaviour required in Room 315 and the way the result could be tested. That lesson guided the later integration work.

# Chapter 4

# Technical Background and Design Choices

## 4.1 ROS 2 as the integration layer

ROS 2 provides processes (nodes), typed interfaces, discovery and distributed communication. Topics suit continuous streams, services suit short request–response operations and actions suit long-running tasks with feedback [15]. These distinctions guided the choice of typed streams, explicit request–response operations and isolated namespaces for the project. The resulting interfaces are presented in chapters 6 and 8.

ROS 2 Quality of Service (QoS) policies configure delivery properties such as reliability and deadline; the standard sensor-data profile explicitly favours timely receipt over guaranteed delivery of every sample [16]. In this project, camera data, state and commands have different reliability and latency needs. The design therefore combines conservative communication defaults with explicit freshness checks. Profiles for other network conditions would require measurement in their target environment.

## 4.2 Gazebo Harmonic and description formats

Gazebo Harmonic is the recommended Gazebo pairing for ROS 2 Jazzy on Ubuntu 24.04 [13]. At its root, the Simulation Description Format (SDF) can contain a model, actor, light or one or more worlds; a world encapsulates models, scene, physics and plugins. SDF also defines sensor elements on links or joints, including their type and properties [17]. The Unified Robot Description Format (URDF) remains useful for robot kinematic trees and `robot_state_publisher`. The `ros_gz_bridge` translates selected Gazebo Transport messages to or from ROS 2 [14].

Simulation serves four complementary purposes in this project:

1. validate coordinate frames, namespaces and controller interfaces across heterogeneous robot models in a controlled environment;
2. support instruction in multi-robot launch and command workflows;
3. execute repeatable rail and safety scenarios at scale;
4. generate labelled overhead observations for perception training.

These purposes impose different fidelity requirements. High-resolution visual meshes support camera-based observation, while simplified collision geometry keeps contact handling efficient and predictable. The shuttle study requires continuous, repeatable transport rather than detailed wheel–rail contact dynamics; it therefore uses kinematic pose control. The corresponding transport assumptions are detailed in chapter 7.

ISO 23247-1 provides an overview and general principles for a manufacturing digital-twin framework, including terms, definitions and framework requirements [9]. In this project, the digital-twin model combines a versioned representation of the simulated environment with clearly defined information flows, assumptions and boundaries. Visual similarity alone is insufficient for that purpose.

## 4.3 Coordinate and rail representations

A rail path is represented by piecewise geometric segments. Each segment has an arc-length coordinate $s$, a physical length $L$, and a normalised coordinate

$$\rho = \frac{s}{L}, \qquad 0 \le \rho \le 1. \tag{4.1}$$

Using equation (4.1), the controller evaluates position and tangent from $s$, then constructs the shuttle pose and heading. Cubic Hermite interpolation provides continuous tangents between calibrated source points. A separate topology graph defines which segments connect for each switch assignment. Geometry therefore determines *where the rail is*, while topology determines *which route is open*. This relationship is shown in figure 4.1.

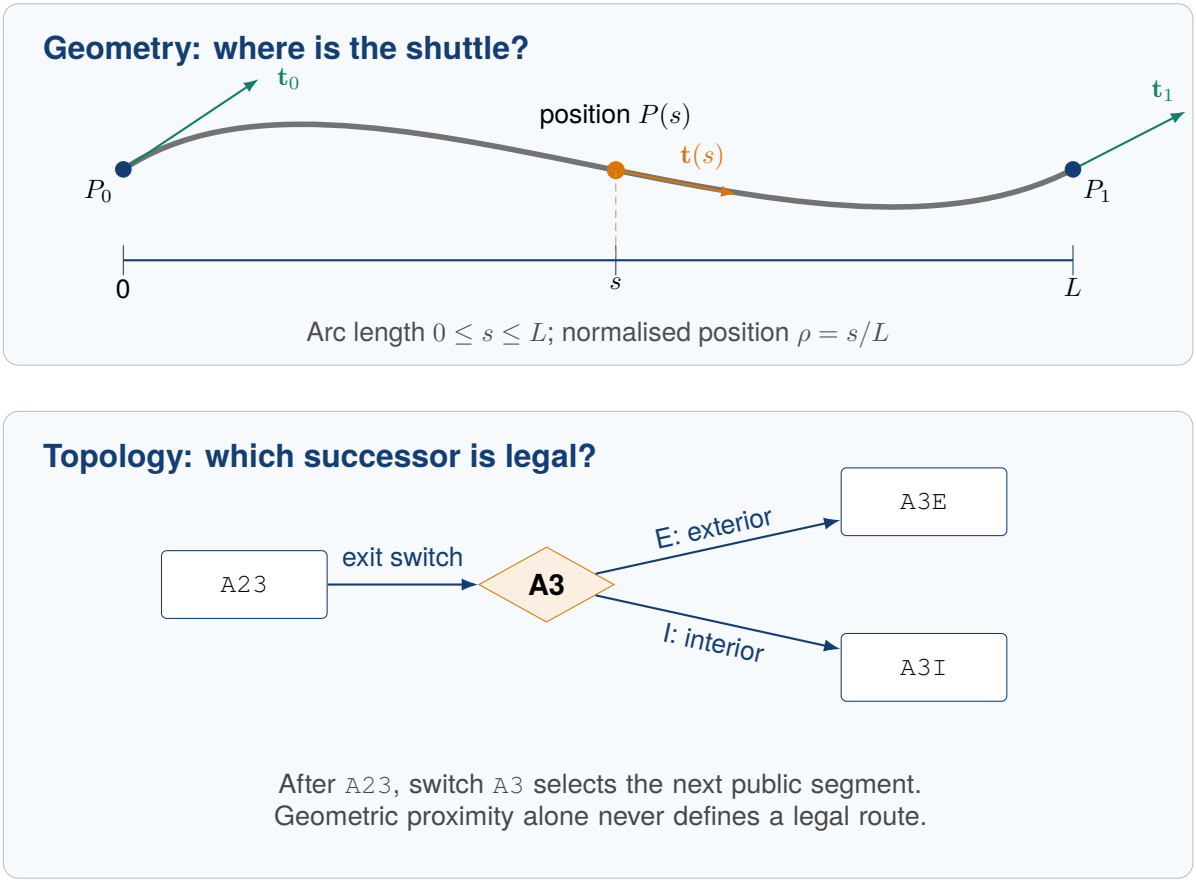



**Figure 4.1:** Complementary rail representations. Hermite geometry supplies position and tangent for pose construction; topology and switch state select the valid successor exposed through the public interface.

The normalised coordinate supports relative placement, while $s$ preserves the metric distance needed for motion control. Chapter 7 presents the transport implementation, and chapters 8 to 10 define the visual-state contract and final arrival checks.

## 4.4 Symbolic task planning

The Planning Domain Definition Language (PDDL) describes objects, predicates, actions, preconditions and effects [12]. PlanSys2 integrates PDDL-based planning into ROS 2 and exposes domain, problem, planner and executor components [11]. The selected solver is POPF. Its original paper describes it as a forward-chaining partial-order planner [2].

Symbolic planning is appropriate for Room 315 because the important decisions are discrete and relational: which shuttle is present, which slot is occupied, which branch is selected, whether

a stopper is open, whether a blocker must be staged and whether the goal has been achieved. Continuous motion remains inside the controller and effect-verification layers.

The planner provides the symbolic sequence, while continuous movement is handled by the controller. The simulated runtime combines the PlanSys2 planner service with project-specific translation, supervision and execution components. These component responsibilities are defined in chapter 10.

## 4.5 Architectural alternatives and final design

The term vision–language–action covers architectures with materially different responsibility splits. RT-2 maps images and language to tokenised robot actions inside one learned policy, while TinyVLA studies a smaller policy in the same end-to-end family [23, 25]. Hybrid systems separate more of the decision process: SayCan combines language-model scores with value functions associated with a predefined set of skills; those value functions ground skill feasibility in the physical environment [6]. OWL-TAMP uses a vision–language model to generate discrete action-ordering and continuous constraints, after which a task-and-motion planner produces a plan that satisfies them [10]. A recent structured long-horizon manipulation study also compares VLA policies with neuro-symbolic sequencing; its result is specific to that task setting rather than a universal ranking of the two families [3]. Table 4.1 summarises the resulting design comparison.

**Table 4.1:** Representative architectures compared by the roles assigned to learning, planning and command execution.

| Approach | Learned output | Planning role | Execution path | Decision for Room 315 |
|---|---|---|---|---|
| End-to-end VLA policy | Robot-action tokens or trajectories | Action selection inside the learned policy; no separate symbolic planner | Decoded action passed to the robot control stack | Not selected: the training corpus labels scene state rather than action trajectories, and the rail rules were already explicit. |
| VLM/LLM with skills or a planner | Ranked skills, subgoals or planning constraints | Skill selector or task-and-motion planner | Selected skill or downstream controller | Related design pattern, but not adopted as-is: open-world constraint generation was unnecessary for the fixed topology. |
| Neuro-symbolic planning with learned control | Symbolic abstractions and low-level continuous-control policies learned from demonstrations | A PDDL planner determines the high-level sequence | Learned controller executes each planned operator | Closest architectural family; adapted because Room 315 learns task and scene inputs but retains an explicit controller. |
| Implemented Room 315 system | Schema-constrained task fields and paired-camera state proposal; neither contains a plan or action | PlanSys2/POPF for normal planning; the stepwise executive handles one route-normalisation prerequisite | Supervisor publishes a typed command; the rail controller applies it | Implemented: matches the state-labelled corpus, fixed topology and simulation objectives. |

For the rest of the report, VLA refers to the Phase 2 research direction and to learned-action systems such as RT-2. The Room 315 implementation is described as a neuro-symbolic language-and-vision interface.

## 4.6 Visual-state model

The perception system uses ResNet-18 as its backbone. The original paper defines the residual architecture and evaluates its 18-layer variant [5]. The paired-view estimator applies one shared set of low-level weights through ResNet `layer3` independently to each image. Each view then

has its own `layer4` and identity-query prediction head, with no cross-camera feature path. This arrangement reuses generic edges, textures and shuttle appearance while allowing rail-specific late features to represent the non-symmetric geometries. Side is derived deterministically from the configured identity—L1–L4 denote the left rail and R1–R4 the right rail—so the learned outputs concentrate on segment, loaded state, bounding box and normalised position.

The fixed-camera, fixed-fleet formulation produces the compact state required by the Room 315 demonstrator and remains specific to this room and fleet. Its inputs, outputs, presence handling and training process are specified in chapter 9. The simulated command and state checks performed by the software supervisor are defined in chapter 8; their engineering limits are discussed in chapter 12.

Chapter 5

# Building the MFJA Digital Twin

## 5.1 Reconstruction strategy

The work began by creating a minimal model of the MFJA third floor. The first objective was to establish a recognisable and geometrically coherent environment before adding behaviour. The third-floor mesh was imported and reoriented, the ground plane was enlarged, and the Room 315 geometry was completed with doors and an interior window. Furniture, protective guards, robot tables and laboratory assets were then added.

A usable digital twin requires each asset to have consistent units, origin, pose, visual material and collision representation. CAD sources frequently place the origin at a manufacturing datum that is inconvenient for simulation, while Stereolithography (STL) files contain no unit metadata. Explicit scale and orientation checks were therefore used to align the visual and collision geometry with the intended room coordinates. The integration process was incremental:

1. load one asset in an isolated scene;
2. verify scale and orientation against known room dimensions;
3. establish a stable model frame;
4. separate visual and collision geometry where necessary;
5. place the asset in the full floor;
6. add a static test for paths and SDF structure.

The KUKA model followed the same process, with a targeted and documented collision-mesh configuration that preserved the required collision behaviour.

## 5.2 World decomposition

The final description package provides three principal world configurations:

- `mfja_3rd_floor.world` for the shared third-floor environment;
- `room_315_only.world` for fast, focused rail, perception and language-integration work;
- isolated industrial-robot worlds assembled by launch for controller and geometry checks.

The internal Gazebo world name forms part of the interface because the pose-update, entity-creation and removal services include it in their service names. The bring-up package passes both the selected world and its internal name explicitly, keeping the shuttle-controller services aligned with the active world. The runbook also includes checks of the available services. The reconstruction progression is shown in figure 5.1.

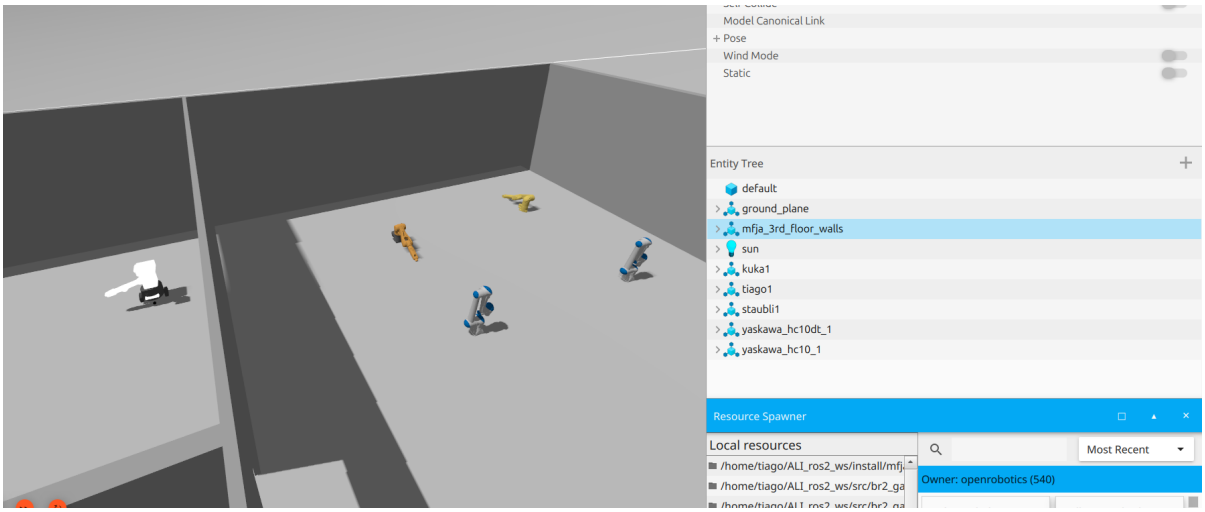
## 5.3 Repository refactoring

As the number of assets and launch components grew, the original flat layout was refactored into a meta-repository on 25 March. The root ceased to be a ROS package, and four packages were assigned separate responsibilities:

**`mfja_3rd_floor_description`** SDF, URDF, worlds, models, meshes and static description tests;

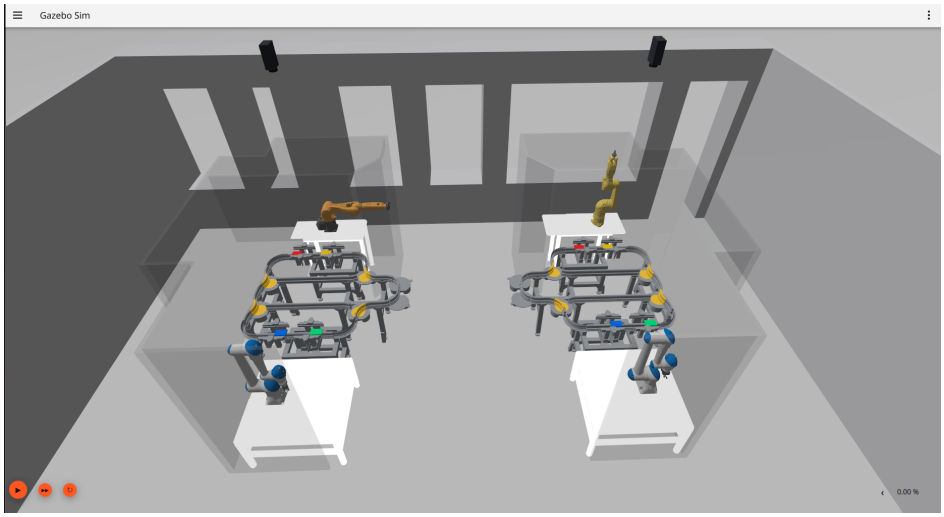
**`mfja_3rd_floor_bringup`** user-facing launch entry points and shared floor/Room 315 orchestration;

**`mfja_rail_interfaces`** custom messages and the shuttle-add service;




(a) Initial minimal floor model.


(b) Reconstructed third-floor context.




(c) Room 315 cells and robots.

**Figure 5.1:** Reconstruction from the initial minimal scene to the complete building context and the detailed Room 315 model. The cell provides the common workspace for transport, perception, planning and supervised execution experiments.

**`mfja_robot_control_config`** robot configuration, rail controllers, planning, perception, learning tools, launches and tests.

This organisation gives the dependencies a clear direction. Description assets remain separate from planning code, and the interface package is independent of the controller implementation. The bring-up package composes the system without containing its control algorithms, while the control/configuration package depends on the description and interface packages. This separation also supports selective `colcon` builds. The ownership and composition boundaries are shown in figure 5.2.

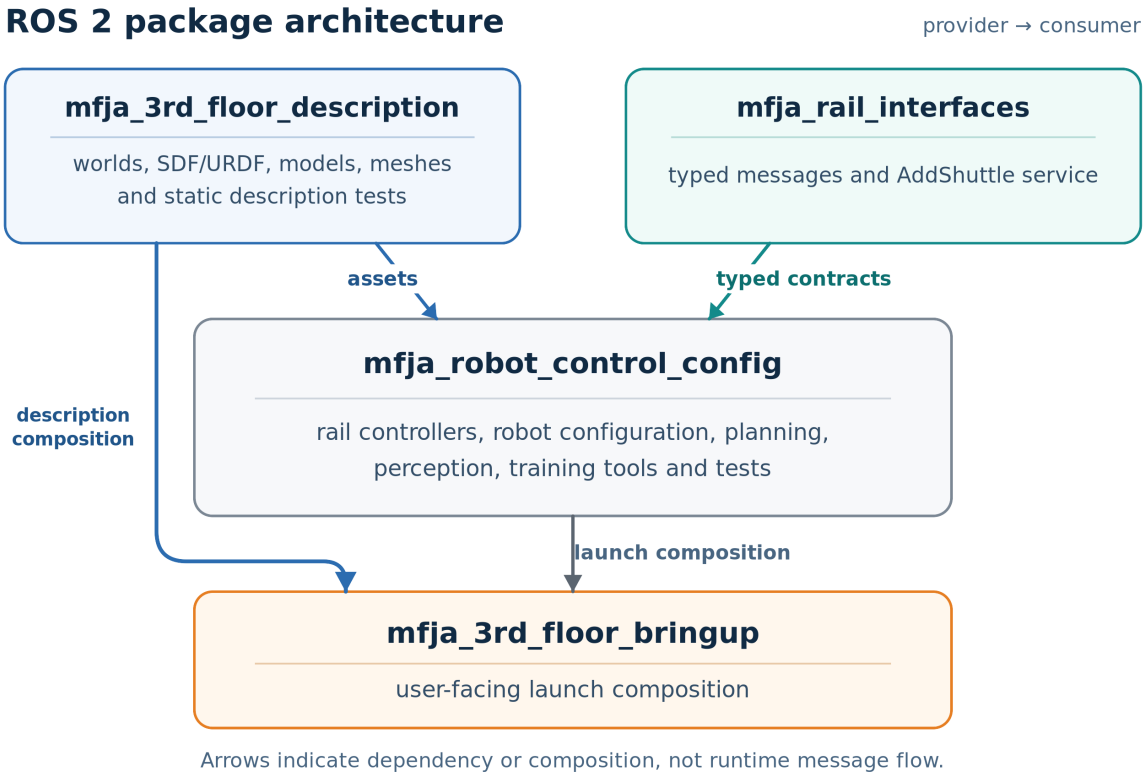



**Figure 5.2:** ROS 2 package ownership and composition. Description assets and typed rail contracts feed the control/configuration package; the bring-up package provides user-facing composition. Arrows represent dependency or launch composition, not runtime ROS message flow.

## 5.4 Model integration

### 5.4.1 Industrial robots

The integrated fixed-base robots are a KUKA KR6 R900 sixx, a Stäubli TX2-60L, a Yaskawa HC10 and a Yaskawa HC10DT. Each model includes a visual and collision tree, support geometry and a controller-facing joint list. The integration preserves vendor-specific joint structure but exposes a common higher-level command tool.

### 5.4.2 TIAGo

TIAGo differs from the industrial arms because it has a mobile base, torso, arm and head; a base-only variant is also available for focused tests. The corresponding embodiment-specific command and feedback surfaces are described in chapter 6.

### 5.4.3 Room and device assets

In addition to the room shell, Room 315 contains separate models for the static left and right cell frames, rail paths, switches, shuttles, carried payloads, cameras and visual markers. The markers support inspection and can be hidden for clean camera capture. The payload model distinguishes the two runtime states used by the task: a shuttle is either *loaded* or *empty*.

## 5.5 Launch architecture

The bring-up package exposes three stable entry points: `full_floor.launch.py`, `room_315_only.launch.py` and `single_industrial_robot.launch.py`. Shared Room 315 arguments are declared once in `room_315_floor_common.py`. This avoids configuration drift in which the full-floor and room-only launch files gradually develop different parameter names or defaults.

Launch choices include graphical user interface (GUI) or headless mode, pause state, selected robots, kinematic shuttles, Room 315 overhead cameras, visual markers, active shuttle identities, starting slots and initial loaded-shuttle selection. The default configuration leaves the scene inactive or readily inspectable, while computationally expensive or research-specific nodes are enabled explicitly. The launch layer also starts the ROS–Gazebo bridges required for entity creation, removal and pose updates.

**Listing 5.1:** Focused Room 315 launch used as the basic transport test.

```
ros2 launch mfja_3rd_floor_bringup room_315_only.launch.py \
  robots:=none \
  start_paused:=false \
  gui:=true \
  enable_room315_kinematic_shuttles:=true
```

## 5.6 Operational profiles

The launch layer combines the same installed assets into several repeatable experimental profiles. An operator can isolate one integration boundary, control the computational load and then scale to the shared environment without changing package ownership or public namespaces. The available profiles are listed in appendix B, and their verification status is presented in chapter 11.

## 5.7 Build and environment reproducibility

The baseline consists of Ubuntu 24.04, ROS 2 Jazzy and Gazebo Harmonic in a standard `colcon` workspace. The package install rules cover the scripts, configuration files, launch files, models and interfaces required by the runtime. The recorded campaigns and the full-floor smoke were launched through the installed ROS 2 package surfaces. The environment and build procedure are specified in appendix A.

## 5.8 Outcome and model fidelity

This phase produced usable simulation models of the full floor and Room 315, selectable robots and a package structure that supports the subsequent transport, perception, language interpretation and planning work. The recorded evaluations include one all-robot full-floor cold start and the Room 315 integrated campaign; later chapters report transport, perception and result checking separately. The transport model and its kinematic assumptions are detailed in chapter 7; the next evaluation steps are discussed in chapter 12.

Chapter 6

# Unified Multi-Robot Access and Control

## 6.1 What "unified" means

In this project, a *unified* access layer is defined by four concrete properties:

1. common YAML configuration declares selectable robot instances;
2. a common launch path spawns the selected instances and their bridges;
3. a separate common command helper validates a named robot and its joint vector;
4. instance-derived namespaces isolate commands and feedback.

The common layer preserves the differences that matter to operation. Fixed-base arms use joint trajectories, while TIAGo additionally exposes base velocity and a linear torso joint. Joint limits, counts, names and units remain robot-specific. The common joint-command helper validates the selector, vector and units for `/joint_trajectory`; interfaces specific to a particular robot structure, such as `/cmd_vel`, remain separate.

## 6.2 Multi-robot operation in one Gazebo process

The first phase established per-instance command interfaces for heterogeneous robots within one Gazebo process. Repeatable operation depends on a consistent identity chain. Each selected YAML entry defines the instance name, model, spawn pose and enabled state. The shared launch rejects duplicate names and derives the namespace and bridge rules from the same identity. Command-helper profiles are validated separately because joint order and units depend on the robot structure. As a result, selection, spawning, commands and feedback remain associated with the selected instance by construction. The retained live smoke covers the all-robot profile; automated suites reported in chapter 11 cover selector rules for the other supported forms.

## 6.3 Configuration-driven robot registry

Robot instances are selected from YAML entries containing the instance name, model, spawn pose and enabled flag. The full floor and Room 315 can use different files while sharing the same loader. The launch layer derives SDF/URDF paths, ROS namespace and bridge topics from those fields. Users can select a full name, derived short alias, numeric order, comma-separated set, `all` or `none`; the principal surfaces are summarised in table 6.1.

The operator helper uses a separate validated Python profile table for aliases, joint ordering, command topics and angular or linear unit semantics. Tests cover both configuration boundaries, although the YAML entries and helper profiles are not generated from a common source.

The YAML-driven launch eliminated duplicated launch scripts for every robot combination. Tests check unique instance names and asset resolution; separate helper tests check profile selection, joint counts, units and command topics.

## 6.4 Dynamic bridge generation

Gazebo and ROS 2 use different transports. The launch process derives bridge configuration for each selected robot and starts `ros_gz_bridge` instances. Joint commands travel from ROS 2 to Gazebo; joint state, odometry and transform information travel back where required. The design follows

**Table 6.1:** Principal robot selectors and command surfaces.

| Selector | Instance | Type | Primary command |
|---|---|---|---|
| `kuka` | `kuka1` | KUKA KR6 | `/kuka1/ joint_trajectory` |
| `staubli` | `staubli1` | Stäubli TX2 | `/staubli1/ joint_trajectory` |
| `hc10` | `yaskawa_hc10_1` | Yaskawa HC10 | `/yaskawa_hc10_1/ joint_trajectory` |
| `hc10dt` | `yaskawa_hc10dt_1` | Yaskawa HC10DT | `/yaskawa_hc10dt_1/ joint_trajectory` |
| `tiago` | `tiago1` | Mobile manipulator | `/tiago1/ joint_trajectory /tiago1/cmd_vel` |
| `tiago_base` | `tiago_base1` | Mobile base with torso | `/tiago_base1/ joint_trajectory /tiago_base1/cmd_vel` |

the official bridge model, in which YAML connects a ROS topic, Gazebo topic, the corresponding message types and communication direction [14].

Deriving the bridges from the selected YAML entries keeps renamed namespaces and bridge configuration synchronised, while per-instance command topics preserve one-to-one addressing. The launch selection, spawned identity and bridge topic are therefore all derived from the same YAML entry, as shown in figure 6.1.

## 6.5 Joint-command helper

Publishing a raw `trajectory_msgs/JointTrajectory` message is cumbersome and prone to input errors. The `robot_joint_command.py` operator tool lists supported names, checks the expected number of positions, maps names to joints, accepts radians or degrees for angular joints and uses metres for linear joints. It then publishes only to the selected namespace.

**Listing 6.1:** Examples of the common robot command tool.

```
ros2 run mfja_robot_control_config robot_joint_command.py --list

ros2 run mfja_robot_control_config robot_joint_command.py \
  kuka --unit deg --positions 34.38 -57.30 63.03 0 34.38 0

ros2 run mfja_robot_control_config robot_joint_command.py \
  staubli --unit rad --positions 0.1 0.4 -0.6 0 0.5 0
```

The helper standardises operator input and creates a short trajectory command with the required joint ordering. Trajectory planning remains separate, as do collision-free arm motion, synchronisation with the transport cell and controller-specific operating constraints.

## 6.6 Interface portability

The public namespaces and typed state boundaries are independent of the current simulator backend. This separation makes provider substitution possible while preserving identity, units, timestamps, freshness and controller-feedback semantics. The implementation and results reported here use the Gazebo providers identified in appendix A.

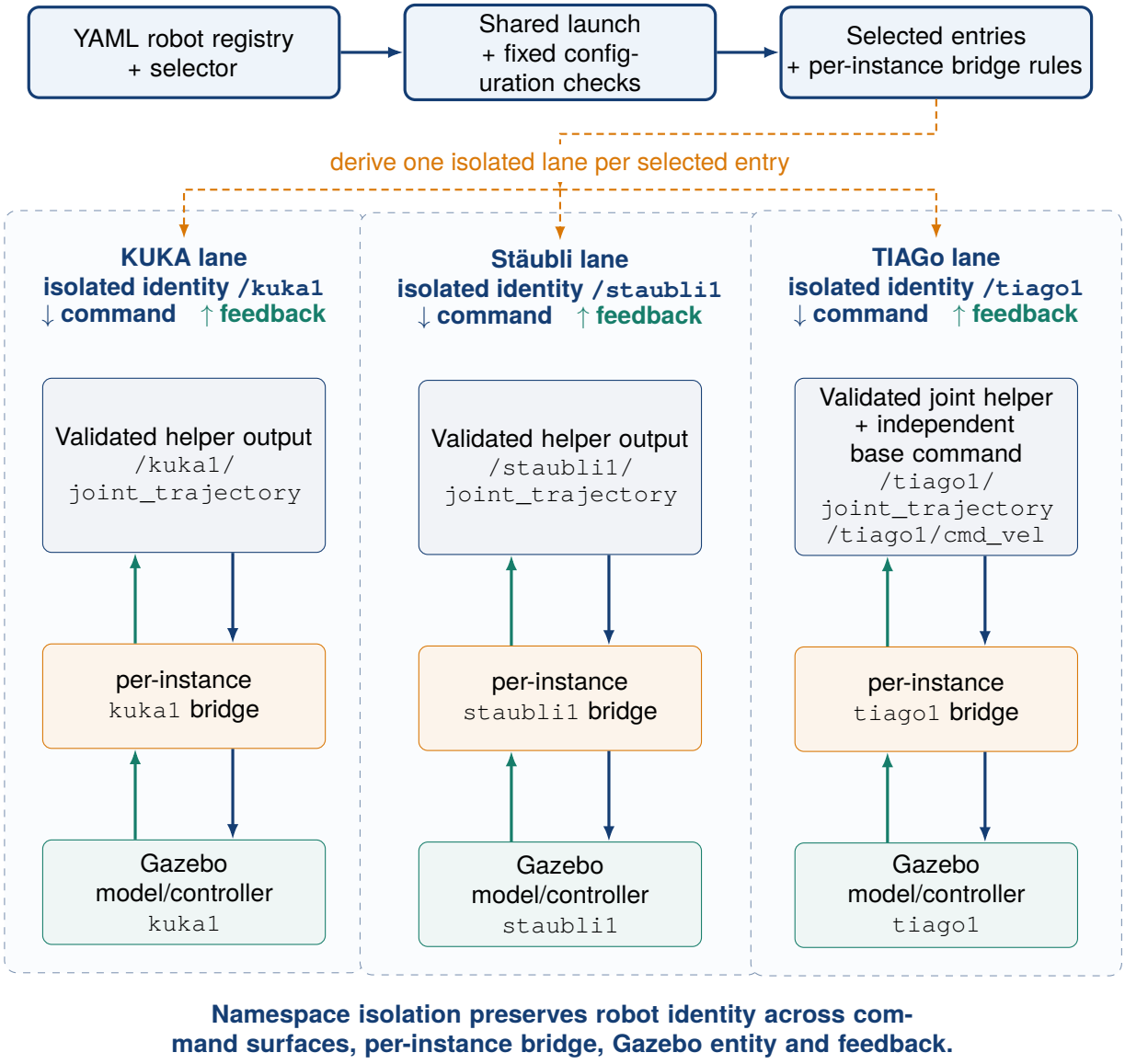



**Figure 6.1:** Configuration-driven multi-robot launch, command and feedback isolation. Selected YAML entries generate per-instance bridges; the selected operator-helper profile addresses the corresponding name. There is no shared global joint-command topic.

## 6.7 Results

The common selector implements `none`, one exact name, comma-separated sets and `all`. In the retained all-robot cold-start smoke, the full-floor entry point spawned the six enabled registry entries. Each exposed a distinct joint command topic and a fresh namespaced state sample; both TIAGo variants also exposed their independent base-velocity topic. The smoke recorded startup and feedback without issuing motion commands. The command helper and launch-selection rules are covered by the named automated suites in chapter 11. The resulting access layer therefore brings the separate simulation models under a common, maintainable interface. The live check covers the `all` selector, while the automated tests exercise the remaining selector rules.

Chapter 7

# Room 315 Kinematic Transport System

## 7.1 Modelling decision: dynamic or kinematic

At the beginning of the rail implementation, two motion strategies were considered:

**Dynamic wheel–rail simulation** Model every shuttle as a driven rigid body and let the Gazebo physics engine determine its motion from wheel or guide contacts with the rail, including the changing geometry at switches.

**Kinematic rail-following simulation** Represent a shuttle by its current graph segment and arc-length coordinate, compute the corresponding pose from the calibrated rail centre-line, and publish that pose to Gazebo.

The dynamic option would be appropriate for estimating traction, contact loads, wheel slip, derailment behaviour or mechanical wear. For the present objectives, however, it would add substantial cost without providing reliable additional information. Each shuttle would introduce several simultaneous wheel/guide–rail contact points, while the switch regions would continually change the active surfaces. Across the eight-shuttle fleet, the physics solver would therefore have to resolve many coupled contact and friction constraints at every simulation step. A credible model would also require validated collision geometry, masses and inertias, wheel and rail profiles, friction, compliance or damping, drive characteristics and switch mechanics. Smaller integration steps and additional solver iterations might then be needed to limit penetration, jitter or loss of contact, further increasing the computational load. Without measured mechanical parameters, this complexity would give an impression of accuracy unsupported by the available data.

The project instead had to test route selection, slot arrival, occupancy, switch and stopper logic, camera perception, planning and supervised execution. The kinematic option directly serves those questions. It provides repeatable trajectories and sensor crossings, predictable logical progression, lower computational cost for multi-shuttle scenarios and faster generation of consistent visual data. It also makes failures easier to localise: geometry determines the pose, topology determines the permitted successor, and the controller and supervisor determine whether motion is allowed.

## 7.2 Kinematic transport backend

The backend advances an arc-length state on a calibrated rail graph and publishes the corresponding Gazebo pose through the world `set_pose` service. This gives the initially static Room 315 rail assemblies predictable motion for repeatable scenario generation, while keeping the implementation focused on planning and supervised execution. The backend provides repeatable control of the route, commanded speed and sensor crossings within the stated kinematic fidelity boundary. The shuttle assets are declared kinematic, gravity is disabled and wheel–rail contact is not used to produce motion. The controller's logical rail state is therefore the state reported to the rest of the system; asynchronous `set_pose` publication and rendering may lag behind that state without changing the logical progression.

## 7.3 From SolidWorks points to a continuous rail graph

The current kinematic model uses ordered rail centre-line points extracted from the SolidWorks geometry. The source set stores 276 points in 14 comma-separated-value (CSV) files; every row contains `index,x,y,z`. The conversion from CAD to runnable motion follows five explicit steps:

1. split the exterior, interior and connector curves at meaningful branch boundaries;
2. preserve point order inside one CSV per segment, so consecutive rows form a measured polyline traversed from index 0 to the last row;
3. associate each public segment with its source CSV and declare its start and end node in network YAML;
4. construct an arc-length parameterisation and a cubic Hermite interpolant between consecutive points, using neighbouring points to estimate tangents;
5. apply the calibrated rail-to-Gazebo transform before publishing the shuttle pose.

The CSV files describe *geometry*, while connectivity is defined separately. Exact floating-point equality between endpoints proved too fragile, so twelve explicit anchor nodes document the segment ends intended to belong to the same physical junction. The configuration records a 0.05 m tolerance for this CAD-to-topology correspondence. Runtime connectivity is taken from the explicit routing tables rather than inferred from coordinate proximity. The loader records the tolerance as configuration metadata rather than applying it during runtime routing.

Each public block has a length and a mapping between public labels and source curve segments. A shuttle state contains its current segment and arc length. The pose evaluator computes

$$\mathbf{p}(s) = \begin{bmatrix} x(s) & y(s) & z(s) \end{bmatrix}^{\mathsf{T}}, \qquad \psi(s) = \operatorname{atan2}\left(\frac{dy}{ds}, \frac{dx}{ds}\right). \tag{7.1}$$

The pose is then transformed into the Gazebo world frame.

Cubic Hermite interpolation was chosen because it allows the position and tangent to be controlled at segment boundaries. Linear interpolation creates visible yaw discontinuities, while an unconstrained global spline may leave the physical rail. Retained tests load every normalised segment with both path backends and preserve the declared topology; separate route and device tests cover successor selection and slot/sensor mappings. In figure 7.1, the black points are the complete ordered CSV input and the coloured curves are dense samples from the same Hermite evaluator used at runtime.

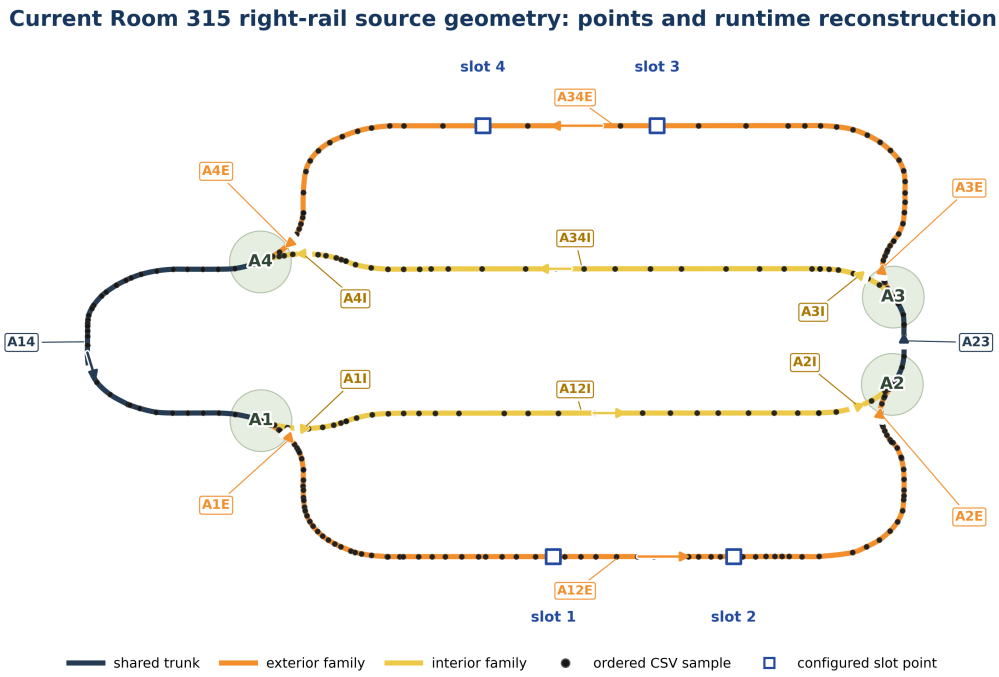



**Figure 7.1:** Current Room 315 centre-line reconstructed from the 276 ordered CAD/CSV samples. Black dots are source samples and coloured curves are dense outputs of the runtime Hermite evaluator. The drawing uses source/CAD orientation, with the configured affine calibration mapping it into Gazebo. This figure checks reconstruction from the configured samples; an independent geometric accuracy study would require separate metrology.

## 7.4 Topology and switch assignments

Active routing combines the rail geometry with switch state. Each rail has four switches, each with exterior or interior state, producing $2^4 = 16$ assignments. The reference left and right network YAML files define nodes, segments, branch connections and public labels. The public segment vocabulary contains 14 blocks:

$$\{\text{A12E, A12I, A14, A1E, A1I, A23, A2E, A2I, A34E, A34I, A3E, A3I, A4E, A4I}\}.$$

The controller selects successor segments according to switch state. The PDDL problem generator reads the same topology, avoiding separate manually written maps for planner and controller routes. A shared topology became especially important when blocker clearance expanded beyond the normal outer loop. Table 7.1 uses only the public ROS naming domain. For the left rail, current and successor segments are converted from internal geometry identifiers to public labels, while switch identifiers already use the public physical labels. The resulting public successor contract is identical for the two rails at the public-topology level.

**Table 7.1:** Public successor map shared by both Room 315 rails.

| Current segment | Route switch | Switch state | Next segment |
|---|---|---|---|
| A23 | A3 | E | A3E |
| A23 | A3 | I | A3I |
| A3E | — | Any | A34E |
| A3I | — | Any | A34I |

| Current segment | Route switch | Switch state | Next segment |
|---|---|---|---|
| A34E | A4 | E | A4E |
| A34E | A4 | I | FALLING |
| A34I | A4 | E | FALLING |
| A34I | A4 | I | A4I |
| A4E | — | Any | A14 |
| A4I | — | Any | A14 |
| A14 | A1 | E | A1E |
| A14 | A1 | I | A1I |
| A1E | — | Any | A12E |
| A1I | — | Any | A12I |
| A12E | A2 | E | A2E |
| A12E | A2 | I | FALLING |
| A12I | A2 | E | FALLING |
| A12I | A2 | I | A2I |
| A2E | — | Any | A23 |
| A2I | — | Any | A23 |

Between segments, the table—not point distance—selects the successor. "Any" marks a fixed transition, while an incompatible converging-switch state has no successor and enters FALLING. In the forward public traversal, A2 is the entry boundary of A23 and A3 selects its exterior or interior successor, as shown in figure 7.2.

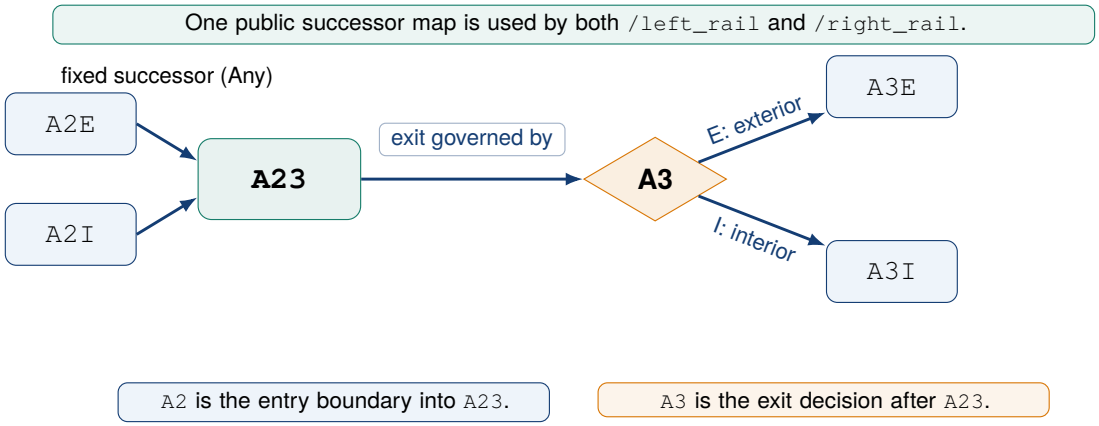



**Figure 7.2:** A2 is the fixed entry boundary into public segment A23. Once the shuttle is on A23, switch A3 selects the exterior or interior successor. The same public rule applies to both rails.

## 7.5 Devices and public naming

The physical documentation groups devices by switches, branches and indexing zones. The implementation assigns consistent public names and defines the devices in YAML. The resulting right-rail layout is shown in figure 7.3.

### 7.5.1 Switches

Switches A1–A4 select exterior or interior branches. Commands and feedback use separate topics. A switch update runs without globally pausing Gazebo, so other robot processes continue normally.

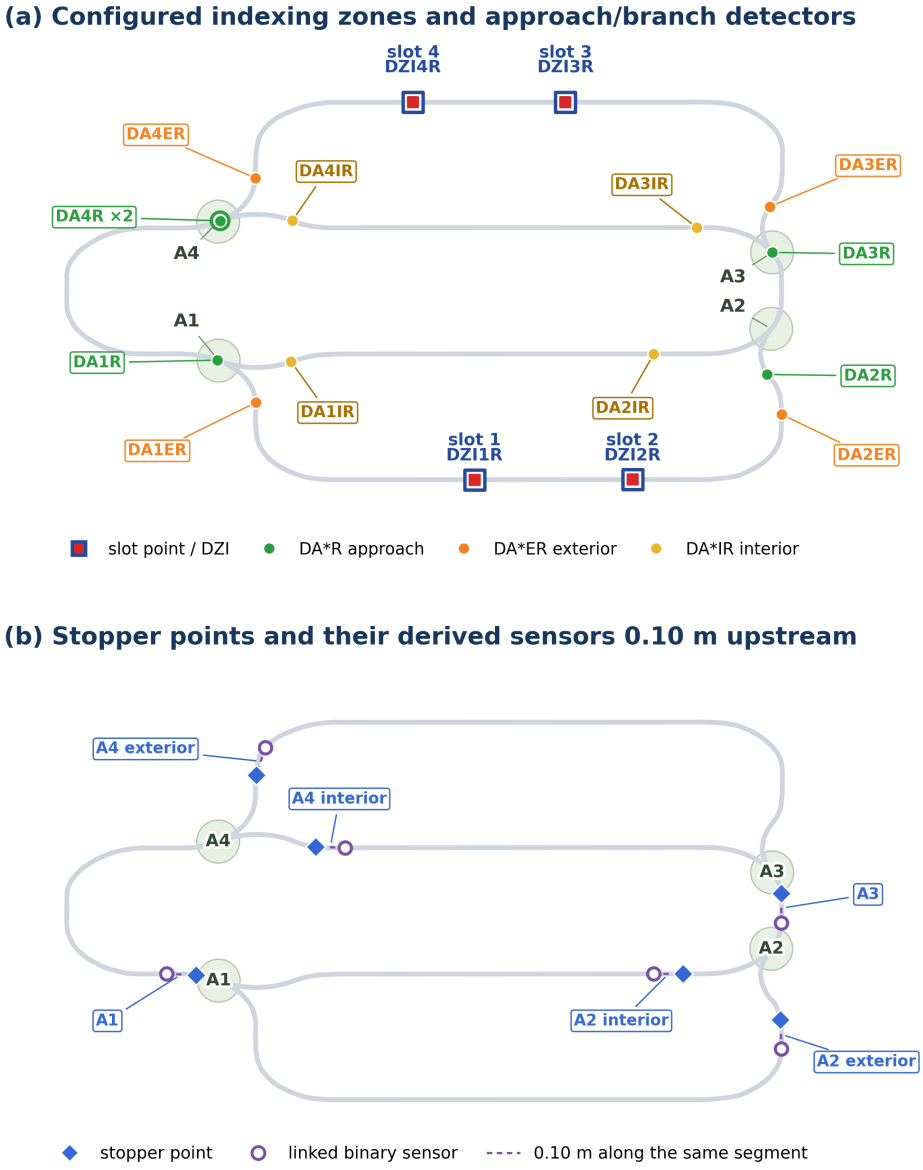



**Figure 7.3:** Right-rail devices evaluated from the configured YAML definitions on the controller's Hermite geometry. Panel (a) shows slots and approach/branch detectors; panel (b) separates physical stopper points from their linked binary sensors 0.10 m upstream.

The protected geometric neighbourhood extends 0.35 m along the rail coordinate; chapter 8 defines how controller feedback authorises or vetoes a change.

### 7.5.2 Stoppers

Stoppers hold or release a shuttle before a protected area or at a target. Their state comes directly from the rail controller. The workflow usually closes a target stopper, opens upstream stoppers, starts the shuttle and waits for the identity-bearing target sensor. A stopped shuttle can still report its configured speed setting, so the state machine uses the explicit controller mode rather than interpreting a nonzero speed field as motion.

### 7.5.3 Binary sensors

Sensors publish continuous binary occupancy readings, not continuous distance. Slot sensors such as DZI1R/DZI2R identify arrivals; approach and branch detectors locate a shuttle near a switch or on an inner/outer route. Continuous pose remains available inside the simulator controller for safety veto and evaluation, but it is not exposed as a learned model input.

## 7.6 Multi-shuttle identity and lifecycle

The current fleet supports four identities on each rail. Stable short identities L1–L4 and R1–R4 map to Gazebo entity names such as `room315_right_shuttle_4`. Identity is not inferred from array order. The operator-facing lifecycle supports:

- add/spawn through a typed low-level service with a name, start slot, speed and initial enabled state;
- per-entity ON, OFF, RESET and REMOVE commands;
- rail-wide `ALL` ON, OFF and RESET commands;
- enabled, disabled, moving, waiting and fault-related state;
- explicit payload state and debug visual identity.

Emergency stop disables all shuttle motion. Any subsequent execution requires an explicit operational decision.

The add service manages simulator entities. Entities created through it are registered for supervised task execution only if they match the reconciled L1–L4 and R1–R4 identities used by the language interface. This keeps the planner's identity domain independent from ad hoc commissioning operations.

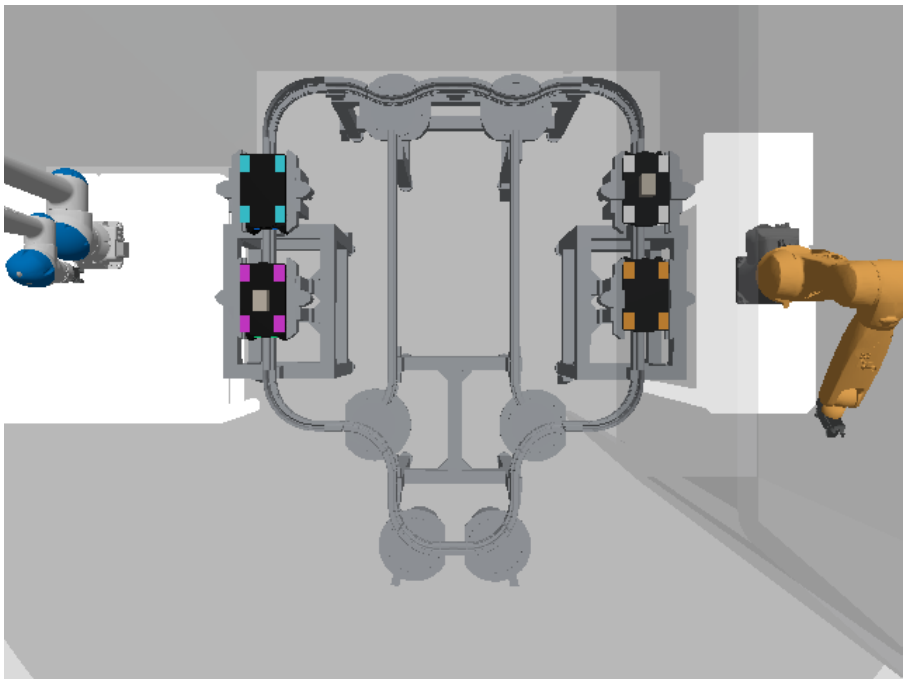
The identity registry reconciles the maximum fleet, configured identities, world entities and default spawn information. Registration proceeds only when these sources agree, ensuring that visual labels and planner objects refer to the same shuttle. Figure 7.4 shows the complete fleet in the two camera views used by the visual pipeline.

## 7.7 Motion, slots and occupancy

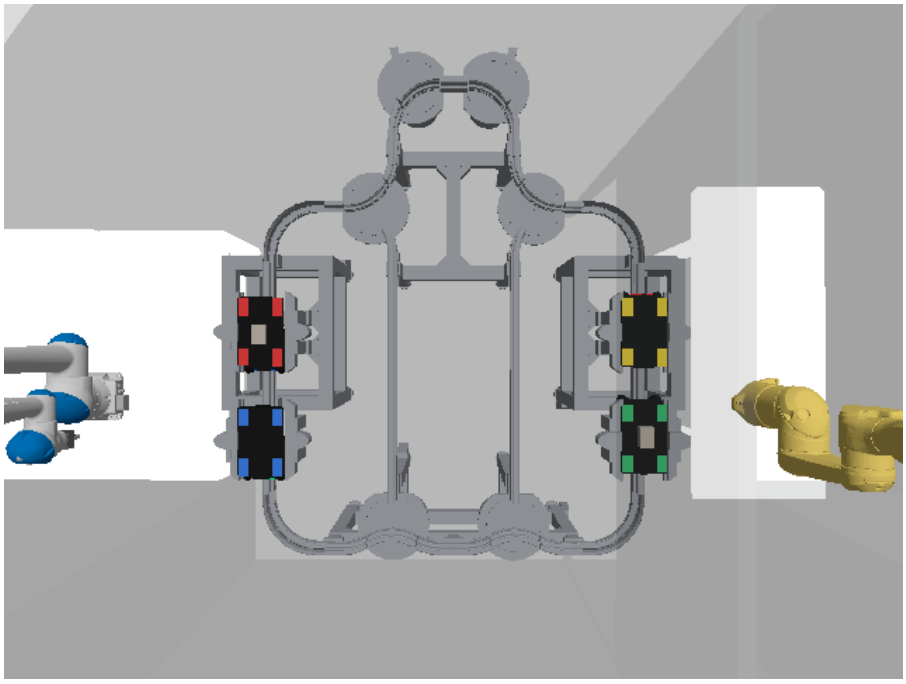
At every update step, an enabled shuttle advances by

$$s_{k+1} = s_k + v\,\Delta t, \tag{7.2}$$

The update in equation (7.2) is subject to the active route, target, stopper and headway constraints. When the shuttle crosses a segment end, the topology selects the next segment and the residual distance is preserved. The controller update loop and configured Gazebo `set_pose` rate limit both default to 30 Hz. Device logic is evaluated on the controller's kinematic update independently of render timing; asynchronous `set_pose` requests may reduce the achieved visual update rate.



**(a)** Left cell: L1 cyan, L2 magenta, L3 orange and L4 white. The centre payload blocks are visible on loaded L2 and L4.



**(b)** Right cell: R1 red, R2 blue, R3 green and R4 yellow. The centre payload blocks are visible on loaded R1 and R3.

**Figure 7.4:** Synchronous overhead Gazebo views of the complete eight-shuttle configuration. Identity colours remain visible while device markers are hidden, making presence and payload appearance readable on both rails.

Slots are calibrated ratios on exterior segments. Arrival requires more than proximity to the target coordinate: the low-level controller uses a target slot, the appropriate stopper, a slot detector and an explicit OFF effect. The planner maintains symbolic occupancy, while the executive verifies the observed identity at the target before declaring completion.

## 7.8 Operator-facing observation and maintenance

The runbook exposes the typed low-level controller API for commissioning and diagnosis. Direct publication can be used to isolate and test a device, while operational tasks continue to pass through the translation and supervision stages. The operator-facing interfaces are listed in appendix B.

The installed device-position tool supports maintenance of the device geometry defined in YAML. It projects a supplied Gazebo XYZ point onto the closest rail segment, reports the resulting arc length, `s_ratio`, projected point and residual distance, and defaults to a dry run. A guarded write can update slots, ordinary position sensors or stopper points; points that exceed the configured distance threshold are rejected unless explicitly overridden. Multi-point stoppers can be addressed by current segment or list index, while their linked detector positions remain derived from the stopper configuration. This tool operates offline: changes take effect only after a relaunch and must then be checked through markers and binary feedback. These maintenance and low-level interfaces remain separate from the closed-loop task interface; their commands and topics are specified in appendix B.

## 7.9 Headway and collision constraints

Multiple shuttles introduce a resource-allocation requirement. The controller enforces separation on the same segment; higher-level reservation ownership and authorisation are defined in chapter 8.

Within the runtime, a *blocker* is another shuttle occupying the requested route, destination slot or required holding capacity. Independent objects placed on the rail are outside the current planning and execution model.

## 7.10 Interior staging and dense blocker cases

Both public interior branches, `A12I` and `A34I`, can serve as capacity-limited holding areas. The planner enumerates admissible holding centres from the calibrated branch length, preserves a 0.35 m end margin and the configured centre-to-centre separation, and prefers the farthest reachable free centre. With the current geometry, each branch admits at most two safely separated shuttles; the positions are calculated rather than fixed. A later `ON` command invalidates the corresponding stop certificate.

These calculated centres impose geometric and capacity constraints on later relocation. The closed-loop selection, re-observation and recovery policy is described in chapter 10.

## 7.11 Validation of the transport model

Static and behavioural checks cover calibrated geometry, topology, devices, identity reconciliation, separation and interior capacity. The corresponding results are presented in chapter 11. The geometric and capacity constraints are enforced through the typed commands and supervisory mechanisms described in chapter 8.

Chapter 8

# Typed Interfaces, Contracts and Safety Supervision

## 8.1 A typed command and state interface

The rail stack uses a dedicated typed interface package with separate command and state topics. Each message specifies its role, required fields and addressed device. This makes multi-shuttle coordination predictable and keeps requests distinct from reported state.

ROS 2 interfaces are defined using `.msg`, `.srv` and `.action` files [15]. The project defines typed messages for named state, switch command/state, stopper command/state, shuttle command/state, sensor feedback/readings and visual state, plus an `AddShuttle` service. The interface package remains independent from Gazebo-specific implementation details.

## 8.2 Command/state separation

The interface distinguishes three message roles:

- a *command* says what a caller requests;
- a *state* says what the responsible controller reports;
- a *result* records the operation status, the readings used to establish it and a reason where applicable.

Command publication and effect observation are separate events. The executive checks controller or supervisor state for the requested effect. Exact slot arrival requires post-command DZI and controller reports, while switch and stopper feedback is subject to the freshness conditions of the corresponding command-result check.

## 8.3 Sparse partial-update semantics

Live switch and stopper commands are sparse named updates. For example, a checked command envelope containing only `switches: {A3: INTERIOR}` requests a change to A3 and says nothing about A1, A2 or A4. An omitted device therefore means "preserve its current state"; no default value is applied to it.

The supervisor checks every named update against the current safety state. After those checks pass, it publishes a typed ROS 2 `SwitchCommand` or `StopperCommand`. Their `switches` and `stoppers` fields are arrays of `NamedState` entries, and the controller applies only the entries present in the corresponding array. Shuttle motion uses its own typed `ShuttleCommand` message. The controllers accept actuation requests through these typed command messages. The translator's auxiliary `event_action` record and the visual model's training-only `target_mask` serve other parts of the workflow.

Figure 8.1 shows the two rules at this boundary: execution must be confirmed through controller feedback, and only explicitly named devices may change.

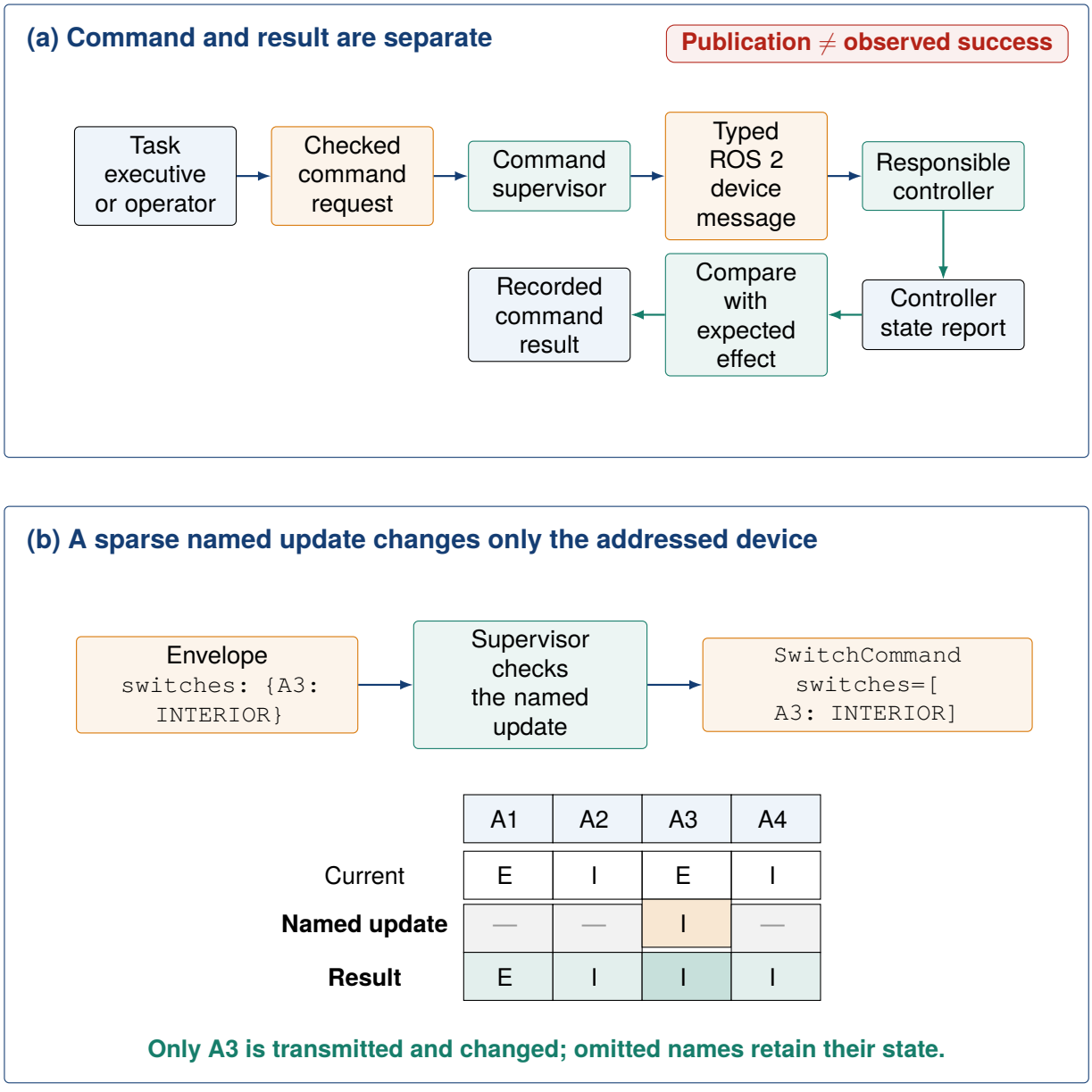



**Figure 8.1:** Live command semantics. The supervisor checks a sparse command dictionary before publishing typed ROS 2 device messages. Controller feedback establishes the observed result, and omitted device names retain their existing state.

## 8.4 Versioned contracts

Operational and task data are represented by versioned Python contracts. Planning and execution use typed records together with checked JSON command dictionaries. Stable identifiers, timestamps and producer/source information are included where required. At the execution boundary, the supervisor converts each authorised request into typed ROS 2 messages. The principal contracts and their permitted producers are summarised in table 8.1.

**Table 8.1:** Principal closed-loop contracts and their producers.

| Contract | Meaning | Allowed producer |
|---|---|---|
| `ObservedFact` | One fact with value, confidence and known/unknown/stale/conflicting status | Visual model or rule-based provider, identified by source |
| `ObservedState` | Versioned state envelope containing visual inputs and fused facts with freshness/status metadata | State-fusion layer |
| `SemanticParseEnvelope` | Strict semantic proposal from user text | Local language model; the schema contains semantic fields and excludes PDDL, plan and command fields |
| `TaskGoalDraft` | Incomplete task proposal after field precedence and fusion | Explicit parser and field-fusion resolver |
| `TaskGoal` | Immutable validated task constraints, optionally grounded to one current instance | Domain validator; before planning, the grounding gateway may derive an instance from current visual facts |
| `PddlPlanStep` | One parsed symbolic action selected from a returned plan | PlanSys2 planner boundary; the executive alone may construct a narrowly defined route-normalisation prerequisite when required |
| `TranslatedPlanStep` | Parsed action, live sparse named command and auxiliary event-action record | Plan translator |
| Checked command dictionary | One actuation-intent request, including only the named device updates to apply | Executive or macro dispatcher; published only after supervisor authorisation |
| `SwitchCommand/ StopperCommand/ ShuttleCommand` | Typed ROS 2 execution messages; device messages carry sparse `NamedState[]` updates | Rule-based software supervisor after current-state validation |
| `PostconditionCheck` and closed-loop records | Recorded effect status, reason and per-step/task outcome | Closed-loop executive using supervisor, controller and observation reports |

The source field determines how a fact may be used. Simulator pose may provide an oracle label or a safety veto, but it is excluded from the visual model input. A language proposal passes through fusion, domain validation and operator confirmation before it becomes a task. The permitted producers in table 8.1 define these boundaries, while figure 10.1 shows the complete runtime sequence.

## 8.5 Observed-state semantics

Every critical fact has four possible statuses: *known*, *unknown*, *stale* and *conflicting*. This is more expressive than a Boolean value plus confidence. Unknown means the value was not observed;

stale means its freshness limit has elapsed; conflicting means authorised sources disagree. The system preserves these statuses instead of silently coercing them to false. Figure 8.2 shows which fact statuses may authorise planner predicates and actuation.

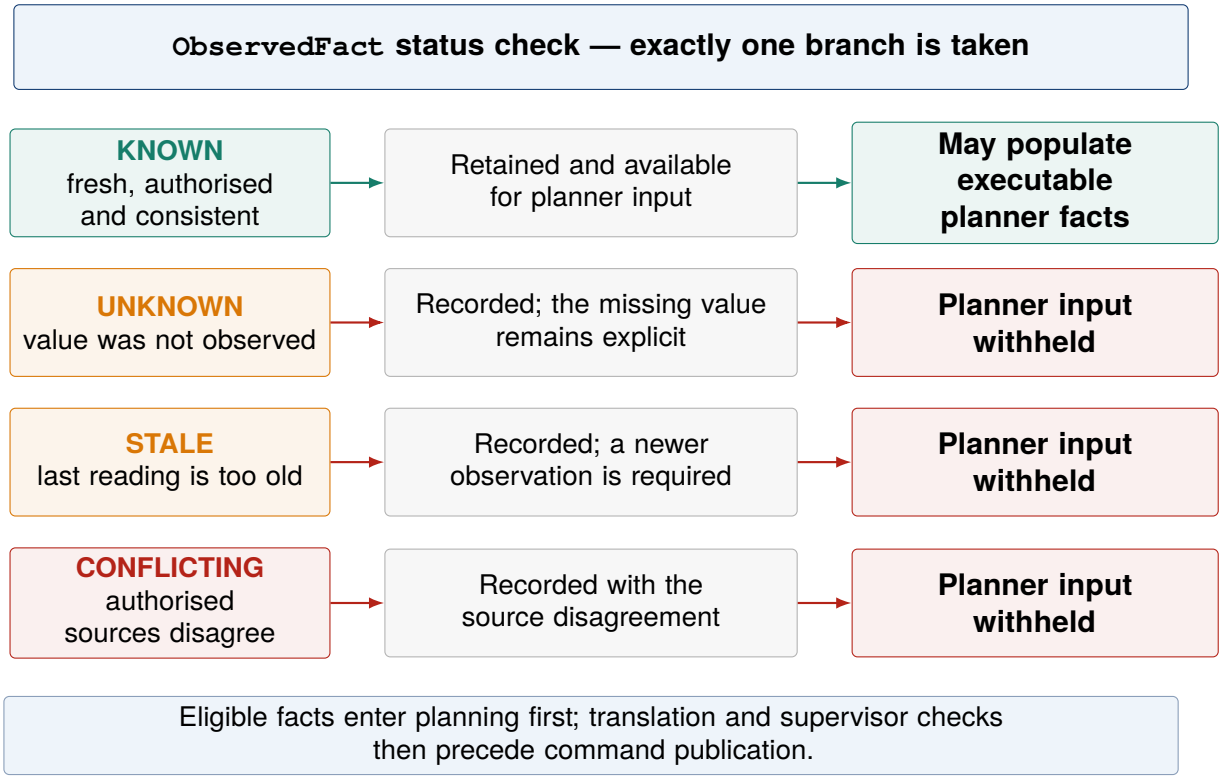



**Figure 8.2:** Status handling before an observed fact enters planner state.

`ObservedState` separates:

**`visual_model_inputs`** the perception facts and image-derived representation intended for the learned boundary;

**`fused_planner_state`** fused facts with their status and freshness metadata after source reconciliation.

The live task gateway retains the latest accepted visual observation together with supervisor and device reports. The provider interface supplies one current snapshot to the executive. Live operation keeps oracle and visual providers distinct, and test-only providers remain isolated from the live path.

## 8.6 Supervisor responsibilities

The Room 315 execution supervisor, implemented in `room_315_vla_supervisor.py`, provides the final software authorisation before a command is published. The component is a rule-based command supervisor, not a learned VLA policy. It subscribes to command dictionaries encoded as JavaScript Object Notation (JSON), validates their sparse named updates, observes the current rail state and publishes typed device or shuttle commands only after the checks pass. The kinematic rail controller consumes these messages and applies the resulting simulated state transition.

The supervisor gates include:

- emergency-stop state and global `stop_all`;
- command structure, identity and rail-side consistency;
- fresh and non-conflicting state;
- occupied target and block reservation;
- same-segment and block headway;

- switch change within the 0.35 m protected distance;
- stopper and route-clearance state;
- post-command occupancy by another shuttle and sensor dropout;
- timeout, unknown localisation and impossible/falling pose;
- limited retry and lexicographic reservation-conflict tie-breaking.

Supervisor metrics and recent decisions are stored for diagnostics. For every published command, the supervisor records the reason for authorisation. A rejected command leaves the requested device unchanged and records the reason for rejection. If an execution exception occurs, or the system shuts down during an active goal, the gateway sends `stop_all`.

## 8.7 Reservations and deadlock

A block reservation associates a route resource with one shuttle and task. Reservations prevent two primitives from entering the same protected block. A separate headway check prevents geometric overlap on the current segment even when the destination block is free.

When crossed reservations form a detected deadlock, a lexicographic identity tie-break grants priority to one identity. The other participants remain safely stopped before observing and planning again. The implemented detector handles these crossed reservations; temporal non-progress and larger wait-for cycles would require a separate detector. Any recovery motion still requires a planned, reachable destination, available capacity and an effect check.

## 8.8 Source of switch feedback

Switch safety relies on a clear separation between data sources. The planner uses current visual locations, in keeping with the visual-state objective. The switch supervisor checks continuous controller coordinates before allowing a change. This check is independent of the learned observation.

The controller coordinate is mapped to the canonical rail graph and the distance to the controlled switch node is measured along the rail, not as a straight Euclidean distance through space. A switch operation is authorised only from current, finite and graph-mappable state; all other cases remain inhibited under the fail-closed policy.

## 8.9 Checking command effects

The runtime defines the expected effects of every translated execution command. Examples include:

- `SET_SWITCHES`: requested switch values match controller status;
- `SET_STOPPERS`: requested stopper values match controller status;
- commanded slot motion: the identity-bearing DZI, controller `reached_target_slot`, explicit `DISABLED` state and required confirmation frames agree;
- `STOP_NOW`: controller reports a fresh explicit `DISABLED` state;
- observation-only inspection: a newer visual observation is available after the request.

The executive associates the result with command, plan-step and observed-state identifiers. Motion arrival and retries following motion require newer state reports. Command-linked observation identifiers are recorded for motion effects; switch and stopper matching currently permits cached state.

## 8.10 Manual and emergency paths

Manual command dictionaries remain available for device debugging. Except for the dedicated emergency stop, they pass through the same structural and supervisor checks. The emergency path

has higher priority and broadcasts a stop without waiting for planner state. A stopper can therefore be tested without a language model while remaining subject to the current safety state.

Chapter 9

# Developing the AI Inputs: English Understanding and Visual-State Learning

## 9.1 Phase 2 objective and implementation sequence

Phase 2 extended the Room 315 transport system with two AI-based inputs. Its objective was to let an operator state a task in English and use the current paired-camera observation to obtain the rail state needed to plan and complete the requested shuttle motion. The language path had to produce a task description compatible with planning, and the image path had to describe the current rail scene.

Development began with two separate questions: could an English request be turned into a valid Room 315 task, and could the paired cameras recover the scene information needed for planning? The two paths were tested separately before they were connected. The implementation sequence is summarised in table 9.1.

**Table 9.1:** Phase 2 development sequence.

| Step | Work performed | Usable result |
|---|---|---|
| 1 | Define language and visual outputs | Semantic proposals contain task fields, while visual proposals contain scene-state values |
| 2 | Integrate a local English model and build task validation | English requests become confirmed `TaskGoal` objects |
| 3 | Generate paired-camera Room 315 data and separate the inputs from labels | A structured corpus represents shuttle presence, location and payload state |
| 4 | Train and select the rail-isolated visual-state estimator | Two fixed-order RGB views produce a side-consistent visual-state proposal |
| 5 | Connect both branches to the runtime checks | A confirmed `TaskGoal` and a current `VisualStateObservation` form the two integration contracts |

## 9.2 Reused components and project-specific development

The two learned components used different adaptation methods. The language model was integrated as a pretrained, quantised model without fine-tuning on Room 315 data. The visual network also reused pretrained image features, but it was subsequently trained on the project's paired-camera corpus.

## 9.3 Building the English task interface

### 9.3.1 Local model selection and configuration

The selected semantic model is `Qwen2.5-1.5B-Instruct-GGUF`. The Qwen2.5 technical report describes the model family and its instruction-tuned, quantised offerings [24]. The official model card documents the GGUF distribution and the Q4_K_M quantisation option [18]; this project uses the `qwen2.5-1.5b-instruct-q4_k_m.gguf` file. The project runs it locally on the CPU through `llama.cpp`. That software provides local LLM inference in C/C++ [4]; the configured artefact hash is listed in appendix A. The runtime is offline and uses a 4096-token context, temperature 0, at

**Table 9.2:** Reused foundations and project-specific Phase 2 work.

| Component | Reused foundation | Developed in this project |
|---|---|---|
| English understanding | Pretrained Qwen2.5-1.5B-Instruct weights and the local `llama.cpp` inference backend | Saved local configuration, constrained prompt, strict JSON schema, explicit-fact extraction, field fusion, clarification, validation and confirmation dialogue |
| Visual state | ImageNet-pretrained TorchVision ResNet-18 | Gazebo corpus, rail-isolated paired-view architecture, 168 learned values, masked multi-head loss, calibrated segment confidence, grouped evaluation and hash-bound runtime integration |
| Planning | PlanSys2 and its POPF solver | Room 315 PDDL model, fresh-problem generation and closed-loop integration (chapter 10) |

most 192 generated tokens and seed 315. These settings record and control the inference conditions and keep each response short enough for the schema check.

The pretrained weights were used without project-specific language-model training or fine-tuning. A versioned prompt constrains the general instruction model to extracting a semantic proposal from an English Room 315 request. The prompt enumerates the permitted task fields, identities, rails, stations and slots. It requires one JSON object of type `SemanticParseEnvelope` and excludes PDDL, plan steps, switch updates, stopper updates and shuttle commands.

### 9.3.2 From model output to a confirmed task

The language path contains four project-specific stages. A rule-based extractor first records facts stated explicitly, such as *green shuttle*, *right rail* or *slot 2*. Qwen then proposes candidate intent fields such as transport, inspection, nearest or payload-qualified selection. The fusion resolver applies fixed precedence, preventing a semantic proposal from overwriting an explicit user fact or confirmed context. This produces a `TaskGoalDraft`, which is not an executable command. The dialogue manager requests any missing information, the domain validator checks rail, identity, station and slot consistency, and the operator confirms the interpretation. An immutable `TaskGoal` is published after these stages are complete.

For example, the request "Move the green shuttle to slot 2." contains a configured colour alias. The explicit extractor resolves *green* to `room315_right_shuttle_3`; the verb and destination are also grounded directly from the text. The semantic envelope is stored with the parse record, and the explicit fields retain precedence. The confirmation therefore states R3, the right rail and slot 2 before the task is published. The complete colour-alias execution is followed in table 10.2.

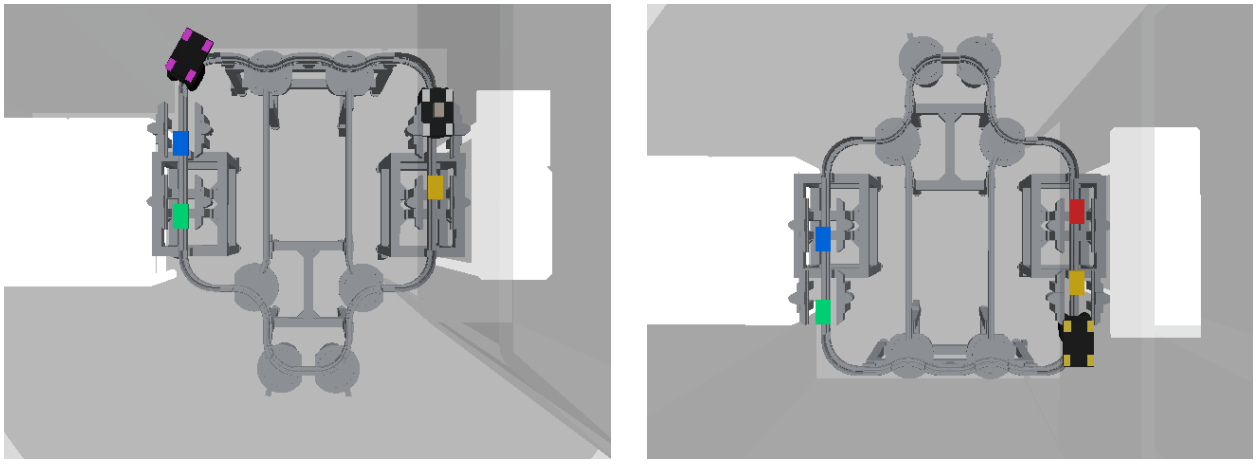
The indirect request "Could you get a loaded shuttle over to the right slot 4?" falls outside the explicit transport-verb patterns. In the saved evaluation record, the semantic model supplied `goal_type=transport`, while field fusion and domain validation constrained the side, payload and destination. A separate ten-case English corpus covers explicit, indirect, ambiguous, inconsistent and cancellation forms; its evaluation is reported in section 11.3.1.

## 9.4 Generating the Room 315 visual corpus

### 9.4.1 Paired-camera corpus and partitioning

For identities declared present by the configured registry, the visual branch estimates the location and loaded state required by the planner. The dataset generator enumerated every non-empty subset of the eight canonical identities L1–L4 and R1–R4, giving $2^8 - 1 = 255$ presence configurations. Eight geometrically different scenes were generated for each configuration by varying valid rail positions, payload states and switch arrangements, producing 2040 scenarios.

For every scenario, the orchestration layer configured the Gazebo scene, allowed it to settle and captured one synchronised image from each overhead rail camera. The resulting corpus contains 2040 paired observations, or 4080 RGB image files. Figure 9.1 shows one such model input.



(a) Left-camera RGB input

(b) Right-camera RGB input

**Figure 9.1:** The two synchronised RGB images form the complete learned input for one scenario. Simulator state is used to construct labels in a separate file and is never appended to the image tensor.

Gazebo supplies the known identity, calibrated rail coordinate, payload state and projected image bounding box used to create supervision. Those values are stored separately from the image references. The model features are the paired RGB overhead images; depth, controller position, point clouds and row-level oracle metadata remain label-generation data. This separation prevents simulator truth from leaking into inference.

All eight variants of one presence configuration remained in the same data partition. The grouped split contained 191 configurations and 1528 scenarios for training, 32 configurations and 256 scenarios for validation, and 32 configurations and 256 scenarios in a locked partition. The locked partition was not loaded or reevaluated during development; the principal Test evidence instead comes from the independently generated one-shot dataset described below.

### 9.4.2 Training, validation and Canary roles

The training pool combines the 1528 grouped training scenes with 4000 training-only hard cases. Its deterministic epoch sampler drew 4000 weighted examples from each source, with replacement, so each epoch contained 8000 records while preserving equal source contribution. Sampling weights increased representation of rare side–segment cells, boundaries, switch zones and multi-blocker scenes.

Checkpoint selection used a separate hard-case validation set of 512 scenes, containing 2254 present-shuttle targets. Only after training and selection had finished was the 256-scene Canary, containing 1085 present-shuttle targets, evaluated as a development regression check. That Canary was not used for checkpoint selection and is not presented as Test evidence. During training and selection, the configuration made Test access an explicit error; the sealed evaluation used a separate, one-shot-only contract.

## 9.5 Designing and training the visual-state estimator

The selected perception contract comprises this rail-isolated visual model and its typed observation schema. The remainder of the report refers to it as the visual model or the runtime, according to context.

**Why the visual contract changed.** The V3 prototype resized both camera images to $224 \times 224$, encoded them with the same ResNet-18 backbone, concatenated the two global feature vectors and used one shared head to predict a 200-value state vector. That formulation discarded the cameras' native $4{:}3$ aspect ratio and allowed either camera to influence every shuttle prediction. It also learned rail side, metric position $s_m$ and segment length even though those quantities are fixed or derivable from identity and topology. The final V4 contract instead preserves the $4{:}3$ input at $320 \times 240$, shares the encoder only through `layer3`, and uses independent `layer4` modules and spatial-query heads for the two rails. It learns 168 values—segment and loaded-state logits, bounding box and $s_{ratio}$ for each fixed identity—then derives side, segment length and $s_m = s_{ratio} L_{segment}$ deterministically. This makes rail isolation explicit and removes redundant learned targets. The Test evidence for the resulting contract is reported separately in section 11.2; the earlier profile is retained only for recovery, not as an active perception path.

### 9.5.1 Rail-isolated 168-value learned contract

Each camera image is resized with its $4{:}3$ aspect ratio preserved to $320 \times 240$ and normalised using the ImageNet channel statistics. A shared-weight ResNet-18 stem, from `conv1` through `layer3`, processes each image separately. The two computation paths then diverge: the left and right cameras have independent `layer4` modules and independent spatial identity-query heads. Four fixed queries on each rail attend to that rail's spatial feature map. No feature from one camera enters the other rail's late branch or prediction head.

The fixed identity order remains L1–L4 followed by R1–R4. Each identity has 21 learned values:

$$21 = \underbrace{14}_{\text{segment logits}} + \underbrace{2}_{\text{loaded-state logits}} + \underbrace{4}_{\text{bounding box } (x,y,w,h)} + \underbrace{1}_{s_{ratio}}, \qquad 8 \times 21 = 168.$$

Rail side is not a learned value. It is derived from identity: L1–L4 map to the left rail and R1–R4 map to the right rail. After segment decoding, the runtime obtains the side-specific segment length from the fingerprinted public topology contract and computes $s_m = s_{ratio} L_{segment}$. This removes side, segment length and $s_m$ from neural prediction while preserving them in the typed observation. A 200-value diagnostic vector can also be assembled, but it is excluded from the control input contract.

Presence remains an independent runtime input rather than a neural output. The training mask gives absent or visually invalid identities zero loss weight. At runtime, absent identities are retained as absent, while every present identity must pass artefact, camera order, freshness, vocabulary, numeric and confidence checks. An unknown identity, side conflict or low-confidence present identity rejects the whole observation instead of publishing a partial planning state.

### 9.5.2 Training and checkpoint selection

The shared feature extractor was initialised from a verified checkpoint, but no joint prediction head was transferred. Compatible weights through `layer3` were loaded into the shared stem; `layer4` was copied independently into the two rail branches, whose parameters and BatchNorm buffers were then optimised separately. The spatial-query heads were initialised independently.

Training used a staged schedule: heads only in epoch 1, independent `layer4` branches from epoch 2, and shared `layer3` from epoch 4 at a lower learning rate. The multi-head objective weighted side-aware segment cross-entropy by 4, normalised-position Smooth-L1 by 2, loaded-state cross-entropy by 1, bounding-box L1 plus generalised IoU by 1, and an expected topological-distance term by 0.25. Appearance perturbations were permitted, but horizontal flips and camera swaps were forbidden because they would violate the fixed side contract. Table 9.3 records the selected run.

All 12 configured epochs completed. Epoch 11 was selected and its checkpoint was then frozen with SHA-256 `869d64049b0092c37d21a4c8b910dc6b91954527e0e49c5694fa82dce570f40d`. Segment logits were temperature-calibrated on validation only before the runtime acceptance threshold was fixed. The runtime binds the complete checkpoint hash, model topology,

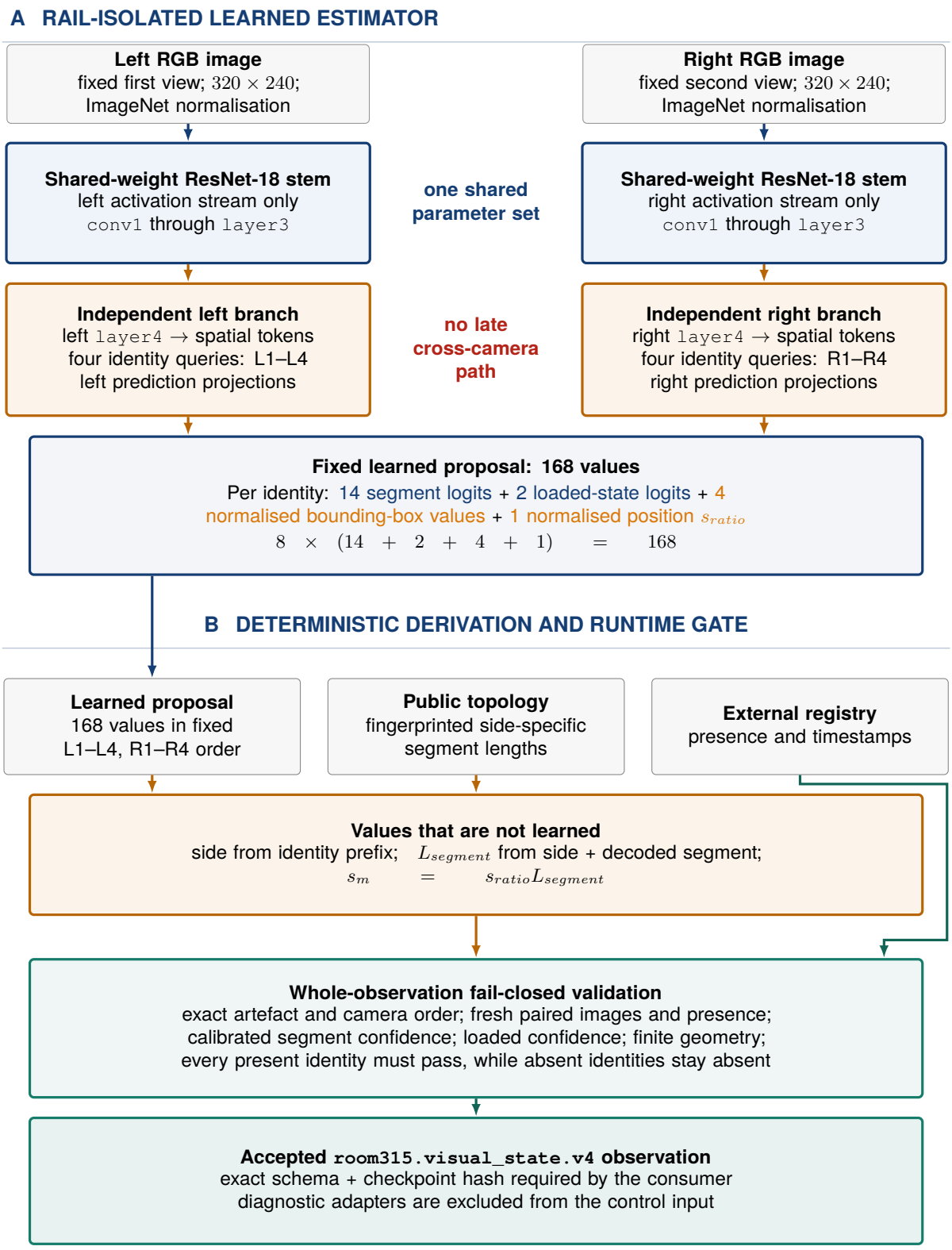



**Figure 9.2:** The model shares low-level ResNet-18 weights through `layer3`, then keeps `layer4` and the four-query prediction head independent for each rail. The model produces 168 learned values; side, segment length and $s_m$ are derived deterministically before whole-observation validation.

**Table 9.3:** Configuration of the selected visual training run.

| Item | Configured value |
|---|---|
| Input | Fixed left–right RGB order; aspect-preserving bilinear resize to $320 \times 240$; ImageNet normalisation |
| Architecture | Shared ResNet-18 through `layer3`; independent `layer4` and four-query spatial head per rail; no cross-camera feature path |
| Optimiser | AdamW; head, `layer4` and shared-`layer3` learning rates $3 \times 10^{-4}$, $10^{-4}$ and $2 \times 10^{-5}$; weight decay $10^{-4}$ |
| Batch and schedule | Batch size 24; 12 epochs; mixed precision; staged unfreezing; early-stopping patience 4 |
| Loss | Segment 4; $s_{ratio}$ 2; loaded state 1; bounding box 1; topology 0.25; presence-aware masks |
| Selection | Lexicographic validation planning score, worst side–segment recall, then correct-segment $s_m$ 95th percentile (p95) |
| Reproducibility | CUDA training, seed 31520260811; Final Test unavailable and forbidden during training, selection, calibration and threshold selection; Canary excluded from training and selection |

preprocessing contract, camera order and public segment-length fingerprint before inference can become ready.

### 9.5.3 Preregistered one-shot Final Test

The frozen checkpoint was evaluated on an independent Gazebo Final Test that was disjoint from training, Validation and the exposed development Canary. Its scenario plan, evaluator, coverage rules, acceptance thresholds and one-shot policy were fixed before inference. A static metadata audit of the 1024-scene base plan found three identity–zone supports below the preregistered minimum of eight. The declared plan therefore included 16 stress scenes, producing 1040 scenes, 2080 individual camera images and 4680 visible-shuttle targets. This decision used only control metadata: no evaluation attempt had been reserved, and no prediction, metric, image, row or label had been inspected. The final control envelopes use the evaluator's canonical schemas, and the captured data remained byte-identical before reservation.

The compatible dataset passed the frozen coverage and disjointness checks, so the global one-shot reservation was created and the checkpoint was evaluated exactly once. The evaluator performed no training, checkpoint selection, calibration refit or threshold selection. The completed immutable result passed 76/76 base acceptance gates and 4/4 preregistered runtime-threshold gates. Detailed metrics and the limits of this synthetic-camera evidence are reported in section 11.2; the sealed result is the principal visual model evidence in the report.

### 9.5.4 Development evidence

Table 9.4 separates the selection evidence from the exposed development Canary. The joint metric requires both the exact segment and $|\Delta s_{ratio}| \leq 0.05$; the metric $s_m$ is reported only when the segment is correct.

**Table 9.4:** Development evidence preceding the sealed Final Test.

| Role | Scenes / present | Segment top-1 | Loaded | Joint at 0.05 | Correct-segment $s_m$ MAE |
|---|---|---|---|---|---|
| Validation (selection) | 512 / 2254 | 99.9113% | 99.8669% | 94.9867% | 0.0187 m |
| Canary (development only) | 256 / 1085 | 100% | 100% | 94.5622% | 0.0201 m |

On validation, segment macro-F1 was 99.8799%, bounding-box mean IoU was 0.8343, correct-segment $s_m$ p95 was 0.0648 m and the worst supported side–segment recall was 97.6190%. On the Canary, segment macro-F1 and worst supported side–segment recall were both 100%, bounding-box mean IoU was 0.8347 and correct-segment $s_m$ p95 was 0.0700 m. These figures describe the simulated datasets and do not establish physical-camera performance.

## 9.6 Integration outputs and runtime qualification

The learned integration contracts remain a confirmed `TaskGoal` and a current `VisualStateObservation`. Calibrated segment confidence and loaded-state confidence accompany the structured state, while the task gateway requires the exact schema and checkpoint hash. Chapter 10 follows how an accepted observation is converted into planning facts.

Observation-only qualification used a same-frame shadow check processed 55 paired observations; the selected model accepted 52, giving 94.5455% coverage, and produced no wrong-side identity. The seven declared Gazebo acceptance scenarios completed 7/7 with 53 ground-truth-matching reobservations. Each accepted reobservation matched every present identity's derived side, segment and loaded state, and met the planning tolerance $|\Delta s_{ratio}| \leq 0.12$.

The visual-input fault suite rejected every declared integrity, camera-order, freshness, presence and identity fault, and the explicit rollback-profile smoke check passed. The observation-only qualification bundle has scope `gazebo_runtime_dry_run_only` and publishes no actuation command.

The command-producing qualification completed all 12/12 positive Gazebo cases with 1784 observations under the selected schema. A separate five-scenario closed-loop fault campaign passed 5/5 cases with 424 observations, produced no false success and left every controller disabled. Neither campaign recorded an observation from the alternate profile. Those two read-only reports were bound into the manually approved state `active_closed_loop_runtime` with scope `gazebo_v4_closed_loop_runtime_only`. In this final profile, the inference node keeps `dry_run_state_fusion=true` and `plansys2_update_enabled=false` so that it cannot mutate ProblemExpert predicates; the separately authorised task gateway still constructs each PDDL problem and owns the Gazebo actuation path. Automatic switching and physical deployment remain forbidden.

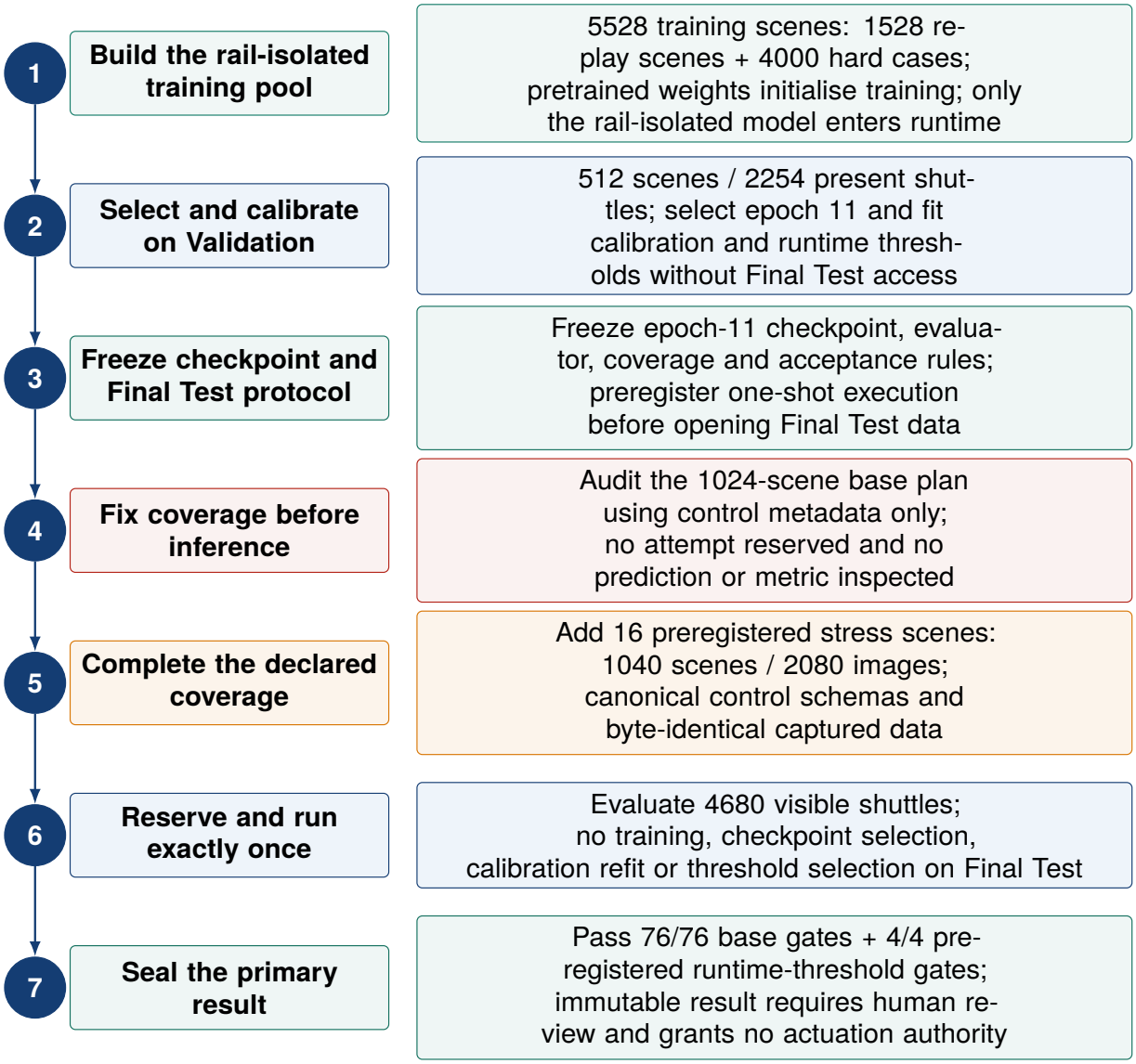



**Figure 9.3:** The model used the declared training pool, was selected and calibrated on Validation, and froze epoch 11 before creating the independent Final Test. A pre-inference metadata audit fixed the declared coverage at 1040 scenes; the captured data then remained byte-identical before the single reserved evaluation.

Chapter 10

# From an English Instruction to Shuttle Motion

## 10.1 Runtime architecture

Figure 10.1 follows the closed-loop runtime from interpretation to an observed Gazebo result. Qwen supplies candidate task fields and the split-rail visual model supplies scene estimates. Whole-observation validation admits a typed `VisualStateObservation` only after its calibrated confidence, freshness and contract checks pass. The task gateway combines that observation with the confirmed task and device evidence, builds a fresh PDDL problem, requests a PlanSys2/POPF plan and selects its first supported step. The deterministic supervisor publishes the corresponding typed ROS 2 command, the rail controller changes the Gazebo state, and the executive compares the post-command visual observation, controller state and identity-bearing DZI evidence with the expected effect. If the goal is not yet complete, the cycle begins again from a new observation. The interface and safety rules behind this sequence are defined in chapter 8.

The inference node's optional direct ProblemExpert predicate mirror remains disabled. This does not interrupt the active command path: the task gateway consumes the accepted visual observation, constructs the current PDDL problem and calls the PlanSys2 planner service directly.

The colours distinguish roles: blue boxes show learned outputs, green boxes show project logic, orange denotes symbolic planning, red marks permission to publish an actuation command, and grey denotes the Gazebo rail controller.

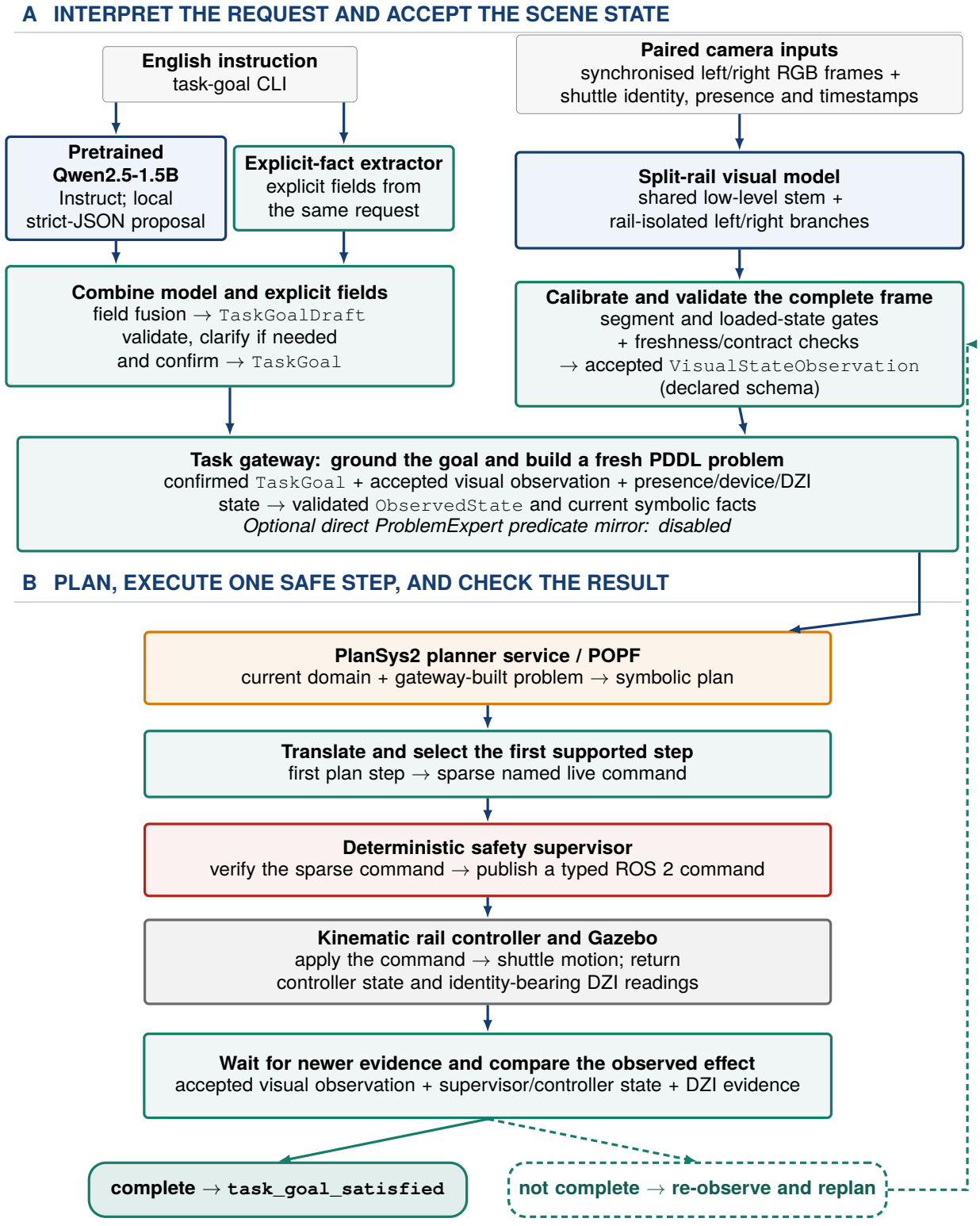



**Figure 10.1:** End-to-end sequence from task and paired-camera inputs to a verified Gazebo result. The optional direct ProblemExpert predicate mirror is disabled; the dashed green path requests a post-command observation while the grounded goal remains incomplete.

## 10.2 Step 1—interpret and confirm the English task

The operator enters an English instruction through the interactive task-goal CLI. The parser pipeline runs the explicit-fact extractor and the local Qwen semantic model described in chapter 9. Their outputs are fused into a `TaskGoalDraft` according to fixed precedence rules. The dialogue manager asks for missing information and presents the completed interpretation for confirmation. The resulting `TaskGoal` contains the task constraints; the planner determines the execution sequence later.

Configured colour names act as identity aliases. For example, the request *Move the green shuttle to slot 4 on the right rail.* resolves to `room315_right_shuttle_3`, the right rail and slot 4 before publication. A request such as *Move the nearest loaded right shuttle to slot 3* remains a selection policy until current state is available. It records the right rail, loaded-payload condition and nearest-selection rule without assigning an identity prematurely.

## 10.3 Step 2—read the current scene and ground the goal

The visual runtime first verifies the immutable runtime manifest, checkpoint, preprocessing contract, camera order, topology fingerprint and confidence policy. It synchronises the left and right RGB topics, requires their declared 4:3 geometry, resizes each image to $320 \times 240$, and preserves their fixed order. The shared low-level stem is followed by independent left and right branches: L1–L4 can depend only on the left image and R1–R4 only on the right image. The 168 learned values cover public-segment logits, loaded-state logits, normalised bounding boxes and normalised longitudinal position $\rho = s/L$, serialised as `s_ratio`. Rail side is derived from identity; segment length and $s_m$ come from the fingerprinted public-topology contract.

For every identity declared present by the independent presence provider, the runtime applies the validation-calibrated segment threshold and the loaded-state decision floor. A stale, missing or conflicting input rejects the complete frame, whereas absent identities emit no accepted visual facts. A typed `VisualStateObservation` additionally carries schema, checkpoint and confidence metadata.

The Gazebo control profile keeps the inference node's `dry_run_state_fusion` guard true and `plansys2_update_enabled` false while separately authorising the task gateway's Gazebo actuation path. These flags prevent the inference node from mutating ProblemExpert predicates; they do not make the gateway observation-only. The gateway verifies the accepted schema and checkpoint, constructs the PDDL problem from that observation and invokes the planner service through its own authorised interface. Physical deployment remains outside this profile.

The task gateway combines the current visual observation and configured presence registry with supervisor device state and identity-bearing rail sensor readings to create an `ObservedState`. Planner localisation excludes the controller's segment and longitudinal-position fields. Continuous controller position is reserved for the supervisor's switch-proximity veto. Because removable external obstacles are disabled in the current configuration, only another shuttle can occupy and block a rail route.

The gateway validates and, when required, grounds the task against the current `ObservedState`. Explicit requests already identify a shuttle, whereas an *any*, *nearest* or station-slot policy is resolved from the current state and route occupancy. A loaded/empty payload condition requires five consecutive fresh visual frames by default before an identity is selected. The grounded task keeps the original goal identifier and remains fixed throughout its execution cycle.

## 10.4 Step 3—build a fresh PDDL problem and plan

### 10.4.1 From current state to decision predicates

For each decision cycle, the executive creates a complete Room 315 PDDL problem from the latest `ObservedState` and the grounded goal. The problem contains the present shuttles, their current symbolic locations, slot occupancy, payload facts, switch and stopper states, and the requested goal. The domain encodes only supported operations and rail-topology constraints.

The PDDL layer receives neither raw images nor free text. Its versioned domain defines typed parameters, preconditions, symbolic effects and a relative cost for every action. The generated problem contains the current known facts, grounded target and goal for the current cycle. Unknown, stale or conflicting state prevents problem generation and is never interpreted as an empty slot or a clear route.

### 10.4.2 How the current facts constrain the next action

For a transport task, each applicable action family requires `validated_state`, the matching `active_goal_side` and their own location, occupancy and route predicates. POPF searches for sequences whose preconditions can hold and whose effects establish the goal, while the generated problem requests `minimize (total-cost)`. These symbolic costs favour plans with fewer unnecessary setup actions or relocations among legal alternatives. Travel-time modelling and supervisor safety checks remain separate. Table 10.1 summarises the principal cases, and the versioned domain contains the complete typed conditions.

**Table 10.1:** Principal symbolic decision cases in the current Room 315 PDDL domain.

| Current symbolic situation | Decisive conditions | Applicable action and symbolic result |
|---|---|---|
| Canonical exterior-route preparation | Validated active side, connected stations, normal-route and named device-group facts | `prepare_switches`, then `open_stoppers`, establish `switches_ready`, `stoppers_open` and `path_ready` when those facts are still required |
| Direct move from an exact slot | Source occupancy agrees with the selected shuttle; target is `slot_free`; route is clear and ready; no clearance is pending | `move_shuttle_to_slot` releases the source occupancy and marks the target occupied and reserved |
| Switch-specific or segment-origin route | A topology block is known; a route to the free target is available and clear; no clearance is pending | `prepare_topology_route` or `prepare_slot_topology_route`, followed by the matching topology move, configures a legal route without inventing a source slot |
| Another shuttle blocks the required route | An ordered clearance is pending; the next relocation or staging destination is capacity-safe and its interior entry is clear | The clearance family enters `clearance_mode` and selects one permitted next relocation. Each completed step produces a fresh problem; finish or pause actions control route restoration |
| Mixed route requiring normalisation | Route reconfiguration is allowed; clearance mode is inactive; no relocation remains | `restore_normal_route` restores the exterior route and its readiness facts |
| Arrival or inspection is ready to close | Target station/slot and `DISABLED` controller-state facts hold, or a confirmed inspection is required | `stop_shuttle` and the terminal actions establish task completion; `inspect_state` completes an inspection without an actuation command |

PDDL effects describe the state expected after an action. The next planning cycle nevertheless uses the observed post-action state rather than assuming that the prediction occurred. The complete B03 run later in this chapter shows this difference between a symbolic effect and an observed controller result.

In the normal planning path, the executive sends the domain and problem to the PlanSys2 planner service, configured with the POPF solver. Planning uses the service directly rather than the PlanSys2

Executor. The executive then translates and validates the symbolic plan and selects its first supported `PddlPlanStep/TranslatedPlanStep` pair. Each executed decision is therefore tied to the state used to plan it.

## 10.5 Step 4—translate, supervise and move

The PDDL translator converts supported actions into `TranslatedPlanStep` records, each containing a sparse, named command dictionary. A switch command names only the switches that must change; a stopper command names only its target devices; and a shuttle command includes the identity, side, direction, speed or target slot required by that action. The auxiliary `event_action` field records the source action for dataset tooling, whereas the first step's `command` field supplies the live request.

The translated request passes through the supervisor checks described in chapter 8. If those checks pass, the supervisor publishes a typed `SwitchCommand`, `StopperCommand` or `ShuttleCommand`. The kinematic controller consumes that message and changes the shuttle state and Gazebo pose.

## 10.6 Step 5—check the result and decide what follows

After executing the selected step, the executive obtains newer state reports and checks its action-specific postcondition. A switch or stopper action must match the requested controller state, while a motion action must establish the expected location or route effect. If the grounded goal remains incomplete, the next cycle builds a new PDDL problem from the observed after-state. Any change in occupancy is included explicitly when replanning.

Final-slot success is reported only when the DZI and controller conditions defined in chapter 8 are met. Otherwise, the executive obtains a newer observation and starts another planning cycle.

## 10.7 Case B03 from instruction to arrival

The recorded instruction was *Move the green shuttle to slot 4 on the right rail.* Table 10.2 follows the main stages of this successful cold-start execution. The campaign case defined the instruction, goal and launch state, then left the action sequence to the planner. In the recorded run, it selected one direct-motion step to R3's slot 4 destination, which maps to DZI4R.

**Table 10.2:** English-to-motion record for campaign case B03.

| Stage | Recorded result |
|---|---|
| Interactive request | *Move the green shuttle to slot 4 on the right rail.* The harness supplied `yes` after the rendered interpretation matched the case declaration |
| Confirmed task | Transport `room315_right_shuttle_3` on the right rail to slot 4 |
| Current state | Visual state supplies current location and occupancy; a fresh PDDL problem is sent to POPF and the executive selects the first supported step |
| Execution step | The recorded initial state places R3 at slot 3 with slot 4 free. The motion portion therefore contains one `move_shuttle_to_slot` step; it requires neither blocker clearance nor a topology relocation |
| Motion command | `move_shuttle_to_slot` for R3 from right slot 3 to right slot 4 within the Stäubli TX2-60L section at 0.2 m/s |
| Arrival | Two separately timestamped DZI4R readings report identity R3; the matching `ShuttleState` reports target slot 4 and `DISABLED`, and a newer visual observation records the final scene |
| Outcome | Every recorded postcondition is satisfied; the final status is `succeeded` with reason `task_goal_satisfied` |

Selection policies such as *nearest loaded shuttle* use the same sequence, with the identity chosen from the current scene before the PDDL problem is built. The recorded explicit-identity, colour-alias and payload-qualified cases on both rails followed this workflow. Chapter 11 presents the campaign outcomes and visual-model measurements.

Chapter 11

# Experiments and Results

## 11.1 Closed-loop campaign result

### 11.1.1 Campaign design and acceptance criteria

The active perception contract is the rail-isolated schema and its hash-identified epoch-11 checkpoint. The closed-loop campaign used a predeclared twelve-row test matrix. Each row fixed a case identifier, one unique English utterance, its predeclared case family and expected grounded goal, together with the rail, active identities, initial slot and payload arrangement. Slot requests fixed one terminal slot; station requests fixed the set of acceptable station slots. The eight B-series rows left their symbolic action sequence to the planner. Only the two P-series and two M-series route-clearance rows additionally fixed the exact four-step trace that execution had to reproduce. The saved environment record identifies the software, language configuration, visual-model checkpoint, qualification manifest, runtime configuration and PDDL domain used by the campaign. The runner rehashed this declared artifact set and checked source/install parity before and after the campaign. Before each case it rehashed the checkpoint, while the task gateway reverified the immutable qualification record and required its qualification-only Gazebo authorisation scope.

Every row used a cold start: a fresh Gazebo process and an isolated ROS domain, with no retained task, observation or controller state from another row. Before the utterance was entered, the runner waited for the world and scene, both rail controllers, both overhead camera streams, the authorised visual runtime, the PlanSys2 planner service, the supervisor and the task gateway. The matrix assigned six rows to each rail, exercised all eight configured identities and partitioned the cases into five mutually exclusive families. Four families describe the language or goal form: colour alias, explicit identity, station target and payload-qualified nearest. The fifth is a route-clearance scenario family whose explicit-identity requests require a blocking shuttle to be positioned first. Table 11.1 lists every declared row alongside the recorded motion and terminal result.

For each case, the runner entered the instruction, compared the displayed interpretation with the expected goal and then replied `yes` through the same interactive confirmation used in normal operation. The CLI's auto-confirm mode was not used. It independently audited every recorded visual observation: accepted observations had to carry the declared schema and checkpoint. The initial and final observations also had to match the declared side, public segment, loaded state and longitudinal coordinate within the predeclared tolerance. A row counted as successful only when the expected terminal state was reached, every step postcondition passed, the required commands passed the supervisor checks and the final DZI and stopped-controller readings agreed. The camera recording and ROS bag also had to close cleanly.

The twelve rows represent supported normal-workflow tasks in valid initial scenes. The automated suites in section 11.3 separately exercise stale state, sensor loss, emergency stop and protected-switch rejection.

### 11.1.2 Recorded result and claim boundary

All 12 planned cases were completed: 12/12 runs succeeded, all 24 recorded step postconditions were satisfied (24/24), and all 48 recorded supervisor action decisions passed their checks (48/48). Across the twelve runs, the executive made 24 plan attempts and recorded four occupancy-triggered replan events; it recorded no unknown-result retries and issued no safe aborts. All tasks ended with `status=succeeded` and reason `task_goal_satisfied`; all final effects were verified and every controller was stopped. The twelve bags contain 1784 schema-compliant observations. Together, the

**Table 11.1:** Closed-loop campaign rows and recorded terminal results.

| Case | Case family | Rail / shuttle | Recorded task path | Steps / postconditions | Status |
|---|---|---|---|---|---|
| B01 | Colour alias | Right / R1 | Slot 1 → slot 2 | 1/1 | Passed |
| B02 | Explicit identity | Right / R2 | Slot 2 → slot 3 | 1/1 | Passed |
| B03 | Colour alias | Right / R3 | Slot 3 → slot 4 | 1/1 | Passed |
| B04 | Station target | Right / R4 | Slot 4 → Yaskawa, slot 1 | 1/1 | Passed |
| B05 | Colour alias | Left / L1 | Slot 1 → slot 2 | 1/1 | Passed |
| B06 | Explicit identity | Left / L2 | Slot 2 → slot 3 | 1/1 | Passed |
| B07 | Colour alias | Left / L3 | Slot 3 → slot 4 | 1/1 | Passed |
| B08 | Station target | Left / L4 | Slot 4 → Yaskawa, slot 1 | 1/1 | Passed |
| P01 | Payload-qualified nearest | Right / R1 | Loaded R1 selected; R2 cleared; slot 1 → slot 3 | 4/4 | Passed |
| P02 | Payload-qualified nearest | Left / L1 | Loaded L1 selected; L2 cleared; slot 1 → slot 3 | 4/4 | Passed |
| M01 | Multi-shuttle route | Right / R1 | R2 cleared; slot 1 → slot 3 | 4/4 | Passed |
| M02 | Multi-shuttle route | Left / L1 | L2 cleared; slot 1 → slot 3 | 4/4 | Passed |

runs trace the complete path from twelve distinct English instructions, through visual perception and fresh PDDL problem construction, to the expected shuttle movements in the selected Gazebo scenes.

The sealed `summary.json` is identified by SHA-256 `1504f25742d135c32ea879651336ff22a64cdfb2e09870be81ea7fb68b279479`. It records `physical_deployment=false`. This is therefore positive-case Gazebo qualification evidence rather than a physical-robot result, a reliability estimate or fault-campaign evidence. It is one of the two immutable records bound to the runtime authority; no physical deployment is claimed.

Figure 11.1 compares the initial and final camera frames from case B03 and shows the corresponding change in the simulated scene.

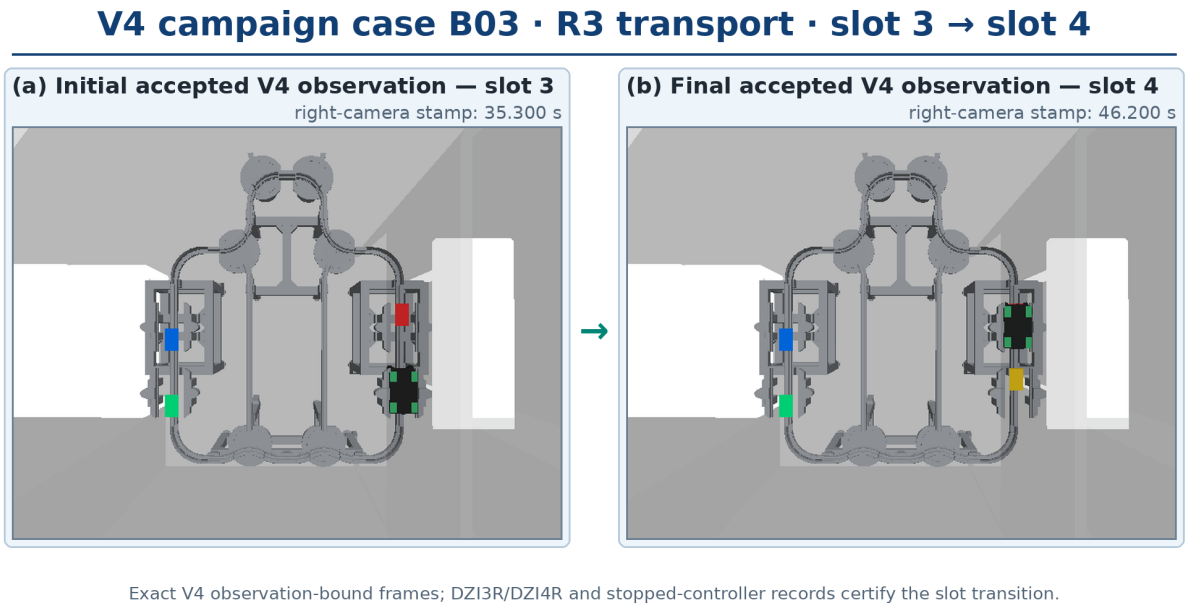



**Figure 11.1:** Exact right-camera frames selected by the initial and final accepted observations in the B03 ROS bag. The frames show the scene change; identity-bearing DZI3R/DZI4R and controller records establish R3's arrival at slot 4 and the final `DISABLED` controller state.

Figure 11.2 gives a compact view of the campaign outcomes and coverage.

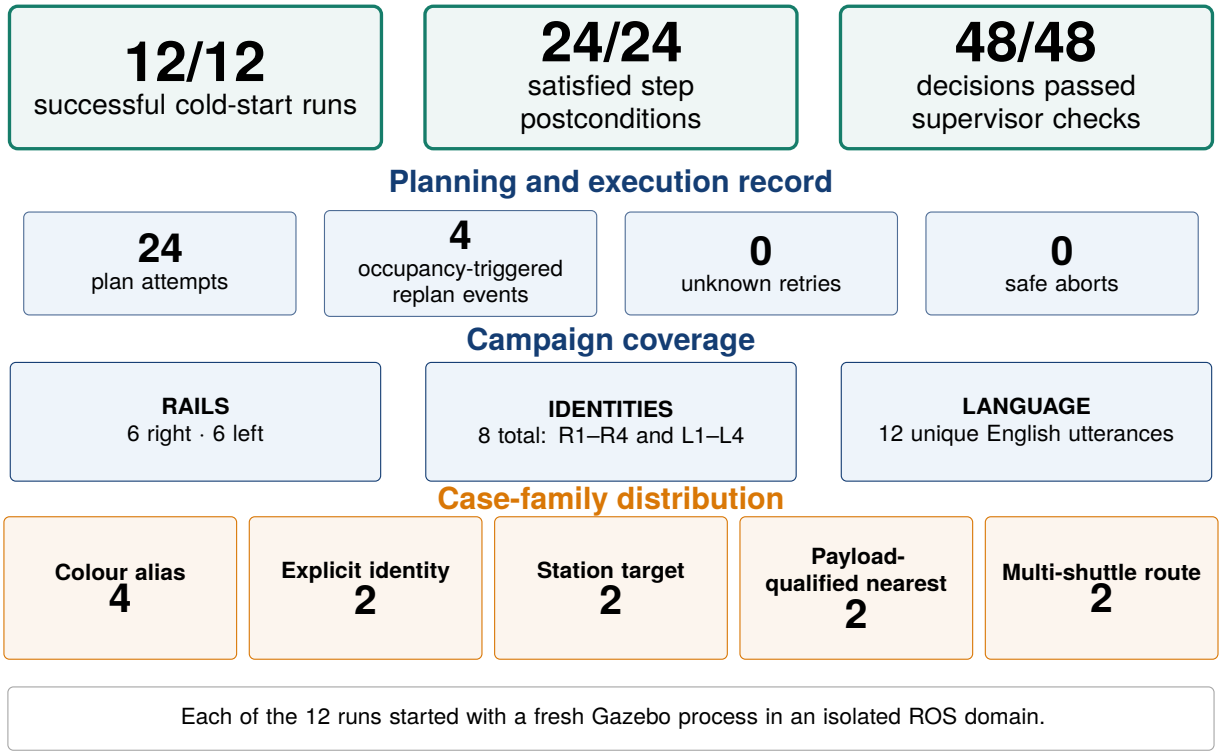



**Figure 11.2:** Outcome counts, planning record and planned coverage of the completed closed-loop campaign. The five case-family counts are mutually exclusive.

## 11.2 Sealed Final Test and runtime qualification

The rail-isolated output contract addresses the camera asymmetry directly. Its fixed slot order assigns L1–L4 to the left image branch and R1–R4 to the right image branch; there is no learned rail-side output. For every fixed identity the learned outputs are the 14 public-segment logits, two loaded-state logits, four bounding-box values and one normalised longitudinal coordinate. Thus the network emits 168 learned values, while side, segment length and metric position are derived from the identity and fingerprinted topology contract. The 200-value compatibility adapter is diagnostic only and is explicitly ineligible as a control input.

### 11.2.1 One-shot sealed Final Test

Epoch 11 was selected using Validation only. The checkpoint has SHA-256 `869d64049b0092c37d21a4c8b910dc6b91954527e0e49c5694fa82dce570f40d`. The declared Final Test combines 1024 base scenes with 16 stress scenes required by the frozen coverage criteria, giving 1040 scenes and 2080 images. Its control envelopes use the evaluator's canonical schemas; the captured data were fixed before inference, and no predictions were inspected while checking coverage.

The global one-shot reservation was created before the evaluator loaded rows, labels or images for inference. The completed attempt then evaluated the unchanged checkpoint once on 4680 visible-shuttle targets. It performed no training, checkpoint selection, threshold selection or calibration refit, and accessed neither the development Canary nor Test data outside the declared partition. The result passed all 80/80 frozen gates: 76 base gates and four preregistered Final Test runtime-threshold gates. The relative evidence package is `report/evidence/room315_visual_v4_submission_2026-08-11/final_test`.

All 28 supported rail–segment cells had observations and no supported segment had zero recall. The worst cell was left–A14, with 94.6429% recall on 56 targets. Boundary-zone segment accuracy was 99.1304% on 230 targets and the switch-zone result was 100% on 134 targets. These stratified results are more informative for planning than a global average alone.

**Table 11.2:** Principal metrics from the one-shot sealed Final Test.

| Metric | Global | Left rail | Right rail |
|---|---|---|---|
| Public-segment top-one accuracy | 99.9359% | 99.8726% | 100.0000% |
| Public-segment top-two accuracy | 100.0000% | 100.0000% | 100.0000% |
| Supported-segment macro F1 | 99.9117% | 99.7460% | 100.0000% |
| Loaded-state accuracy | 99.9145% | 99.9151% | 99.9140% |
| Joint segment and $\lvert\Delta s_{ratio}\rvert \leq 0.05$ | 90.8974% | 89.8089% | 92.0000% |
| Joint segment and $\lvert\Delta s_{ratio}\rvert \leq 0.12$ | 99.0385% | 98.9384% | 99.1398% |
| Correct-segment $s_m$ MAE | 0.01573 m | 0.01552 m | 0.01594 m |
| Correct-segment $s_m$ 95th percentile | 0.05850 m | 0.05660 m | 0.06030 m |
| Mean bounding-box IoU | 0.83400 | 0.83103 | 0.83701 |

At the frozen segment-confidence threshold, 4678/4680 targets were retained: 99.9573% coverage and 99.9572% selective segment accuracy, so two targets abstained. Loaded-state coverage was 4680/4680 with 99.9145% selective accuracy; joint confidence coverage was 4678/4680. In the camera counterfactuals, blanking or shuffling the opposite camera changed the own-side outputs by at most zero at recorded precision. Swapping camera order reduced segment top-one accuracy from 99.9359% to 7.7991%, a 92.1368-point drop, and reduced the joint 0.05 result from 90.8974% to 0.8120%. This supports the fixed-camera-order and rail-isolation contracts rather than a claim of camera interchangeability.

The sealed result is the principal visual-model result in this report. Its Final Test contains generated Gazebo scenes with fixed simulated cameras and the known Room 315 fleet. It is not physical-camera evidence, a control trial or authorisation for planner mutation or actuation.

### 11.2.2 Development evidence

The selected checkpoint passed 76/76 Validation gates and 74/74 gates on the exposed development Canary. Validation supplied checkpoint selection and segment calibration; the Canary served only as a regression check. Neither is substituted for the sealed Final Test in the main claim. Table 11.3 preserves their development metrics.

**Table 11.3:** Validation and regression metrics supporting the sealed Final Test.

| Metric | Validation | Exposed development Canary |
|---|---|---|
| Public-segment top-one accuracy | 99.9113% | 100.0000% |
| Public-segment macro F1 | 99.8799% | 100.0000% |
| Worst side–segment-cell recall | 97.6190% | 100.0000% |
| Loaded-state accuracy | 99.8669% | 100.0000% |
| Joint segment and $\lvert\Delta s_{ratio}\rvert \leq 0.05$ | 94.9867% | 94.5622% |
| Correct-segment $s_m$ MAE | 0.01867 m | 0.02015 m |
| Correct-segment $s_m$ 95th percentile | 0.06483 m | 0.07002 m |
| Mean bounding-box IoU | 0.83427 | 0.83466 |
| Acceptance gates | 76/76 | 74/74 |

The calibrated segment-confidence threshold was fitted on Validation only. Loaded confidence remains an uncalibrated softmax decision value, and the $s_{ratio}$ tolerance is an acceptance rule rather than a probability claim. The runtime therefore rejects a complete frame when any identity declared present fails its required input, confidence or contract check; identities declared absent contribute no accepted visual facts.

### 11.2.3 Observation-only qualification and rollback evidence

The selected checkpoint was evaluated without being connected to the command path. In the same-frame dense-scene shadow run, 55 observations were processed and 52 were accepted, giving 94.5455% coverage. The run recorded no processing errors and no identity on the wrong rail. It owned no command publisher, made no PlanSys2 mutation and recorded no actuation command.

Seven separately launched Gazebo scenes then supplied explicit ground truth that was excluded from model input. All seven records were quantitatively complete. Across them, all 53 accepted reobservations matched side, public segment and loaded state exactly and met the planning tolerance $|\Delta s_{ratio}| \leq 0.12$. The tighter 0.05 threshold was diagnostic: 150/169 identity checks passed it. The denominator is larger because each accepted frame can contain several present identities.

The fail-closed campaign rejected all eight injected faults: wrong manifest, checkpoint and topology hashes; reversed camera order; stale left image; stale right image; stale presence; and an unknown identity. The separately allowlisted rollback path also passed its ten-check smoke test, including strict checkpoint loading and an accepted dry-run observation under its exact schema and hash.

The observation-only qualification manifest has SHA-256 `8cb1674fe50bc8fbd372ade36fcb05d188f519ab36ea6d22dbb23ecfac3286c3`. Its authority is limited to a Gazebo runtime dry-run: state fusion remained dry-run, PlanSys2 updates remained disabled and no task-execution node or actuation publisher was authorised. A dense-scene smoke run produced 10/10 accepted ground-truth-matching reobservations with zero PlanSys2 mutations and zero actuation commands. These qualification records contain no control action and do not claim a physical deployment.

**Table 11.4:** Observation-only qualification evidence and authority boundary.

| Check | Recorded result | Control boundary |
|---|---|---|
| Same-frame shadow | 55 paired frames; 52 accepted (94.5455%); no errors or wrong-side identities | Isolated shadow topics; zero PlanSys2 mutations and zero actuation commands |
| Seven-scene acceptance | 7/7 complete; 53/53 matching reobservations at the 0.12 planning tolerance; 150/169 identity checks at 0.05 | Observation only; scenario ground truth excluded from model input |
| Visual-input fault injection | 8/8 specified faults rejected fail closed | No plan client, PlanSys2 update or actuation command |
| Rollback smoke | 10/10 checks passed for the separately allowlisted schema and checkpoint | Dry-run observation only |
| Dense-scene dry-run | Immutable manifest review passed; 10/10 matching reobservations | Gazebo dry-run only; PlanSys2 and task execution disabled; zero actuation |

### 11.2.4 Closed-loop fault qualification and Gazebo runtime authority

An independent immutable campaign exercised five faults in fresh Gazebo processes and isolated ROS domains. F01 and F02 supplied a corrupt authorisation and an invalid runtime manifest, respectively; the task gateway refused both before motion. F03 removed the planner service. F04 injected an emergency stop during a deliberate long move, and F05 removed the right-sensor feedback path during a long move without substituting an emergency stop. None of the five scenarios reported a false success, all final controller modes were `DISABLED`, and the bags contained 424 schema-compliant observations. The campaign therefore passed 5/5 declared scenarios and records `physical_deployment=false`.

The read-only fault `summary.json` has SHA-256 `330a207603425ce46b72c689c78cbe92aa804570c2a9e35caab8e51d6a8d8fd7`; its evidence manifest has SHA-256 `71c6286f021d4dce993d40603df5714506f0732d0739a937f1ab0cfce7ee7d55`, and the source `SHA256SUMS` has SHA-256 `be40c331b30c7375886b42a576ec2ac97e21c0bcd940140a77a2c170d102ecd8`. Manual runtime authorisation binds the exact qualification manifest, the

positive campaign summary and this complete fault bundle. The authority manifest has SHA-256 `506cae0511cf1675fdd666103ce7fc0b5980eb5e68d4cbadf0af99d9ee9560da`; the runtime-state record has SHA-256 `14cedafe28c999786a66934a523db5757e1ccdd7ae34705d5a2df58488fc8df1`, and the bundle `SHA256SUMS` has SHA-256 `5fae4bc7430606bc474dbbc78c9f89de29b3747d1425054e806b6fae62783f55`. The resulting state is `active_closed_loop_runtime`, with scope `gazebo_v4_closed_loop_runtime_only`, manual approval, automatic activation disabled and no physical-deployment approval. Its runtime guards explicitly record `actuation_enabled=true` for the separately authorised task gateway.

The active profile deliberately keeps the inference node's `dry_run_state_fusion` guard true and `plansys2_update_enabled` false. These node-local guards prevent a redundant ProblemExpert predicate mirror; they do not make the full runtime observation-only. The separately authorised task gateway constructs the PDDL problem from accepted observations and owns the Gazebo actuation path. A B01 partial runtime smoke reverified that exact manifest: 1/1 selected case passed with one actuating step and postcondition, two accepted supervisor decisions and 88 schema-compliant observations; terminal state, final effect and controller stop all passed. Its single-case scope does not replace the preceding 12/12 campaign, and it records `physical_deployment=false`; its summary has SHA-256 `15d5f9cf9503f414c0baa3a4ff84932a717aab3add2f3334e88577b4fc70c576`. Across the positive, fault and runtime-smoke bags, every observation matched the active schema and checkpoint allowlist.

## 11.3 Automated software checks

Before the complete runs, automated tests checked the software used by the workflow. Static and unit checks cover geometry, configuration and schema rules. Component and controlled-integration tests exercise providers, task construction, planning, translation, supervision and effect checking in controlled harnesses. They also include fault and protective cases. The integrated campaign then evaluates the normal-path components together in the Gazebo workflow.

On 7 August, three named suites were executed. The focused contract and executive suite passed 190/190 checks in 4.10 s, including the Torch-dependent visual-runtime case through the campaign's visual Python path. A transport suite passed 96/96 checks in 1.48 s, covering kinematic paths, topology, headway, reservations, fleet supervision, devices, identity and static shuttle clearance. The complementary operator-capability suite passed 42/42 checks in 0.41 s, covering the robot command helper, launch propagation, rail devices, marker/sensor lifecycle and switch-feedback rules. The suites overlap in a small number of files and are therefore reported separately rather than summed. Full commands and results are described in appendix A.

### 11.3.1 Language-interface evaluation

The nine non-control cases in the unchanged ten-case English corpus were each executed once through the configured local semantic gateway, strict envelope parser and field fusion; successfully parsed drafts were then passed to domain validation. The dialogue manager handled the cancellation case before model inference. All ten declared final outcomes matched: four supported goals produced the expected constraints, three incomplete or ambiguous requests required clarification, two inconsistent or invalid requests were rejected, and the cancellation command produced no task. All five incomplete, ambiguous, inconsistent or invalid requests ended in clarification or rejection.

The configured local backend was invoked exactly once in all nine non-control cases. Eight outputs passed the strict semantic-envelope schema without fallback. For the invalid-slot case, the model output itself contained slot 9; the schema rejected that envelope and the request remained an error. The cancellation command bypassed inference. This ten-utterance study evaluates the implemented Room 315 English contract. The retained case records and configuration are described in appendix A.

### 11.3.2 All-robot launch check

A separate cold-start check launched the full-floor profile with `robots:=all` in an isolated ROS domain and Gazebo partition. All six configured robot models were present; each exposed its namespaced joint command and feedback topics and produced a fresh `JointState` sample, while both TIAGo variants exposed independent base-velocity topics. The active log before planned shutdown was free of the declared error patterns. This check covers launch composition, namespace separation and feedback availability. Motion, simultaneous operation and repeatability require dedicated runs.

Figure 11.3 places these results alongside the other evaluation layers. Software tests study local behaviour and protective cases, the offline evaluations measure the language and visual interfaces, and the campaign follows the complete normal workflow. Full evaluation records are indexed in appendix A; operator tools and runbooks are listed in appendix B.

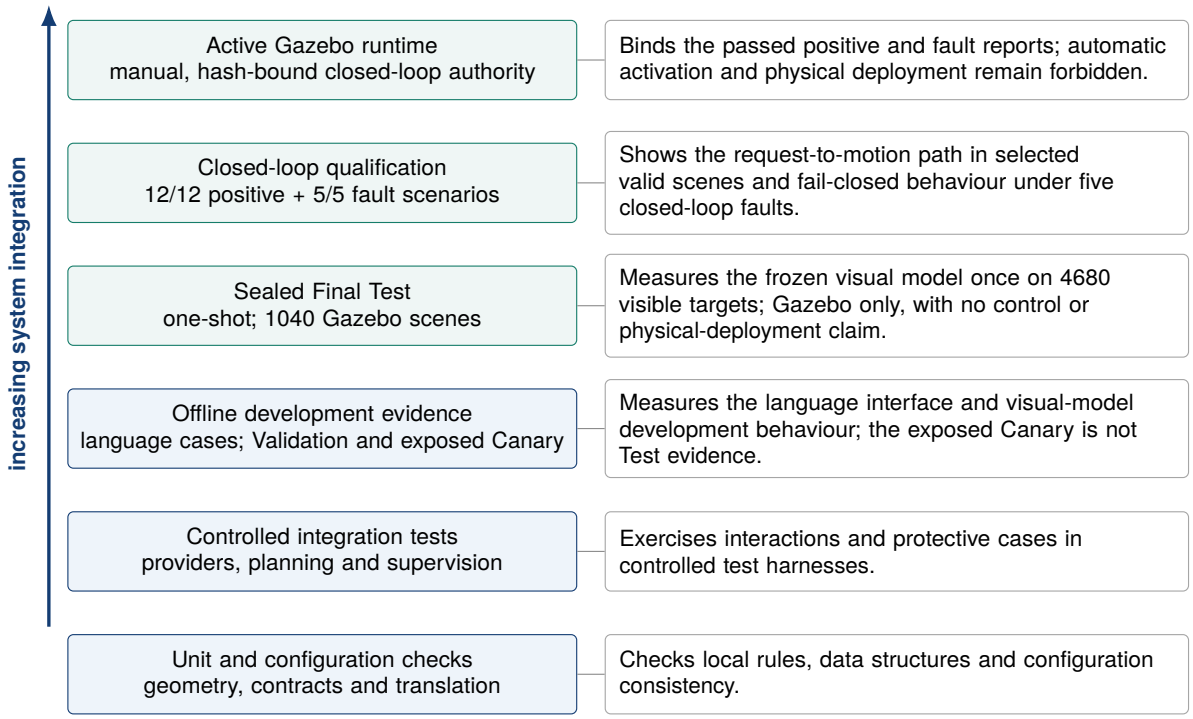



**Figure 11.3:** The sealed Final Test is the principal independent visual-model result. Development checks, observation-only qualification, the positive and fault campaigns, manual Gazebo runtime authority and software tests answer different questions. The Final Test and dry-run records contain no control action; the two closed-loop campaigns provide the bounded positive and fail-closed evidence for command authority.

## 11.4 Review of the internship objectives

Table 11.5 returns to the O1–O4 objectives introduced in chapter 1 and summarises how each one was assessed.

**Table 11.5:** Assessment of the four internship objectives.

| Objective | Main result | How it was evaluated |
|---|---|---|
| O1—simulation platform | Third-floor and Room 315 Gazebo models plus one all-robot cold-start check: six of six configured models, namespaced command/feedback topics and fresh state samples | One full-floor cold-start launch and interface check |
| O2—multi-shuttle transport | Calibrated Hermite rail geometry, explicit switch-dependent topology, eight stable shuttle identities, typed devices, reservations, headway checks and limited recovery on both rails; 96/96 named transport checks and route-clearance cases in the integrated campaign | Named software tests and declared simulated campaign scenes |
| O3—contracts and supervision | Versioned command, state and effect contracts; sparse device updates; freshness and consistency gates; supervisor rejection cases; and post-step effect checks covered by the 190/190 focused checks and the integrated runs | Named software tests and simulated campaign records |
| O4—neuro-symbolic workflow | Rail-isolated checkpoint with a passed one-shot sealed Final Test; closed-loop qualification with 12/12 positive Gazebo runs and 1784 observations; 5/5 closed-loop fault cases with 424 observations, no false success and disabled controllers; manually approved Gazebo runtime | Ten language cases, 1040 sealed Test scenes with 4680 visible targets, twelve positive campaign scenes, five fault scenes and observation-only qualification checks |

Chapter 12

# Limitations and Perspectives

## 12.1 Limitations

The project is not limited to the camera-based Room 315 workflow. Its first phase modelled and integrated the industrial and mobile robots, exposed namespaced joint or base commands and feedback, and enabled their motion in Gazebo. The retained all-robot check has a narrower evidential scope: it confirms one full-floor cold start, the presence of all six configured models, their isolated command and feedback interfaces, and fresh state samples; it does not measure simultaneous-motion accuracy or repeatability. What remains outside the implemented and evaluated scope is autonomous manipulation rather than arm motion itself: collision-aware arm-trajectory generation and a complete pick, grasp, handover or place workflow with object and contact verification were not developed.

The integrated language, perception and task-execution campaigns concern the Room 315 Gazebo workflow. The shuttle backend is kinematic, which supports controlled studies of topology, coordination and closed-loop task execution. It omits torque, slip, braking, mechanical play and payload inertia. The supervisor provides software checks for this simulated workflow; it is not a certified machinery-safety function.

The perception study uses the known shuttle fleet, fixed simulated cameras and generated scene families. The rail-isolated checkpoint was selected and calibrated on Validation; the exposed Canary remained a development-regression set. The principal result uses an independently generated, preregistered and one-shot Final Test, disjoint from the training, Validation and Canary sources. This strengthens the evidence for unseen generated Room 315 scenes, but the measurements still characterise the present camera geometry and synthetic distribution rather than unseen hardware, lighting or installation conditions. Identity presence comes from an independent registry and rail side is fixed by the L/R identity contract; these are system assumptions, not learned achievements. Segment confidence is Validation-calibrated, while the loaded-state softmax value is not a calibrated probability.

The twelve English-to-motion rows are command-producing results for selected valid Gazebo scenes; the Final Test is the separate independent perception result. One dense same-frame shadow run, seven observation-only scenes and one dense dry-run smoke provide qualification evidence without control action. The manual authorisation binds the positive and fault campaigns to the closed-loop Gazebo runtime. This authority does not extend to a physical deployment.

Each positive row, closed-loop fault scenario and all-robot launch check was executed once. The visual-input suite rejected 8/8 faults, the closed-loop campaign passed 5/5 faults without false success, and the separately allowlisted rollback smoke passed. These checks do not measure recovery during long live-graph outages or repeated switching between profiles. Repeating all runtime scenes across controlled seeds is still needed to quantify variation. The shuttle-arrival path uses command-linked observations. Applying the same correlation to switch and stopper reports is a remaining interface improvement. The implemented deadlock rule handles crossed reservations; longer wait-for cycles and temporal non-progress require further detection.

The documented CLI validates a task against the Room 315 domain and obtains operator confirmation at the configured risk levels. The execution subscriber assumes a trusted ROS graph and the official CLI producer. A multi-user simulation service would therefore need authenticated task sources or graph isolation, together with sender identity in each task.

The language study covers ten selected English requests, including clarification, rejection and cancellation. Open-vocabulary and multilingual evaluation remain future extensions.

## 12.2 Next evaluation steps

Table 12.1 orders the next simulation-focused tasks by the question each one would answer.

**Table 12.1:** Priority experiments for extending the evaluation.

| Priority | Engineering task | Purpose |
|---|---|---|
| P0 | Generate a domain-shifted evaluation partition | Measure transfer to varied camera poses, lighting, textures and sensor noise without tuning on the completed one-shot Final Test |
| P0 | Repeat the seven-scene review across controlled seeds | Measure same-frame acceptance, exact segment/payload agreement and longitudinal-error variation in sparse, dense and blocker arrangements |
| P0 | Inject and recover faults in the live ROS graph | Extend the completed eight-case fail-closed contract check with timed camera/presence loss, node restart, planner timeout and emergency-stop recovery |
| P0 | Repeat the positive and fault campaigns across controlled seeds | Quantify completion, abstention and recovery variation while preserving the exact schema, checkpoint, terminal-effect and controller-stop checks |
| P1 | Relate switch and stopper acknowledgements to commands | Correlate each device result with a matching state update newer than the command |
| P1 | Publish a reproducible evaluation release | Preserve the lightweight evidence index, source identity and hash-identified checkpoint/dataset objects needed to audit the one-shot Test and Gazebo reviews |

## 12.3 Continued use at MFJA

The package can support practical teaching sessions in which students launch a chosen robot set, inspect namespaces and typed interfaces, modify a rail scenario, generate a PDDL problem and follow one supervised task step to its observed result. Configuration-driven launch and cold-start campaign rows allow every session to begin from a declared state, while the runbooks provide the operational sequence and diagnostic checks.

The same assets provide a reference benchmark setup. An alternative planner, perception checkpoint or task-grounding method can be compared on the existing case matrix without changing controller behaviour or the arrival checks. The completed twelve-case campaign supplies the normal-workflow scenario set, while the seven acceptance scenes provide an observation-only perception matrix. Repeated model selection must use Validation; the exposed Canary should remain a regression record rather than become an optimisation target.

Further synthetic-data work can add camera perturbations, partial occlusion, station clutter and payload variation while preserving configuration-family grouping. These additions should form a separate domain-shift study; the completed Final Test must remain a frozen one-shot record and must not become an optimisation target.

## 12.4 Focused research extensions

Planner development should keep the PDDL domain aligned with the canonical rail graph and require an executable translation and visible effect check for every action. Any revision of the runtime or expansion of command authority must remain bound to its exact schema and checkpoint allowlist, presence-aware abstention, calibrated segment confidence and immutable operator authorisation. Independently tested atomic tasks can then be extended to controlled concurrency, with reservations and deadlock handling evaluated before any increase in parallelism.

## Chapter 13

# Conclusion

The four objectives were completed in simulation. Objectives O1 and O2 produced a maintainable ROS 2 Jazzy and Gazebo Harmonic environment with selective robot launch, calibrated rail cells, identifiable shuttles and switch-dependent routes. One all-robot cold start confirmed access to the six configured models, and all 96 transport checks passed across the geometry, topology, coordination, device and identity functions.

Objectives O3 and O4 extended this foundation with typed command and state interfaces, task planning, language interpretation and paired-camera perception. The command-producing integration campaigns and the independent perception evaluation have distinct scopes. The active schema separates the two rail branches, derives side from the fixed identity contract, adds confidence-bearing typed state and enforces exact schema/checkpoint allowlists with fail-closed input validation.

The language interface also produced the intended outcome in all ten evaluation cases, including clarification, rejection and cancellation.

The complete path from an English request to shuttle motion was exercised in 12 isolated cold-start runs. Every run reached its intended terminal state; all 24 recorded step postconditions and all 48 supervisor decisions passed. The executive made 24 planning attempts with four occupancy-triggered replans. The twelve bags contained 1784 schema-compliant observations, every final effect was verified and every controller stopped. This integration result is separate from the visual-model Final Test rather than a measurement of it.

Epoch 11 was selected and calibrated on Validation only, then frozen. Its one-shot Final Test comprised 1040 independent scenes and 4680 present-shuttle instances, with no overlap against training, Validation or Canary in the predeclared audit dimensions. Segment top-one accuracy was 99.9359%, segment macro F1 was 99.9117%, loaded-state accuracy was 99.9145%, and joint segment/longitudinal accuracy was 90.8974% at $|\Delta s_{ratio}| \leq 0.05$ (99.0385% at $0.12$). Correct-segment metric-position MAE was 0.01573 m, its 95th percentile was 0.05850 m, and mean bounding-box IoU was 0.8340. The worst supported rail–segment-cell recall was 94.6429%. All 80/80 frozen gates passed: 76 general acceptance gates and four Test-specific runtime-threshold gates. Opposite-camera blanking and shuffling caused exactly zero cross-rail output change, directly confirming the intended rail isolation; swapping camera order reduced segment top-one accuracy by 92.1368 percentage points and confirmed that order must be enforced.

Observation-only qualification accepted 52 of 55 paired shadow frames without processing errors or wrong-side identities; all seven acceptance scenes completed with 53/53 exact planning-tolerance re-observations; all eight injected faults were rejected fail closed; and the separately allowlisted rollback smoke passed. These records were dry-run only. The closed-loop fault campaign passed 5/5 scenarios with 424 schema-compliant observations, no false success and every controller disabled. The positive and fault reports are bound into a manually approved `active_closed_loop_runtime` profile with scope `gazebo_v4_closed_loop_runtime_only`. The inference node's dry-run fusion guard prevents direct ProblemExpert mutation, while the separately authorised task gateway owns the Gazebo planning and actuation path. Automatic activation and physical deployment remain forbidden. A B01 partial runtime smoke also passed with 88 schema-compliant observations, one verified actuating step and a stopped final controller; this single selected case confirms the authorised bundle but does not replace the 12/12 campaign.

For MFJA, the project provides a reusable platform for practical teaching, software integration, synthetic-data generation and further planning or perception experiments. For me, the most important lesson was learning how to divide an initially broad integration request into smaller

parts that I could implement and test one at a time. I also prepared the configuration, regression tests and run instructions so that the host team can continue using and extending the work.

The next steps are to test camera and appearance domain shift, repeat the positive and fault campaigns across controlled seeds, exercise timed restart and long-duration recovery, and develop a physical-deployment approval process. The reported command authority remains restricted to Gazebo; no physical deployment is claimed.

# Bibliography


[1] AIP-PRIMECA Occitanie. MFJA 3rd floor Gazebo simulation. https://github.com/aip-primeca-occitanie/mfja_3rd_floor_gz, 2026. Project repository, accessed 6 August 2026.

[2] Amanda Jane Coles, Andrew I. Coles, Maria Fox, and Derek Long. Forward-chaining partial-order planning. In *Proceedings of the Twentieth International Conference on Automated Planning and Scheduling*, pages 42–49, 2010. doi: 10.1609/icaps.v20i1.13403.

[3] Timothy Duggan, Pierrick Lorang, Hong Lu, and Matthias Scheutz. The price is not right: Neuro-symbolic methods outperform VLAs on structured long-horizon manipulation tasks with significantly lower energy consumption. In *2026 IEEE International Conference on Robotics and Automation (ICRA)*, 2026. URL https://hrilab.tufts.edu/publications/dugganetal26icra.pdf.

[4] Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++. Software repository, 2023. URL https://github.com/ggml-org/llama.cpp. Accessed 7 August 2026.

[5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.

[6] Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, et al. Do As I Can, Not As I Say: Grounding language in robotic affordances. In *Proceedings of the 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR, 2023. URL https://proceedings.mlr.press/v205/ichter23a.html.

[7] INSEE. NAF rev. 2, subclass 85.42Z: Higher education. https://www.insee.fr/fr/metadonnees/nafr2/sousClasse/85.42z, 2025. Accessed 4 August 2026.

[8] INSEE. Avis de situation au répertoire SIRENE: Université de Toulouse, SIRET 938 271 392 00012. https://api-avis-situation-sirene.insee.fr/identification/pdf/93827139200012, 2026. Official SIRENE record; accessed 11 August 2026.

[9] International Organization for Standardization. ISO 23247-1:2021 automation systems and integration—digital twin framework for manufacturing—part 1: Overview and general principles, October 2021. URL https://www.iso.org/standard/75066.html.

[10] Nishanth Kumar, William Shen, Fabio Ramos, Dieter Fox, Tomás Lozano-Pérez, Leslie Pack Kaelbling, and Caelan Reed Garrett. Open-world task and motion planning via vision-language model generated constraints. *IEEE Robotics and Automation Letters*, 11(3):3366–3373, 2026. doi: 10.1109/LRA.2026.3656799. URL https://arxiv.org/abs/2411.08253.

[11] Francisco Martín, Jonatan Ginés Clavero, Vicente Matellán, and Francisco J. Rodríguez. PlanSys2: A planning system framework for ROS2. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 9742–9749, 2021. doi: 10.1109/IROS51168.2021.9636544.

[12] Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. PDDL—the planning domain definition language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control, October 1998. URL https://ipc08.icaps-conference.org/deterministic/data/mcdermott-et-al-tr-1998.pdf. Version 1.2.

[13] Open Robotics. Getting started with Gazebo—Gazebo Harmonic documentation. https://gazebosim.org/docs/harmonic/getstarted/, 2026. Accessed 4 August 2026.

[14] Open Robotics. Use ROS 2 to interact with Gazebo. https://gazebosim.org/docs/harmonic/ros2_integration/, 2026. Accessed 4 August 2026.

[15] Open Robotics. Interfaces (topics, services, actions)—ROS 2 Jazzy documentation. https://docs.ros.org/en/jazzy/Concepts/Basic/Interfaces-Topics-Services-Actions.html, 2026. Accessed 5 August 2026.

[16] Open Robotics. Quality of service settings—ROS 2 Jazzy documentation. https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Quality-of-Service-Settings.html, 2026. Accessed 4 August 2026.

[17] Open Source Robotics Foundation. SDFormat 1.12 specification: Root and sensor elements. https://sdformat.org/spec/1.12/ and https://sdformat.org/spec/1.12/sensor/, 2026. Version 1.12; accessed 12 August 2026.

[18] Qwen Team. Qwen2.5-1.5B-Instruct-GGUF model card. Hugging Face model repository, 2024. URL https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF. Official model card documenting the GGUF distribution and Q4_K_M quantisation; accessed 11 August 2026.

[19] Université de Toulouse. La Maison de la Formation Jacqueline Auriol (MFJA). https://www.univ-tlse3.fr/lieux-de-ressources/la-maison-de-la-formation-jacqueline-auriol-mfja, 2026. Accessed 4 August 2026.

[20] Université de Toulouse. Schéma de promotion des achats publics socialement et écologiquement responsables 2026–2029. https://www.univ-tlse3.fr/medias/fichier/2026-03-ca-083_1773392372102-pdf, 2026. Institutional figures on p. 5; definitive SPASER adopted by the university council on 9 March 2026; document dated 13 March 2026; accessed 12 August 2026.

[21] Université d'Évry Paris-Saclay. Évaluation de la formation en entreprise (Module Stage M2): Le rapport. Internal university guidance, 2026. Official internship-report instructions, June 2026, p. 17, Section V: use of generative artificial intelligence.

[22] Université d'Évry Val d'Essonne and Université de Toulouse. Convention de stage 2025–2026: Développement d'une interface unifiée pour robots industriels et mobiles en simulation et réel, 2026. Signed internal document, p. 1: host Université de Toulouse, assigned service MFJA, academic supervisor Nicolas Seguy, host tutor Diane Bury, official subject and entrusted activities; 2 March–28 August 2026, 26 weeks and 868 hours of effective presence.

[23] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, Yaxin Peng, Feifei Feng, and Jian Tang. TinyVLA: Toward fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 10(4): 3988–3995, 2025. doi: 10.1109/LRA.2025.3544909. URL https://arxiv.org/abs/2409.12514.

[24] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*, 2024. doi: 10.48550/arXiv.2412.15115. URL https://arxiv.org/abs/2412.15115.

[25] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 2023. URL https://proceedings.mlr.press/v229/zitkovich23a.html.

# Chapter A

# Reproducibility Records

## A.1 Software environment and handover

The reproducibility package identifies the final implementation and the records used for its evaluation. The project source repository is identified by [1]; a reproducible handover must separately archive or commit the exact source, current configuration and report bundle used for this evaluation. The active evidence package retains byte-exact lightweight copies of the positive campaign, fault campaign, authorised runtime and B01 smoke records. The source manifests and checksum lists identify their external raw bundles without embedding the ROS bags in the report repository. Records excluded from the reported claims are listed separately.

The reference environment is Ubuntu 24.04, ROS 2 Jazzy, Gazebo Harmonic and Python 3.12. Manifests and hashes provide stable identifiers for generated data and model results. The report sources and lightweight evidence are stored under `report/` and are committed with the implementation and submission PDF.

The primary visual-model record is the repository-relative package `report/evidence/room315_visual_v4_submission_2026-08-11`. Its manifest binds the packaged files, the external checkpoint and sealed dataset, the two closed-loop campaigns, the runtime authorisation, its B01 smoke and the exact JSON Pointers used for the reported claims. The adjacent `SHA256SUMS` file permits an independent integrity check. Some byte-exact machine-generated copies retain archival paths from the evaluation host; the package explicitly marks those embedded paths as non-normative.

The workspace can be identified and built from its root with:

```
source /opt/ros/jazzy/setup.bash
git -C src/mfja_3rd_floor_gz rev-parse HEAD
colcon build --symlink-install --base-paths src/mfja_3rd_floor_gz
source install/setup.bash
```

**Source and report checks.** The current source identity and test inventory can be recorded from the repository root with:

```
git rev-parse HEAD
git status --short
python3 -m pytest --collect-only -q \
  mfja_robot_control_config/test \
  mfja_3rd_floor_description/test
```

A reproducible handover records this output and archives or commits the same source. Test collection confirms discovery, while the executed results are presented in chapter 11. From `report/`, `make check` builds the PDF, rejects unresolved references or citations and reports the page size and count. The final build log and delivery-PDF fingerprint are stored beside the active evidence index; keeping these identities outside the PDF avoids a self-referential document hash.

The repository revision carrying this report commits the implementation, active and recovery configurations, tests, submission PDF and lightweight evidence package together. A clean checkout of that revision therefore reconstructs the source-side handover; the checkpoint, sealed dataset and raw ROS bags remain external hash-identified objects because they are deliberately excluded from Git. The immutable evaluation records retain their original generation-time source and environment identities.

User-facing launch profiles and their current arguments are documented in `docs/INSTALLATION.md`, `docs/QUICK_START_AND_FEATURE_GUIDE.md` and `docs/ROOM315_LANGUAGE_TO_MOTION_RUNTIME.md`. The browser-based `runbook.html` supplies copy-ready operator workflows and diagnostic checks. Appendix B provides a concise index of the corresponding capabilities and operator interfaces.

The visual tools expose separate commands for the observation-only shadow, seven-scene acceptance, eight visual-input fault checks and explicit rollback smoke. The closed-loop tools independently run the twelve positive cases and five live fault scenarios. Runtime authorisation consumes the expected hashes of both completed campaigns and creates a manually reviewed Gazebo-only bundle; it neither approves physical deployment nor launches the system. The active task configuration remains fail closed with `execution_enabled=false`, so an operator must explicitly opt in after the runtime evidence is reverified.

## A.2 Evaluation files and identifiers

Table A.1 associates each reported result with the files and identifiers needed to inspect it. All authored locations are repository- or release-relative. Package subpaths are relative to `report/evidence/room315_visual_v4_submission_2026-08-11` unless stated otherwise.

**Table A.1:** Files and identifiers for the reported results.

| Result group | Files and identifiers |
|---|---|
| Visual evidence package | The primary package is `report/evidence/room315_visual_v4_submission_2026-08-11`. Its final `evidence_manifest.json` has SHA-256 `840787b617e0f671628dfc7d8122ff559d76664506ff8f94ac31d043737da446`; the package-wide `SHA256SUMS` file has SHA-256 `dec74565a8ccfba57a32d83dfdd03236daeafeff226d5243594e328fa4adc5ab`; the package README has SHA-256 `33b3a44f2e5e12b34721abcd1b47af609db3b55557c94c6871fa9c6eeb602a3e`. The manifest records 67/67 successful byte-exact source-copy comparisons, 69 checksum entries and 87 claim-to-JSON-Pointer mappings. It excludes the checkpoint binary, Test images, rows, labels, predictions and logs while retaining their controlling fingerprints. |
| Selected visual model | Epoch 11 is pinned by checkpoint SHA-256 `869d64049b0092c37d21a4c8b910dc6b91954527e0e49c5694fa82dce570f40d`. The package subdirectory `model/` contains the effective configuration, input fingerprints, topology contract, final training report, Validation metrics, acceptance report and frozen segment calibration. Checkpoint selection and calibration used Validation only. The separately labelled `canary/` records an exposed development regression check; it is not the Final Test. |

| Result group | Files and identifiers |
|---|---|
| Sealed Final Test dataset | The external sealed dataset contains 1,040 scenes, 2,080 images and 4,680 visible shuttle slots, and has fingerprint SHA-256 `226f4b5889f7d8b66bccc87d3e8dd494f710c8f4b228fab4851d6da7448e2eef`. The controls under `final_test/final/controls/` include finalization SHA-256 `1a51589587ae954f614b0da44e7c5c4f7a0ceb8c2ad43314fac63d9e03ce5eee`, seal SHA-256 `ca6c1dcb6bb06c8ba1f6c9f861af44c032c4f43cee6ecd80e40182868686bbe9` and a disjointness audit with SHA-256 `cda625ae9f03211a61e88434c2f713d2f6a9f40c27e996f8e7da9d5f4da292c2`. All 12 declared overlap dimensions are zero and all 69/69 coverage checks pass. |
| Final Test one-shot contract | The frozen evaluation contract `final_test/final/controls/evaluation_contract.json` has SHA-256 `e04a227c589046ee5ebc561de51002a01f3b989effcd7f3d81ba9673ef3d8608`. The single reserved attempt has key `2e18d6124b24438243a93d207d0c245fc1b87e25d2c08d1edca4040637e90aad`. Its immutable completed ledger, `final_test/final/results/attempt.completed.json`, has SHA-256 `4068df2d7bad8592cb1fdbfac69d9b22661124241c19faba9e0d023a7f0df71a`; the final report has SHA-256 `f789bde50f49fa61412c8d407b95129f0e0e9bde13ad810d9999f91a7a87daea`. The record states `completed`, `passed` and no automatic runtime switch. |
| Final Test measurements | On 4,680 visible slots, segment top-1 accuracy is 99.936%, macro-F1 is 99.912% and loaded-state accuracy is 99.915%. Joint correct-segment and within-5%-of-segment accuracy is 90.897%; within 12% it is 99.038%. Conditional on the correct segment, physical-position MAE is 0.01573 m and p95 is 0.05850 m; mean bounding-box IoU is 0.8340. Left/right segment top-1 is 99.873%/100%. Blank and shuffled opposite-camera checks have zero maximum cross-side influence, whereas camera-order swapping reduces segment top-1 by 92.137 percentage points. Exact values and pointers are in `final_test/final/results/final_report.json` and `evidence_manifest.json`. |
| Final Test coverage provenance | A support-only pre-inference audit of the 1,024-scene base plan found three aggregate conditions below their frozen minimum of eight. The declared Test therefore contains 16 additional stress scenes and meets every support threshold. No prediction or metric was inspected while fixing coverage. The byte-exact records are under `final_test/provenance/` and `final_test/final/controls/`. |
| Observation-only runtime qualification | Package subdirectories `runtime/shadow/`, `runtime/campaign/`, `runtime/safety/` and `runtime/promotion/` retain the same-frame comparison, 7/7 Gazebo-scene campaign, 8/8 fail-closed fault cases, 10/10 explicit rollback checks and the manual dry-run decision. That authority was limited to Gazebo dry-run; PlanSys2 mutation, task execution, actuation and automatic switching remained disabled. |

| Result group | Files and identifiers |
|---|---|
| Positive closed-loop campaign | The byte-exact `runtime/closed_loop/positive/summary.json` has SHA-256 `1504f25742d135c32ea879651336ff22a64cdfb2e09870be81ea7fb68b279479`; its evidence manifest and source checksum list have SHA-256 values `15d6362980751dd3e2bdd7fdab2b5ccb70118f73e1bd2cb2cf71d87bc96fdece` and `8ec1b8a456c5f83034fce0d92a3dc067d9d271e02e66a4471bc51acd9dc0e9f8`. The campaign passed 12/12 cases, recorded 1784 observations under the selected schema and none under the alternate schema, verified 24/24 postconditions and 48 supervisor decisions, and left every controller stopped. |
| Closed-loop fault campaign | The byte-exact `runtime/closed_loop/fault/summary.json` has SHA-256 `330a207603425ce46b72c689c78cbe92aa804570c2a9e35caab8e51d6a8d8fd7`; its evidence manifest and source checksum list have SHA-256 values `71c6286f021d4dce993d40603df5714506f0732d0739a937f1ab0cfce7ee7d55` and `be40c331b30c7375886b42a576ec2ac97e21c0bcd940140a77a2c170d102ecd8`. F01–F05 passed 5/5 with 424 selected-schema observations, none under the alternate schema, no false success and every final controller disabled. |
| Gazebo runtime authorisation | The manifest under `runtime/closed_loop/active_runtime/` has SHA-256 `506cae0511cf1675fdd666103ce7fc0b5980eb5e68d4cbadf0af99d9ee9560da`; the state-record and source-checksum-list SHA-256 values are `14cedafe28c999786a66934a523db5757e1ccdd7ae34705d5a2df58488fc8df1` and `5fae4bc7430606bc474dbbc78c9f89de29b3747d1425054e806b6fae62783f55`. The state is `active_closed_loop_runtime` under scope `gazebo_v4_closed_loop_runtime_only`; manual review is approved, while automatic profile selection and physical deployment are not approved. |
| Authorised-runtime B01 smoke | The selected-case summary under `runtime/closed_loop/active_runtime/smoke_B01/` has SHA-256 `15d5f9cf9503f414c0baa3a4ff84932a717aab3add2f3334e88577b4fc70c576`; its evidence manifest and source checksum list have SHA-256 values `dd2b3e7a6b3cb07a14538287329a13d3607bec8fd483a9210eebbec21e3b6574` and `e215ad6391dade682b397614ddcf8ece5ba76653b9728c28b8302c25126ea28d`. The `partial_smoke` record passed B01 with 88 selected-schema observations and none under the alternate schema, one actuating step, one satisfied postcondition, two accepted supervisor decisions, a verified final effect and a stopped controller. It is one selected-case smoke, not a replacement for the 12/12 campaign, and records `physical_deployment=false`. |

| Result group | Files and identifiers |
|---|---|
| Runtime defaults | The current files `mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml` and `task_execution_runtime.yaml` have SHA-256 values `22f12a9f96b3d54e0ab3d0bc05c202024ac6912cb50dd6e29ceb4a0a564d24f8` and `08eaedd7d6feed3dd1268ef18bfa2545348f203f1ff0dad3c2e9fb1a9f25b6ca`. The latter selects the exact runtime authorisation but defaults to `execution_enabled=false`. |
| Archived campaign source snapshot | Git HEAD `c44c501640e3dab35ab241384c99963e1b9d8745`, the stored binary worktree patch, and the 483-file source manifest with aggregate SHA-256 `326ccd537a65c43bca5780610d4fb9a0d7bd95791b047f3a1e174fc481f889f9`. The two untracked files named by the source record are retained under `report/evidence/current_source_supplement_2026-08-07`. |
| Configured language runtime | `Qwen2.5-1.5B-Instruct-GGUF`, `qwen2.5-1.5b-instruct-q4_k_m.gguf`, SHA-256 `6a1a2eb6d15622bf3c96857206351ba97e1af16c30d7a74ee38970e434e9407e`. The campaign used `task_goal_understanding.local.yaml`, SHA-256 `6d8c8658aa03273c45d738f8fd7437bf516713ebca47cb508fc1b999d07d35ea`; it pins the offline `llama.cpp` parameters and prompt-schema version. |
| Language-contract evaluation | `report/evidence/room315_language_semantic_evaluation_v3` retains the unchanged ten-case corpus, semantic configuration, runner, source snapshot, raw backend outputs, strict envelopes, fusion traces and environment. The summary, manifest and checksum-list SHA-256 values are respectively `38d015617856e3127da5a7082701caf5ab40523c6271cc7830d521800c8ecffd`, `d6584fb276f12630c60bb2d3860988d06d0c63d2a0c2e2931f38aaf33c0574de` and `f3e4260f6514f35486783c22b74b932de264ad7af9b18c85e6868fdea859211b`. The final protocol records 10/10 declared outcomes, nine model invocations, eight strict envelopes without fallback, one expected schema rejection and no unsafe automatic resolution. |

| Result group | Files and identifiers |
|---|---|
| Explicit rollback profile | Checkpoint SHA-256 `8a2d865e3d3551ec4284b53aa913d66f24640e23556f2f26b49a165f3ce8d51d`, bound to the preserved `training_config.json`, `metrics.json`, `run_metadata.json`, target/vectoriser sidecars and runtime preflight. Configuration files `visual_state_runtime_v3_rollback.yaml` and `task_execution_runtime_v3_rollback.yaml` have SHA-256 values `a61b72dacbb928b3170d2439c744ff0f2e2ccb227a83e70aab84e4cbdac26cd4` and `8c57c69448cb2c649ff972ef4ec327d7af32458c0c777048f6a25f3e0ad955d7`. The evidence root `report/evidence/visual_model_evaluation_2026-07-30` has checksum-list SHA-256 `912438606d432c8acc48bbf20aa624c65c29ea51ddef03142d75d2259c13c3b9`. Its preserved `split_manifest.json` (SHA-256 `8516f38ffbfdfb5a70cab17bb480e31f4c1f2b0d4c360b9d91a0bf10c122ca93`) records 1528 training, 256 validation and 256 locked-Test scenarios. The leakage audit (SHA-256 `205e1d8f47109aa17b936190b86c66d3e7c6b0fbf49116c9477f75d1eeb97a03`) and final review (SHA-256 `aa21e1991a1ffac6070231cae25a880528bf4bc6e19b27f39e450483c1f9e7e0`) retain the split and Test-evaluation checks. |
| Recorded 7 August verification | Three logs retain each command and result from the recorded source snapshot: `pytest_current_source_focused_2026-08-07.log`, 190/190, SHA-256 `cf4804917505d61465559924b3c776dbc7e2c3418dfeec16bf8bc2de11a17110`; `pytest_current_source_transport_2026-08-07.log`, 96/96, SHA-256 `9bb7bf0448f323c87b16b9bb0eaca4fef74b6bfdf53473af084aaedf45ba1404`; and `pytest_current_source_runbook_capabilities_2026-08-07.log`, 42/42, SHA-256 `fbd5891ec92e20a6b3a43528bdecfc0ff6f4554253ba34bbb181a6910ea6347c`. Their coverage is described in chapter 11. |
| Multi-robot cold start | `report/evidence/multi_robot_cold_start_smoke_2026-08-07` retains the predeclared protocol, active and complete logs, ROS graph, model list and six fresh state samples. `result.json` has SHA-256 `59b60b1fddf37b74b86013dce195438259f41933db335aa0ccff85e83e014fae`; the checksum-list SHA-256 is `5c3d60f5d6324eafb989b577369b50f686e8c8d62d37a8bf27008cfff384d8c0`. |
| B03 camera figure | The exact frames, extraction script and provenance are retained under `report/evidence/room315_v4_closed_loop_campaign_figures_2026-08-12`. The composite figure SHA-256 is `3afef390900901598926f0d11226bb460a70b818ce0c4c765594aefb6fd614dc`; the derivative checksum-list SHA-256 is `9914684e2333a5ccc0053c0521f4bbdade217374c253875c31475fcaa20af39c`. The exact image used in figure 11.1 is bound to this record. |

The repository-relative package preserves 67 byte-exact payloads and 87 claim-to-JSON-Pointer mappings. Its copied source manifests and source checksum lists identify the deliberately excluded ROS bags and other large objects. The release archive and current-source supplement remain available to reconstruct that recorded source state, but they do not substitute for the selected-model records.

The evidence layers remain deliberately separated: Validation-only checkpoint selection, the exposed development Canary, the sealed one-shot Final Test, observation-only checks, the 12-case positive campaign, the five-case fault campaign, manual runtime authorisation and its one-case authorised-runtime smoke. The inference node retains its dry-run state-fusion guard, while the separately authorised task gateway owns the Gazebo actuation path. Every current runtime claim remains limited to Gazebo; automatic profile switching and physical deployment are not approved. Rollback material is retained only to reconstruct and verify that explicit fallback path.

## Chapter B

# Capability and Interface Reference

This appendix provides a concise reference to the current operator tools and public interfaces. Full operating procedures remain in `runbook.html`, while chapter 11 reports the configurations exercised in the experiments.

## B.1 Operator capability index

**Table B.1:** Current capabilities represented in the operator runbook.

| Runbook area | Available interface or profile | Use |
|---|---|---|
| Room 315 and world profiles | Focused Room 315; full third floor with `robots:=none`, an exact/comma-separated robot selection or `all`; and an isolated fixed-base robot with its table | Launch compositions for exercising integration boundaries separately or together |
| Runtime presentation | GUI or headless execution and paused/running start; light/full GUI choices on the Room 315 and full-floor paths; optional device markers | Inspection and resource-control options; markers support debugging and calibration and can be hidden during image capture |
| Visual-runtime review | The rail-isolated visual contract is selected through profiles that pin the exact manifest, schema and checkpoint | A visual-profile selection authorises observations only. Planner or command use requires its own immutable runtime authorisation, and no profile authorises physical deployment |
| Robot inventory and control | Four fixed-base arms, full TIAGo and base-only TIAGo from the current full-floor registry; namespaced joint command/state surfaces and independent TIAGo base velocity | The helper checks profiles, vector length and units; collision-free trajectories require a separate planner |
| Dual-rail setup | Independent left/right enablement, shuttle count, start slots, stored speed setting and initial enabled state | Current low-level simulator configuration. Supervised tasks retain the fixed L1–L4/R1–R4 identity domain |
| Shuttle lifecycle | Typed add service; per-entity ON, OFF, RESET and REMOVE; rail-wide `ALL` ON, OFF and RESET; continuous state feedback | Additional names support commissioning; supervised tasks use the fixed L1–L4/R1–R4 identity set |
| Devices and sensors | Per-rail switch and stopper command/state topics plus continuous aggregate binary readings for station, branch and stopper-linked detectors | Raw command publication for low-level diagnosis; closed-loop tasks reach the same devices through translation and supervision |
| Geometry maintenance | Guarded dry-run projection from Gazebo XYZ to a rail segment and `s_ratio`, with YAML updates available for slots, ordinary sensors and single-/multi-point stoppers | Offline maintenance utility. A configuration write requires relaunch and subsequent marker/feedback checking |
| Runtime diagnosis | ROS graph, topic, service and interface inspection together with build/source procedures | Support for reproducibility and troubleshooting |

## B.2 Public rail interfaces

The `mfja_rail_interfaces` package is independent of Gazebo-specific controllers, allowing simulator nodes, tests and alternative actuation providers to share public types.

**Table B.2:** Condensed rail-interface reference.

| Interface | Principal semantics |
|---|---|
| `NamedState` | Canonical device name and symbolic state, embedded in switch/stopper arrays |
| Switch command/state | Partial named command and reported switch state |
| Stopper command/state | Partial named command and reported stopper state |
| `ShuttleCommand` | Named `ON/OFF/RESET/REMOVE`, start slot, stored speed setting and optional target slot |
| `ShuttleState` | Identity, mode, controller segment/arc length, pose, stored speed and reached target; these fields support runtime checks |
| `SensorFeedback` | Binary occupancy readings with detected identity, segment and calibrated coordinates |
| `VisualShuttleState` | Fused identity/presence validity, derived side, public segment, box, $s_m$, ratio, segment length, loaded state and the two decision confidences |
| `VisualStateObservation` | Image stamps, exact schema/checkpoint identity, readiness, validation reasons, provenance and eight fixed-identity shuttle records |
| `AddShuttle` service | Requested identity/start state and explicit acceptance result |
| `JointTrajectory` `JointState` | Namespaced robot joint command and feedback; the operator helper validates profile, vector length and angular unit conversion |
| `Twist`/odometry/TF | Independent TIAGo mobile-base command and feedback surfaces for the full and base-only variants |

Three fields require careful interpretation. `ShuttleState.speed` stores a speed setting rather than instantaneous velocity. `reached_target_slot` indicates low-level setpoint completion; the full task effect requires additional evidence. `VisualStateObservation.accepted` indicates that the structural, freshness, presence and required-confidence checks passed for the complete frame, while individual learned values remain estimates. In this contract, `side` is derived from the fixed L/R identity prefix and is not predicted. `segment_confidence` uses Validation-only temperature calibration; `loaded_confidence` is an uncalibrated softmax decision value and must not be interpreted as a probability guarantee. An absent identity has no accepted visual facts.

## B.3 Canonical assets

**Table B.3:** Canonical Room 315 mapping.

| Rail | Shuttles | Station slots |
|---|---|---|
| Left | L1–L4 | 1–2 Yaskawa HC10; 3–4 KUKA KR6 |
| Right | R1–R4 | 1–2 Yaskawa HC10DT; 3–4 Stäubli TX2 |

Rail geometry, public segment names, switch assignments and branch-capacity rules are defined and justified in chapter 7. Runtime topic strings remain configurable; the installed configuration and live ROS graph are the current operational references.

## B.4 Runtime responsibility summary

**Table B.4:** Responsibilities in the Room 315 task workflow.

| Workflow decision | Responsible component |
|---|---|
| Interpret an English request | The local model proposes semantic fields; explicit parsing, field fusion, domain validation and dialogue produce the confirmed `TaskGoal` |
| Estimate visible shuttle state | The paired-camera model estimates public segment, longitudinal location, box and payload state for identities supplied by the configured presence registry; the fixed identity contract supplies rail side |
| Select a visual input contract | The task runtime requires an exact visual schema and checkpoint SHA-256 allowlist. An immutable authorisation manifest and operator review can select a software profile, but do not themselves enable planning or actuation |
| Create the symbolic plan | PlanSys2/POPF solves the current PDDL problem in a separately command-authorised Gazebo profile. Selecting the visual contract does not by itself grant planner mutation or actuation |
| Select the next operation | The project executive takes the first supported plan step and observes the result before replanning |
| Translate the operation | The plan translator creates the sparse named command for the addressed device or shuttle |
| Publish the command | The software supervisor checks the current state and publishes the corresponding typed ROS 2 message |
| Apply and confirm the result | The kinematic controller applies the simulated transition; the executive compares device and sensor reports with the expected effect |

Canonical source roots and artefact identities are listed in appendix A.